



universidad
de león

Escuela de Ingenierías



Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

FinSight: diseño e implementación de una plataforma
web de análisis financiero de acciones mediante
aprendizaje profundo, GraphRAG y grandes modelos
de lenguaje

FinSight: design and implementation of a web
platform for financial stock analysis using deep
learning, GraphRAG, and large language models

Autor: Javier Miñambres Calvo

Tutor: Enrique López González

(Julio, 2026)

UNIVERSIDAD DE LEÓN Escuela de Ingenierías Industrial, Informática y Aeroespacial GRADO EN INGENIERÍA INFORMÁTICA Trabajo de Fin de Grado
ALUMNO: Javier Miñambres Calvo
TUTOR: Enrique López González
TÍTULO: FinSight: diseño e implementación de una plataforma web de análisis financiero de acciones mediante aprendizaje profundo, GraphRAG y grandes modelos de lenguaje
CONVOCATORIA: Julio, 2026
RESUMEN: Este Trabajo de Fin de Grado presenta el diseño y la implementación de FinSight, una aplicación web que acerca el análisis financiero de acciones a usuarios sin formación especializada, ante la dificultad de interpretar información bursátil dispersa en numerosas fuentes. El sistema combina cuatro análisis y los sintetiza en un informe en español generado por un modelo de lenguaje, con un aviso legal permanente sobre su carácter informativo y no asesor. El análisis de sentimiento se apoya en una arquitectura GraphRAG sobre un grafo de conocimiento en Neo4j; la predicción de tendencia a diez sesiones emplea redes neuronales recurrentes GRU entrenadas por acción; el análisis fundamental convierte las métricas de la empresa a lenguaje natural, y el técnico deriva una puntuación de los indicadores de precio y volumen. La solución es un monolito modular con frontend en React y backend en FastAPI, con un servicio LLM centralizado que alterna entre OpenAI y la opción de usar un modelo local con Ollama, y un reentrenamiento diario automático mediante APScheduler. El diseño cumple los criterios de accesibilidad WCAG 2.1 AA, a la seguridad y al cumplimiento normativo (RGPD, LSSI-CE, MiFID II y el Reglamento de Inteligencia Artificial). El resultado es una plataforma gratuita que integra cuatro perspectivas en un informe en lenguaje no técnico.
Palabras clave: Análisis de sentimiento, GraphRAG, aprendizaje profundo, LLM, GRU, análisis fundamental, análisis técnico, finanzas, business intelligence.

UNIVERSIDAD DE LEÓN Escuela de Ingenierías Industrial, Informática y Aeroespacial GRADO EN INGENIERÍA INFORMÁTICA Trabajo de Fin de Grado
STUDENT: Javier Miñambres Calvo
TUTOR: Enrique López González
TITLE: FinSight: design and implementation of a web-based platform for financial analysis of shares using deep learning, GraphRAG and large language models
CALL: July, 2026
ABSTRACT: This project presents the design and implementation of FinSight, a web application that brings stock financial analysis closer to users without specialised training, addressing the difficulty of interpreting market information scattered across many sources. The system combines four analyses and synthesises them into a report in Spanish generated by a large language model, with a permanent legal notice on its informational, non-advisory nature. The sentiment analysis relies on a GraphRAG architecture over a knowledge graph in Neo4j; the ten-session trend prediction uses GRU recurrent neural networks trained per stock; the fundamental analysis translates a company’s metrics into natural language, and the technical one derives a signal from price and volume indicators. The solution is a modular monolith with a React frontend and a FastAPI backend, with a centralised LLM service that switches between OpenAI and the option to use a local model served with Ollama, and automatic daily retraining through APScheduler. The design addresses accessibility (WCAG 2.1 AA), security, and regulatory compliance (GDPR, LSSI-CE, MiFID II and the Artificial Intelligence Act). The result is a free platform that integrates four analytical perspectives into a single, understandable conclusion.
Keywords: Sentiment analysis, GraphRAG, deep learning, LLM, GRU, fundamental analysis, technical analysis, finance, business intelligence.

Índice general

Índice de figuras	V
Índice de tablas	VII
Glosario de términos	IX
Introducción	1
1. Estudio del problema	6
1.1. El contexto del problema	6
1.2. El estado de la cuestión	7
1.3. La definición del problema	8
2. Gestión de proyecto software	10
2.1. Alcance del proyecto	10
2.2. Definición del proyecto	11
2.2.1. Descripción y justificación	11
2.2.2. Objetivos	12
2.2.3. Equipo y partes interesadas	13
2.2.4. Metodología y entregables	13
2.3. Presupuesto	14
2.3.1. Coste de personal	14
2.3.2. Coste del hardware	14
2.3.3. Coste de servicios	15
2.3.4. Costes indirectos	16

2.3.5.	Coste total	16
2.4.	Plan de trabajo	16
2.4.1.	Identificación de tareas	17
2.4.2.	Estimación de tareas	18
2.4.3.	Planificación de tareas	20
2.5.	Gestión de recursos	21
2.5.1.	Especificación de recursos	21
2.5.2.	Asignación de recursos	22
2.6.	Gestión de riesgos	23
2.6.1.	Identificación de riesgos	24
2.6.2.	Análisis de riesgos	24
2.7.	Legislación y normativa	26
2.7.1.	Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico	26
2.7.2.	Protección de datos personales: RGPD y LOPDGDD	27
2.7.3.	Reglamento de Inteligencia Artificial (AI Act)	29
2.7.4.	Directiva sobre mercados de instrumentos financieros (MiFID II)	30
2.7.5.	Normativa de accesibilidad web	31
2.7.6.	Licencias de software de terceros	31
2.7.7.	Normativa de aplicación indirecta o condicionada al crecimiento del servicio	33
2.7.8.	Normativa no aplicable: justificación de exclusiones	34
2.7.9.	Resumen de disposiciones normativas	36
3.	Solución	39
3.1.	Descripción de la solución	39
3.2.	El proceso de desarrollo	42
3.2.1.	Análisis	43
3.2.2.	Diseño del sistema	50
3.2.3.	Dataset	51
3.2.4.	Grafo de conocimiento	62
3.2.5.	Servicio LLM centralizado	74

3.2.6. Modelo de procesamiento de series temporales	80
3.2.7. Implementación	89
3.2.8. Métricas	91
3.2.9. Pruebas	94
3.3. El producto del desarrollo	97
3.3.1. Página de inicio	98
3.3.2. El informe de análisis	98
3.3.3. Cobertura parcial y estados de error	101
3.3.4. Consentimiento, privacidad y aviso legal	103
3.3.5. Tema visual y páginas de servicio	103
4. Evaluación	108
4.1. Proceso de evaluación	108
4.1.1. Forma de evaluación	108
4.1.2. Casos de prueba	109
4.2. Análisis de resultados	109
Conclusión	112
Lista de referencias	117
A. Control de versiones	125
B. Seguimiento de proyecto fin de grado	132
B.1. Forma de seguimiento	132
B.2. Planificación inicial	133
B.3. Planificación final	134
C. Manual de usuario	135
C.1. Acceso y requisitos	135
C.2. Buscar un valor y generar el informe	136
C.3. Cómo leer el informe	136
C.4. Cuando falta algún análisis	137
C.5. Tema claro y oscuro	138

C.6. Privacidad y cookies	138
C.7. Preguntas frecuentes	138

Índice de figuras

2.1. Diagrama de Gantt	20
3.1. Esquema del sistema diseñado	50
3.2. Esquema del grafo de conocimiento	64
3.3. Subgrafo de ejemplo del grafo de conocimiento	72
3.4. Flujo de la recuperación GraphRAG	75
3.5. Patrón Adaptador aplicado al servicio LLM	76
3.6. Flujo de la red GRU de producción	88
3.7. Página de inicio	98
3.8. Autocompletado del buscador	99
3.9. Informe completo de un valor	100
3.10. Desplegable de ratios fundamentales	101
3.11. Informe con soporte parcial	102
3.12. Banner de consentimiento de cookies	103
3.13. Diálogo de preferencias de cookies	104
3.14. Página de inicio en tema oscuro	105
3.15. Página de política de privacidad	106
3.16. Página de términos de uso	107
3.17. Página de error para rutas inexistentes	107
A.1. Repositorio del proyecto previo	126
A.2. Historial de commits de FinSight	127
A.3. Pull request con la CI en verde	129
A.4. Ejecuciones de integración continua	130
A.5. Tablero Kanban del proyecto	131

B.1. Seguimiento de hitos en GitHub	133
B.2. Diagrama de Gantt inicial	134
B.3. Diagrama de Gantt final	134

Índice de tablas

1.1. Herramientas comerciales existentes	7
2.1. Elementos que quedan fuera del alcance del proyecto	11
2.2. Equipo del proyecto, responsabilidades y dedicación estimada	13
2.3. Presupuesto de personal	14
2.4. Costes de infraestructura	15
2.5. Presupuesto total	16
2.6. Iteraciones del proyecto y su correspondencia con los <i>milestones</i> de GitHub	19
2.7. Estimación del esfuerzo en horas por iteración y perfil	19
2.8. Hitos del proyecto, entregables, semana de cierre y dependencias	21
2.9. Recursos materiales del proyecto	22
2.10. Recursos de software y servicios	22
2.11. Asignación de recursos a tareas e iteraciones	23
2.12. Riesgos identificados en el proyecto	24
2.13. Análisis y mitigación de los riesgos	25
2.14. Resumen de disposiciones normativas aplicables a FinSight	36
3.1. Ejemplo del conjunto de datos FinEntity	53
3.2. Ejemplo del conjunto de datos FinMarBa	54
3.3. Distribución de etiquetas en Financial PhraseBank y FinMarBa	54
3.4. Muestra del fichero AAPL.csv	56
3.5. <i>tickers</i> usados en la fase de investigación	57
3.6. Composición del grafo de conocimiento por tipo de nodo y de relación .	70
3.7. Diez activos con más menciones en el grafo	71
3.8. RMSE por <i>ticker</i> del retorno a diez días	85

3.9. Resultados GRU ajustada	87
3.10. Reparto de la batería de pruebas por subproyecto y utillaje empleado .	95
4.1. Casos de prueba de aceptación	110
A.1. Reparto de commits por tipo	126
B.1. Planificación inicial por iteraciones	133

Glosario de términos

Catálogo de términos específicos del contexto del trabajo.

ADR : Architecture Decision Record. Documento breve que registra una decisión de diseño relevante, su contexto y sus consecuencias.

API : Application Programming Interface. Interfaz de programación que permite la comunicación entre sistemas software mediante definiciones y protocolos estandarizados.

APScheduler : Advanced Python Scheduler. Librería Python para programar la ejecución periódica y automática de tareas en segundo plano.

ARIMA : AutoRegressive Integrated Moving Average. Modelo estadístico clásico para el análisis y la predicción de series temporales.

BiLSTM : Bidirectional LSTM. Variante de la red LSTM que procesa la secuencia en ambos sentidos temporales.

CI/CD : Continuous Integration / Continuous Delivery. Prácticas de integración y entrega continuas que automatizan la verificación e incorporación de los cambios de código.

CNN : Convolutional Neural Network (Red Neuronal Convolucional). Arquitectura de red neuronal que extrae patrones locales mediante operaciones de convolución.

CORS : Cross-Origin Resource Sharing. Mecanismo de seguridad HTTP que controla qué dominios externos pueden acceder a los recursos de un servidor.

CPU : Central Processing Unit. Procesador de propósito general del ordenador; en producción la inferencia se ejecuta sobre CPU al no disponer de GPU.

CSRF : Cross-Site Request Forgery. Ataque que engaña al navegador para ejecutar peticiones no autorizadas en nombre de un usuario autenticado.

Deduplicar : Eliminar los elementos repetidos de un conjunto de datos para conservar una sola copia de cada uno. En la construcción del grafo de conocimiento, las noticias idénticas se identifican mediante un hash de su texto y se funden en un único nodo. También incluye fusionar dos entidades que hagan referencia al mismo activo nombrado de formas diferentes.

DL : Deep Learning (Aprendizaje Profundo). Subcampo del aprendizaje automático basado en redes neuronales con múltiples capas.

Embeddings : Representaciones vectoriales de un texto que capturan su significado y permiten medir la similitud semántica entre fragmentos.

EOD : End-of-Day. Datos de cierre de sesión bursátil: precio de apertura, máximo, mínimo, cierre y volumen al finalizar la jornada de mercado.

ERS / SRS : Especificación de Requisitos de Software / Software Requirements Specification. Documento que describe el comportamiento esperado de un sistema.

ETF : Exchange-Traded Fund (fondo cotizado). Fondo de inversión que se negocia en bolsa como si fuera una acción.

FastAPI : Framework web moderno de Python para construir APIs REST con tipado y documentación automática.

GARCH : Generalized Autoregressive Conditional Heteroskedasticity. Familia de modelos econométricos para modelar la volatilidad de las series financieras.

GPU : Graphics Processing Unit. Procesador especializado en cálculo paralelo, útil para entrenar redes neuronales; no está disponible en el entorno de producción gratuito.

GraphRAG : Graph Retrieval-Augmented Generation. Arquitectura que enriquece el contexto de un LLM mediante recuperación de información estructurada en una base de datos de grafos.

GRU : Gated Recurrent Unit. Red neuronal recurrente con mecanismo de puertas, más ligera que la LSTM, empleada en el módulo de predicción de tendencia.

HTML : HyperText Markup Language. Lenguaje de marcado con el que se estructura el contenido de las páginas web.

HTTP / HTTPS : HyperText Transfer Protocol (Secure). Protocolo de comunicación de la web; su variante segura cifra el tráfico mediante TLS.

IEEE 830 : Estándar IEEE Recommended Practice for Software Requirements Specifications (1998), que define el formato y contenido de una ERS.

JSON : JavaScript Object Notation. Formato ligero de intercambio de datos, usado en la comunicación entre el frontend y la API.

Kanban : Metodología de gestión visual del trabajo basada en un tablero de tarjetas, la limitación del trabajo en curso y la mejora continua.

LLM : Large Language Model (Modelo de Lenguaje Grande). Modelo de inteligencia artificial entrenado sobre grandes corpus de texto para generar y comprender lenguaje natural.

LRU Cache : Least Recently Used Cache. Política de caché que expulsa el elemento menos usado recientemente cuando se alcanza la capacidad máxima.

LSTM : Long Short-Term Memory. Tipo de red neuronal recurrente diseñada para capturar dependencias temporales de largo alcance, especialmente adecuada para series temporales.

MAE : Mean Absolute Error (Error Absoluto Medio). Métrica de error que promedia el valor absoluto de las diferencias entre la predicción y el valor real.

MiFID II : Markets in Financial Instruments Directive II. Directiva europea (2014/65/UE) sobre mercados de instrumentos financieros, que regula la información financiera dirigida a inversores.

Neo4j : Sistema de gestión de bases de datos orientado a grafos, utilizado para almacenar y consultar las noticias de referencia del módulo GraphRAG.

OHLC : Open-High-Low-Close. Conjunto de cuatro valores de precio (apertura, máximo, mínimo y cierre) que describen el comportamiento de un activo durante una sesión bursátil.

Ollama : Herramienta para ejecutar modelos de lenguaje de forma local, empleada como proveedor LLM durante el desarrollo.

Optuna : Librería de optimización de hiperparámetros que combina muestreo dirigido y poda temprana de las pruebas menos prometedoras.

RAG : Retrieval-Augmented Generation. Técnica que mejora la generación de texto de un LLM proporcionándole contexto recuperado dinámicamente de una fuente de datos.

REST : Representational State Transfer. Estilo arquitectural para diseño de APIs que opera sobre recursos HTTP identificados por URIs.

RGPD : Reglamento General de Protección de Datos. Reglamento UE 2016/679 que regula el tratamiento de datos personales en la Unión Europea.

RMSE : Root Mean Squared Error (Raíz del Error Cuadrático Medio). Métrica de error principal del módulo de predicción, expresada en las unidades del objetivo.

SPA : Single Page Application (Aplicación de Página Única). Aplicación web que se carga una sola vez y actualiza su contenido de forma dinámica sin recargar la página.

SSA : Sparrow Search Algorithm. Metaheurística de enjambre para la optimización de hiperparámetros.

TDD : Test-Driven Development (Desarrollo Dirigido por Pruebas). Práctica de desarrollo que consiste en escribir la prueba antes que la implementación.

TFG : Trabajo de Fin de Grado. Proyecto académico que el alumno realiza al finalizar el Grado universitario.

Ticker : Símbolo bursátil que identifica de forma abreviada a un valor cotizado (por ejemplo, AAPL para Apple).

Transformer : Arquitectura de red neuronal basada en mecanismos de atención, base de los modelos de lenguaje modernos.

TTL : Time To Live. Tiempo máximo durante el cual un dato almacenado en caché se considera válido antes de ser descartado o regenerado.

TTM : Trailing Twelve Months (últimos doce meses). Magnitud financiera calculada sobre los doce meses consecutivos más recientes, encadenando los últimos trimestres publicados, que ofrece una visión actualizada y anualizada al margen del cierre del ejercicio fiscal.

VMD : Variational Mode Decomposition. Algoritmo de descomposición de señales que separa una serie temporal en modos de distinta frecuencia. Se emplea como preprocesamiento en algunos enfoques híbridos de predicción; en este trabajo se evaluó y se descartó al no aportar ventaja con una descomposición causal.

WCAG : Web Content Accessibility Guidelines. Pautas de accesibilidad para el contenido web del W3C. El nivel AA de la versión 2.1 es el estándar legal en España.

WIP : Work In Progress (Trabajo en Curso). Número de tareas activas de forma simultánea; su limitación es un principio del método Kanban.

XSS : Cross-Site Scripting. Vulnerabilidad web que permite inyectar scripts maliciosos en páginas vistas por otros usuarios.

yfinance : Librería de Python que obtiene datos históricos de mercado desde Yahoo Finance.

Introducción

El acceso a los datos financieros no ha dejado de crecer, pero saber interpretarlos sigue siendo una ventaja reservada a unos pocos. Los grandes fondos operan con herramientas que cuestan miles de euros al año y reaccionan en cuestión de segundos, mientras que el inversor de a pie se enfrenta a las mismas noticias y cotizaciones sin el tiempo ni los conocimientos para convertirlas en decisiones. Plataformas como Investing.com, Morningstar o Bloomberg ayudan a paliar la falta de datos, pero leerlos, correlacionarlos e interpretarlos sigue siendo tarea del usuario.

La inteligencia artificial permite acortar esa distancia, y de ahí nace FinSight. La herramienta aprovecha el procesamiento de lenguaje natural y el aprendizaje profundo para analizar una acción desde cuatro perspectivas, el sentimiento de las noticias, la tendencia del precio, el análisis fundamental y el técnico, y explicar el resultado en lenguaje sencillo. Los datos ya están disponibles; lo que falta es su análisis, y ese es el hueco que este trabajo pretende cubrir.

Planteamiento del problema

El problema tiene dos caras. Por un lado, la cantidad y la complejidad de los datos financieros los vuelve inabarcables de forma manual para quien no es profesional del sector. Por otro, las noticias que mueven el mercado están dispersas entre muchas fuentes, y encontrarlas, leerlas y valorar su impacto consume un tiempo del que el usuario medio no dispone. Las herramientas existentes alivian la falta de datos, pero no la de análisis: algunas reflejan un sentimiento basado en la opinión de sus usuarios, sin noticias que respalden esa tendencia.

FinSight no busca competir con plataformas ya consolidadas como Yahoo Finance, sino complementarlas agilizando y explicando el análisis. Los modelos de lenguaje resultan clave para interpretar muchos datos en poco tiempo, aunque tienen límites: el mercado reacciona a expectativas más que a hechos, de modo que una misma noticia puede ser buena o mala según lo que ya estuviera descontado. Un beneficio que sube un 20 % parece positivo, pero si se esperaba un 35 % y los inversores ya habían reaccionado, el efecto será negativo. Por eso el informe no descansa en una sola señal, sino que combina los cuatro análisis para alcanzar una conclusión más fiable.

Objetivos

El objetivo principal es desarrollar una plataforma web gratuita y accesible que acerque el análisis financiero de acciones a todo el mundo. De aquí se derivan los siguientes objetivos:

- Desarrollar un sistema de análisis de sentimiento de noticias financieras usando la tecnología GraphRAG. Generar un resumen con las noticias más relevantes e influyentes y una breve explicación del por qué de esas decisiones.
- Implementar un módulo de predicción de la tendencia del precio de una acción basado en redes neuronales recurrentes GRU, entrenadas por acción a partir de sus datos históricos.
- Integrar un sistema de análisis fundamental cuyo resultado sea un resumen en lenguaje natural donde se expliquen las métricas más importantes y cómo afectan a la empresa.
- Integrar una estrategia de análisis técnico para enriquecer el informe final con otro punto de vista.
- Entregar al cliente el informe final del análisis de la acción que él haya escogido. Se busca que sea un análisis holístico y bien fundamentado en datos reales.
- Desarrollar una aplicación web accesible, que cumpla con el estándar WCAG 2.1 AA.

- Proteger la herramienta frente a ataques cibernéticos. Implementar rate limiting en las llamadas a APIs, sanitizar todas las entradas, configurar CORS y prevenir ataques de tipo XSS.
- El producto debe cumplir con la normativa y legislación a la que está sujeto.

Quedan fuera del alcance de este proyecto funciones como la integración con brókeres externos que permita la ejecución de órdenes desde la plataforma o la proyección de datos intradía.

Metodología

El desarrollo de FinSight se ha llevado a cabo mediante una metodología iterativa e incremental, adaptada a las características concretas del proyecto. Se ha descartado un enfoque en cascada porque los requisitos de los módulos de inteligencia artificial no pueden especificarse completamente a priori: el comportamiento del modelo GRU y del sistema GraphRAG solo puede evaluarse empíricamente durante el desarrollo. La adopción de un marco Scrum completo con iteraciones de duración fija tampoco resulta apropiada para un proyecto unipersonal con un plazo ajustado. En su lugar, el proyecto se organiza en seis iteraciones de duración variable, cada una de las cuales produce un incremento funcional verificable e integrable, manteniendo en todo momento el sistema en un estado desplegable.

Para la gestión visual del trabajo se ha adoptado Kanban [3], una metodología originada en los sistemas de producción Toyota y adaptada al desarrollo de software. Kanban se fundamenta en la visualización del flujo de trabajo, la limitación del trabajo en curso (Work In Progress, WIP) y la mejora continua del proceso.

A diferencia de Scrum, Kanban no impone ciclos de tiempo fijos, aunque sí fechas límite, ni roles predefinidos, lo que lo hace especialmente adecuado para proyectos unipersonales con flujo de trabajo continuo. Su integración con el ciclo de iteraciones descrito anteriormente permite combinar la planificación a medio plazo propia del desarrollo iterativo con la visibilidad inmediata del estado de cada tarea que Kanban proporciona.

El tablero Kanban del proyecto se ha implementado mediante GitHub Projects, con las siguientes columnas que representan los estados posibles de cualquier unidad de trabajo:

1. **Backlog**: tareas identificadas y especificadas mediante issues de GitHub, pendientes de ser incorporadas a la iteración activa.
2. **Iteración actual**: tareas seleccionadas para la iteración en curso, ordenadas por prioridad.
3. **En progreso**: tarea en desarrollo activo. Se aplica un límite de WIP de una tarea simultánea para evitar la fragmentación de la atención y favorecer la finalización sobre el inicio de nuevas tareas.
4. **En revisión (PR)**: tarea con implementación completada, cuya pull request asociada está siendo verificada por el pipeline de integración continua.
5. **Hecho**: tarea integrada en main, con todos los checks de CI superados y el issue correspondiente cerrado.

Las transiciones entre columnas están parcialmente automatizadas: la apertura de un issue lo sitúa automáticamente en “Backlog”, la apertura de una pull request lo traslada a “En revisión”, y la integración en main lo marca como “Hecho”. Esta automatización reduce la fricción administrativa y asegura que el tablero refleje en todo momento el estado real del proyecto sin depender de actualizaciones manuales.

Estructura del trabajo

El documento se organiza en cuatro capítulos, precedidos por esta introducción y seguidos de la conclusión y la lista de referencias.

La introducción sitúa el proyecto en su contexto, expone la motivación que lo justifica, fija los objetivos, describe la metodología empleada y presenta la organización del documento.

El capítulo 1 estudia el problema desde tres ángulos: el contexto del proyecto, con las necesidades del usuario objetivo y las limitaciones de las herramientas de análisis

financiero actuales; una revisión del estado de la cuestión, que repasa las soluciones existentes, los modelos de lenguaje aplicados a las finanzas, las arquitecturas Graph-RAG y los modelos de series temporales para la predicción bursátil; y, por último, la definición precisa del problema, que delimita el alcance técnico y funcional del trabajo.

El capítulo 2 recoge la gestión del proyecto software: el alcance y la definición formal del sistema, el presupuesto, el plan de trabajo (identificación, estimación y planificación de tareas), la gestión de recursos y de riesgos, y la legislación aplicable y de la que queda exento.

El capítulo 3 presenta la solución desarrollada. Describe la arquitectura del sistema y sus cuatro módulos de análisis, recorre el proceso de desarrollo en sus fases de análisis de requisitos, diseño, implementación y pruebas, y termina exponiendo el producto resultante con sus funcionalidades y su comportamiento en producción.

El capítulo 4 se dedica a la evaluación del sistema. Explica el proceso seguido, con la forma de evaluación adoptada y los casos de prueba diseñados, y analiza los resultados obtenidos para valorar hasta qué punto se cumplen los objetivos y cómo se comporta el sistema en los distintos escenarios de prueba.

La conclusión sintetiza las aportaciones del trabajo, recoge las limitaciones detectadas durante el desarrollo y reflexiona sobre el cumplimiento de los objetivos planteados.

Capítulo 1

Estudio del problema

Este capítulo sitúa FinSight en su contexto, revisa el estado de la cuestión en las disciplinas que integra y delimita el problema que resuelve.

1.1. El contexto del problema

La inversión en los mercados bursátiles se ha abierto a un público cada vez más amplio, pero el acceso a los datos no ha venido acompañado de un acceso equivalente a su interpretación. Los grandes fondos operan con terminales profesionales como Bloomberg o Refinitiv, que cuestan miles de euros al año y reúnen datos, noticias y analítica en un mismo entorno. El inversor particular, en cambio, se apoya en portales gratuitos que muestran cotizaciones y titulares, pero dejan en sus manos la tarea de leer, correlacionar e interpretar una información dispersa en numerosas fuentes.

En paralelo, el procesamiento de lenguaje natural y el aprendizaje profundo han alcanzado una madurez que permite automatizar parte de ese trabajo. Los grandes modelos de lenguaje resumen y explican texto financiero, las redes neuronales recurrentes modelan series de precios y las arquitecturas de recuperación aumentada anclan las respuestas en fuentes concretas. FinSight se sitúa en esa intersección, aprovechando estos avances para acortar la distancia entre el inversor profesional y el particular.

1.2. El estado de la cuestión

El análisis financiero para el usuario final descansa sobre dos bloques: las herramientas comerciales que distribuyen datos y la investigación académica sobre técnicas de análisis automático. A continuación se revisan ambos.

En el plano comercial, las terminales profesionales como Bloomberg y Refinitiv ofrecen datos en tiempo real y analítica avanzada, con un coste y una complejidad fuera del alcance del usuario medio. Morningstar aporta análisis fundamental y calificaciones de activos, aunque como servicio de pago que sigue exigiendo formación para interpretarlo. En el extremo gratuito, portales como Yahoo Finance concentran cotizaciones y noticias, pero entregan datos sin sintetizarlos en una conclusión.

El caso más cercano a este trabajo es Investing.com, disponible como plataforma web y como aplicación móvil. Su versión gratuita se limita a datos, cotizaciones y noticias, pero su suscripción de pago, InvestingPro, incorpora un apartado de análisis que, salvando las distancias, recuerda al de FinSight: estimaciones de valor razonable, una puntuación de salud financiera y herramientas de análisis asistidas por inteligencia artificial ¹. La diferencia está en que ese análisis queda tras un muro de pago, mientras que FinSight busca ofrecerlo de forma gratuita y en español. La Tabla 1.1 resume este panorama.

Herramienta	Enfoque	Limitación para el usuario no experto
Bloomberg y Refinitiv	Datos en tiempo real y analítica profesional	Coste de miles de euros al año y elevada complejidad de uso.
Morningstar	Análisis fundamental y calificaciones de activos	Servicio de pago que aún exige conocimiento para interpretarlo.
Yahoo Finance	Cotizaciones y noticias gratuitas	Entregan datos sin interpretarlos ni sintetizarlos.
Investing.com (InvestingPro)	Datos gratuitos y, en su versión de pago, análisis asistido por IA	El apartado de análisis solo está disponible mediante suscripción.

Tabla 1.1: Herramientas comerciales de información financiera y sus limitaciones para el usuario sin formación especializada.

¹Web de InvestingPro: <https://www.investing.com/pro/>

En el plano académico, el análisis de sentimiento financiero ha evolucionado desde los enfoques basados en léxicos y aprendizaje automático clásico [34] hasta los modelos de tipo *transformer* especializados, como FinBERT [5], y el uso de grandes modelos de lenguaje [27, 28]. En esta línea han aparecido conjuntos de datos anotados a nivel de entidad, como FinEntity [60] y FinMarBa [29], que relacionan el sentimiento con los activos concretos a los que afecta. El límite de estos métodos es conocido: el mercado reacciona a expectativas más que a hechos, por lo que el sentimiento aislado, sin contexto ni anclaje en fuentes, resulta poco fiable.

Para dotar de ese contexto a los modelos de lenguaje, la generación aumentada por recuperación (RAG) los alimenta con información recuperada de forma dinámica [30]. Su variante sobre grafos, GraphRAG [12], estructura ese conocimiento en una base de grafos que captura las relaciones entre entidades, noticias y eventos y habilita consultas por vecindad; existen además bibliotecas que facilitan su implementación sobre Neo4j [44].

En la predicción de series bursátiles, los modelos clásicos ARIMA [7] y las familias GARCH para la volatilidad [6, 13] conviven con las redes neuronales recurrentes, LSTM [19] y GRU [8], capaces de capturar dependencias temporales no lineales, y con las arquitecturas de atención [61]. La literatura reciente explora híbridos que descomponen la señal de precio antes de entrenar la red, combinando VMD o SSA con LSTM [31, 33, 57, 58], junto con estudios comparativos de predicción de tendencia [2, 65]. Aun así, la ventaja predictiva sobre el azar suele ser reducida y los modelos más complejos no siempre compensan su mayor coste.

1.3. La definición del problema

El problema que aborda FinSight nace de un desajuste sencillo de enunciar: los datos financieros están disponibles, pero su interpretación no lo está para el usuario medio. De la revisión anterior se desprenden varias limitaciones concretas:

- El coste de las herramientas que integran datos y análisis las deja fuera del alcance del particular.

- Las alternativas gratuitas ofrecen datos, pero exigen formación financiera para interpretarlos.
- Las noticias relevantes están repartidas entre muchas fuentes, y leerlas y correlacionarlas consume un tiempo del que el usuario no dispone.
- Un modelo de lenguaje no accede por sí solo a información fresca ni al histórico del mercado, que además reacciona a expectativas más que a hechos.
- Los modelos de predicción de series bursátiles apenas superan al azar, y los enfoques más pesados no justifican su coste.
- El servicio se ofrece de forma gratuita sobre infraestructura sin GPU, lo que limita el tamaño y la latencia de los modelos.

Frente a estas limitaciones, FinSight no propone un algoritmo nuevo, sino la integración de varias técnicas en una única herramienta accesible. Para el sentimiento se valoraron un clasificador especializado como FinBERT, un modelo de lenguaje sin contexto y una arquitectura GraphRAG; se optó por esta última porque ancla las respuestas del modelo en un grafo de noticias, entidades y eventos, lo que aporta trazabilidad y explicación. Para la tendencia se compararon modelos clásicos, LSTM, arquitecturas de atención e híbridos con VMD; se adoptó una red GRU compacta, validada de forma empírica, porque equilibra precisión y coste y se ejecuta en CPU sobre infraestructura gratuita, mientras que la descomposición VMD se descartó al no compensar su complejidad. El análisis fundamental y el técnico, de corte clásico, se traducen a lenguaje natural, y un modelo de lenguaje sintetiza los cuatro resultados en un informe en español.

La solución se plantea, por tanto, como integración de disciplinas: procesamiento de lenguaje natural, aprendizaje profundo, bases de datos de grafos e ingeniería del software [16], junto con el dominio financiero y las exigencias de accesibilidad y cumplimiento normativo. Ninguna de estas piezas es novedosa por separado; la aportación del trabajo está en combinarlas en una herramienta gratuita y comprensible para quien no es experto.

Capítulo 2

Gestión de proyecto software

Se ha realizado una simulación de la gestión del proyecto software, la cual simulará un proyecto real, realizado con las condiciones habituales del entorno empresarial. El objetivo del capítulo es plasmar los conocimientos adquiridos a lo largo de la titulación y no la forma en la cual se ha gestionado el Trabajo Fin de Grado.

2.1. Alcance del proyecto

El alcance delimita qué construye este proyecto y dónde están sus fronteras, de modo que el equipo y el cliente compartan la misma idea del producto antes de empezar. El resultado es FinSight, una plataforma web que, a partir del nombre o el símbolo de una acción del mercado estadounidense, genera un informe de análisis en español, comprensible sin conocimientos financieros previos y accesible de forma gratuita desde el navegador.

Dentro del alcance entra el desarrollo completo de esa plataforma, con sus dos mitades. En el lado del servidor, una API que orquesta cuatro módulos de análisis: el de sentimiento de noticias mediante una arquitectura GraphRAG sobre un grafo de conocimiento, el de predicción de la tendencia del precio con un modelo de aprendizaje profundo sobre series temporales, el de análisis fundamental y el de análisis técnico. Los cuatro se ejecutan por demanda y un modelo de lenguaje sintetiza sus resultados en una conclusión global, siempre en español. En el lado del cliente, una aplicación web de

página única que presenta el informe, con un buscador de valores validado, estados de carga y de error, y un diseño accesible conforme al nivel AA de las WCAG. El proyecto cubre además las salvaguardas transversales: el aviso legal permanente, la gestión del consentimiento de cookies, las medidas de seguridad (limitación de peticiones, saneamiento de entradas, configuración de CORS y prevención de XSS) y el despliegue en una plataforma en la nube.

El proyecto opera bajo varias premisas que condicionan su alcance. El análisis se apoya en datos de cierre de mercado, no en cotizaciones intradía; el sistema se despliega sobre infraestructura gratuita, sin GPU y con arranque en frío tras periodos de inactividad; y el consumo de las fuentes externas de noticias y precios está sujeto a las cuotas de sus proveedores, lo que obliga a apoyarse en cachés. Por su carácter meramente informativo, la herramienta no emite recomendaciones de inversión, algo que el aviso legal recuerda de forma permanente.

Para fijar los límites con claridad, la Tabla 2.1 recoge lo que queda expresamente fuera del alcance de este proyecto.

Fuera del alcance	Motivo
Análisis en tiempo real e intradía	El sistema trabaja con datos de cierre de mercado; no se contemplan cotizaciones en tiempo real ni operativa intradía.
Predicción de tendencia fuera del Nasdaq	El modelo de predicción se valida sobre el universo del Nasdaq; para otros mercados esa función no está soportada, aunque el resto de análisis sigue disponible.
Cuentas de usuario y datos personales	No hay registro ni inicio de sesión, y no se almacenan datos personales más allá del consentimiento de cookies guardado localmente en el navegador.

Tabla 2.1: Elementos que quedan fuera del alcance del proyecto.

2.2. Definición del proyecto

2.2.1. Descripción y justificación

FinSight es una plataforma web de análisis de acciones dirigida al inversor particular sin formación financiera especializada. El problema que aborda tiene tres caras: el coste

de las herramientas profesionales, fuera del alcance de la mayoría; la barrera de conocimiento necesaria para interpretar los datos; y el tiempo que exige leer y correlacionar noticias dispersas antes de sacar una conclusión. Los datos están disponibles, pero su análisis no. El proyecto responde a esa carencia apoyándose en la inteligencia artificial para procesar la información y explicarla en lenguaje sencillo, combinando cuatro perspectivas de análisis en un único informe que el usuario obtiene bajo demanda.

2.2.2. Objetivos

El objetivo general es desarrollar una plataforma web gratuita y accesible que acerque el análisis financiero de acciones del Nasdaq a cualquier persona, entregando un informe consolidado y comprensible en español. De él se derivan los siguientes objetivos específicos:

- Desarrollar un módulo de análisis de sentimiento de noticias financieras mediante una arquitectura GraphRAG, que resuma las noticias más influyentes y explique el porqué de su valoración.
- Implementar un módulo de predicción de la tendencia del precio a medio plazo, con un horizonte de diez sesiones, basado en aprendizaje profundo sobre series temporales.
- Integrar un módulo de análisis fundamental cuyo resultado sea una explicación en lenguaje natural de las métricas más relevantes de la empresa.
- Integrar un módulo de análisis técnico que enriquezca el informe con la señal derivada de los indicadores de precio y volumen.
- Entregar un informe final holístico que sintetice los cuatro análisis en una conclusión global fundamentada en datos reales.
- Construir una aplicación web accesible que cumpla el nivel AA de las WCAG 2.1.
- Proteger la herramienta con limitación de peticiones, saneamiento de entradas, configuración de CORS y prevención de XSS.
- Cumplir la normativa y la legislación a las que está sujeto el servicio.

2.2.3. Equipo y partes interesadas

El proyecto se aborda con un equipo reducido de tres perfiles, cuyas responsabilidades y dedicación estimada recoge la Tabla 2.2. El desarrollo de la aplicación recae en el programador; el asesor financiero aporta la estrategia de los análisis técnico y fundamental; y el jefe de proyecto supervisa la marcha del trabajo. Como partes interesadas, más allá del equipo, se sitúan el usuario final, un inversor particular que busca entender una acción con rapidez, y la dirección que promueve el producto y espera una herramienta accesible y conforme a la normativa.

Rol	Responsabilidad principal	Dedicación
Jefe de proyecto	Supervisión, planificación y seguimiento del proyecto.	10 h
Programador	Desarrollo de la aplicación web completa (frontend y backend) y de las pruebas.	150 h
Asesor financiero	Definición de la estrategia de los análisis técnico y fundamental.	20 h

Tabla 2.2: Equipo del proyecto, responsabilidades y dedicación estimada.

2.2.4. Metodología y entregables

El desarrollo sigue una metodología iterativa e incremental, organizada en iteraciones de duración variable que producen incrementos funcionales verificables, con la gestión visual del trabajo apoyada en un tablero Kanban sobre GitHub Projects y en una canalización de integración continua que actúa como puerta de calidad antes de integrar cualquier cambio; las decisiones de diseño relevantes se registran como *Architecture Decision Records*. Esta metodología se detalla en la introducción.

Los entregables principales del proyecto son la plataforma FinSight desplegada, con su aplicación web y su API; los modelos de predicción entrenados por *ticker* y el grafo de conocimiento que sustenta el análisis de sentimiento; el repositorio de código con su canalización de integración continua y las decisiones de diseño documentadas; y la presente memoria junto con la documentación asociada.

2.3. Presupuesto

A continuación se detalla un presupuesto estimado para el coste total de este proyecto.

2.3.1. Coste de personal

Tarea	Perfil	Horas	Euros/Hora	Total bruto
Desarrollo aplicación y pruebas	Programador Junior	150	60	9000,00€
Desarrollo estrategia financiera	Asesor Financiero	20	150	3000,00€
Supervisión del proyecto	Jefe de Proyecto	10	120	1200,00€
Total				13200,00€

Tabla 2.3: Presupuesto de personal.

En esta tabla se muestra el salario bruto del personal. A mayores, hay que añadir un 30 % extra relativo a los costes de seguridad social, lo que supone un importe adicional de 3960 €.

2.3.2. Coste del hardware

Para la realización de este proyecto se requieren los siguientes dispositivos:

1. Portátil MSI Katana 15 B13VFK:

Procesador: Intel i7-13620H

RAM: 32GB

Disco duro: 1TB SSD NVMe

Tarjeta gráfica: Nvidia RTX 4060 8GB

- Precio (sin IVA): 1247,41 €

2. Portátil Huawei Matebook D14:

Procesador: AMD Ryzen 5 3500U

RAM: 8GB

Disco duro: 512GB SSD

Tarjeta gráfica: Radeon Vega 8 Graphics

- Precio (sin IVA): 295,82 €

Del importe de los dispositivos, se imputa al proyecto la cantidad de 154,32 €, correspondiente a la amortización de los equipos durante el período de desarrollo del proyecto, considerando una vida útil de 5 años.

La amortización imputada al proyecto se ha calculado mediante la siguiente expresión:

$$\text{Amortización} = \frac{\text{Coste de adquisición}}{\text{Meses de vida útil}} \cdot \text{Meses de uso} \quad (2.1)$$

2.3.3. Coste de servicios

Durante el desarrollo de este proyecto ha sido necesaria la utilización de la plataforma de OpenAI. La creación del grafo de conocimiento demandó el uso de modelos LLM más potentes que los ejecutados localmente.

Modelo	Categoría	Tokens usados	Coste por millón de tokens	Total
GPT 5.4-mini	Tokens de entrada	3,631 M	0,65€	2,36€
GPT 5.4-mini	Tokens de salida	0,503 M	3,90€	1,96€
text-embedding-3-large	Indiferente	0,273 M	0,11€	0,03€
Total				4,35€

Tabla 2.4: Costes de infraestructura.

Por otro lado, el propio uso de la aplicación también requiere el uso del modelo de embeddings y el LLM de razonamiento. El primero es usado para calcular los embeddings de las noticias a analizar, necesarios para realizar la búsqueda por similitud semántica. El LLM de razonamiento es usado para analizar el sentimiento de una noticia, para explicar los motivos de las decisiones tomadas en cada módulo, y también para

sintetizar la información y generar el informe final. El coste de tener esta aplicación desplegada dependerá de la cantidad de usuarios de la misma y del número de análisis que soliciten. Los precios por cada millón de tokens consumidos están especificados en la tabla 2.

2.3.4. Costes indirectos

Los costes indirectos incluyen los gastos derivados de desarrollar este proyecto en una oficina. Se consideran de este tipo el agua, la luz, el internet, etc. Se estima que estos costes representan un 10 % de los costes directos del proyecto sin contar los costes de personal, esto es, sobre el hardware y la infraestructura.

2.3.5. Coste total

Concepto	Coste
Costes de personal	13200,00€
Costes de seguridad social (30 %)	3960,00€
Costes de hardware	154,32€
Costes de infraestructura	4,35€
Subtotal	17318,67€
Costes indirectos (10 % sin personal)	15,87€
Beneficio industrial (11 %)	1906,80€
IVA (21 %)	4040,68€
Total Proyecto	23282,02€

Tabla 2.5: Presupuesto total.

2.4. Plan de trabajo

El plan de trabajo traduce el alcance definido en la sección anterior en un conjunto de tareas concretas, les asigna un esfuerzo estimado y las ordena en el tiempo. Parte de la organización iterativa e incremental descrita en la metodología: el trabajo se reparte en seis iteraciones de duración variable, cada una cerrada con un incremento funcional

verificable. En GitHub, cada iteración se corresponde con un *milestone* que agrupa los *issues* de las tareas que la componen, de modo que la planificación descrita aquí coincide con la ejecución registrada en el repositorio.

2.4.1. Identificación de tareas

El desarrollo se apoya en una fase previa de investigación y en las seis iteraciones posteriores. La fase previa es anterior al proyecto: durante el primer cuatrimestre del curso se estudió el problema del análisis de sentimiento en textos financieros y se construyó un prototipo de línea de comandos que validó el enfoque. Ese trabajo no se imputa al presupuesto de FinSight, pero aporta la base de conocimiento que se reutiliza en la primera iteración del módulo de sentimiento, por lo que se recoge como hito de partida.

A partir de ahí, las seis iteraciones se estructuran de la forma siguiente:

1. **Iteración 0. Base del proyecto.** Preparación del esqueleto de la aplicación (backend en FastAPI y frontend en React), la estructura de carpetas, la gestión de dependencias con uv y la canalización de integración continua: análisis estático con Ruff y ESLint, ejecución de pruebas y compilación de producción, más CodeQL en las integraciones a `main`. Deja el repositorio en un estado desplegable sobre el que construir el resto.
2. **Iteración 1. Análisis de sentimiento.** Módulo de sentimiento sobre una arquitectura GraphRAG. Comprende el servicio LLM centralizado según el patrón Adaptador (proveedores OpenAI y Ollama tras una interfaz común), la construcción del grafo de conocimiento en Neo4j a partir de los conjuntos FinEntity y FinMarBa, la recuperación por vecindad en el grafo y la cadena de proveedores de noticias con Finnhub como fuente primaria y NewsAPI como reserva. Reutiliza la investigación de la fase previa.
3. **Iteración 2. Predicción de tendencia (*deep learning*).** Es la iteración más extensa. Parte de un estudio comparativo de modelos de series temporales (ARIMA, GARCH, redes LSTM y GRU y arquitecturas híbridas) que concluye en una red GRU sin descomposición VMD, decisión registrada en un *Architecture Deci-*

sion Record. Incluye la optimización de hiperparámetros con Optuna, la receta de entrenamiento congelada, el núcleo del modelo (preprocesado, arquitectura y persistencia de artefactos), la canalización de entrenamiento por *ticker*, la carga perezosa de modelos con caché LRU y el reentrenamiento diario automático mediante APScheduler.

4. **Iteración 3. Análisis fundamental y técnico.** Integración de los motores de análisis fundamental y técnico aportados por el colaborador de Finanzas. Cubre la resolución del universo de valores (S&P 500 y MSCI World), el control de calidad de los datos, los servicios que exponen ambos análisis a través de la API y el módulo compartido de obtención de datos de mercado que ambos reutilizan.
5. **Iteración 4. Informe consolidado y caché.** Orquestación de los cuatro módulos en paralelo con `asyncio.gather`, síntesis mediante el LLM de la conclusión global en español, cachés con TTL por módulo y la interfaz completa (sistema de diseño, buscador de valores, presentación del informe, consentimiento de cookies y páginas de servicio), junto con las pruebas de la parte cliente y su accesibilidad.
6. **Iteración 5. Pruebas y despliegue.** Aseguramiento de la calidad con pruebas de contrato, unitarias y de accesibilidad, endurecimiento de la configuración de producción (CORS, HTTPS y limitación de peticiones) y despliegue de la aplicación en la plataforma en la nube.

La Tabla 2.6 resume las iteraciones, su objetivo y el número de *issues* que agrupa cada *milestone* en GitHub.

2.4.2. Estimación de tareas

La estimación del esfuerzo se ha realizado por juicio de experto y por analogía con el prototipo de la fase previa, ajustando cada iteración según el número de *issues* que agrupa y su complejidad técnica. El límite de una sola tarea activa, propio del tablero Kanban, hace que las iteraciones del programador se ejecuten de forma esencialmente secuencial, lo que simplifica el reparto. Las cifras se afinaron durante el desarrollo con el propio avance del tablero.

Iteración	Objetivo	Issues
Fase previa	Investigación del análisis de sentimiento y prototipo de línea de comandos	–
Iteración 0	Base: esqueleto, integración continua y estructura del proyecto	4
Iteración 1	Sentimiento: GraphRAG sobre Neo4j y servicio LLM centralizado	5
Iteración 2	Predicción de tendencia: estudio comparativo, GRU y reentrenamiento	13
Iteración 3	Análisis fundamental y técnico: integración de los motores	5
Iteración 4	Informe consolidado, cachés e interfaz completa	10
Iteración 5	Pruebas de calidad y despliegue en la nube	8
Total		45

Tabla 2.6: Iteraciones del proyecto y su correspondencia con los *milestones* de GitHub.

El reparto respeta los totales fijados en el presupuesto: 150 horas del programador, 20 del asesor financiero y 10 del jefe de proyecto, 180 en conjunto. Las 20 horas del asesor corresponden a la definición de la estrategia de los análisis técnico y fundamental, que se entrega antes de la iteración 3 para que el programador pueda integrarla. La Tabla 2.7 detalla la distribución por iteración y perfil.

Iteración	Programador	Asesor	Jefe de proyecto	Total
It. 0 (Base)	15	–	1	16
It. 1 (Sentimiento)	25	–	2	27
It. 2 (Deep learning)	45	–	2	47
It. 3 (Fundamental y técnico)	25	20	2	47
It. 4 (Informe y caché)	25	–	1	26
It. 5 (Pruebas y despliegue)	15	–	2	17
Total	150	20	10	180

Tabla 2.7: Estimación del esfuerzo en horas por iteración y perfil.

2.4.3. Planificación de tareas

Con las tareas identificadas y estimadas, la planificación las sitúa en un calendario de doce semanas de trabajo a tiempo parcial, coherente con las 180 horas presupuestadas. El límite de una tarea en curso obliga a un desarrollo secuencial de las iteraciones del programador. En paralelo, el asesor financiero define la estrategia de los análisis técnico y fundamental durante la segunda iteración, de modo que esté disponible cuando comienza su integración, y el jefe de proyecto supervisa el trabajo en las transiciones entre iteraciones. La fase previa de investigación, anterior a este calendario, queda representada como hito de partida (H0).

Cada iteración cierra con un hito verificable: el incremento se integra en `main` solo cuando la canalización de integración continua está en verde. La Figura 2.1 muestra el diagrama de Gantt del proyecto y la Tabla 2.8 recoge los hitos, sus entregables y las dependencias entre iteraciones.

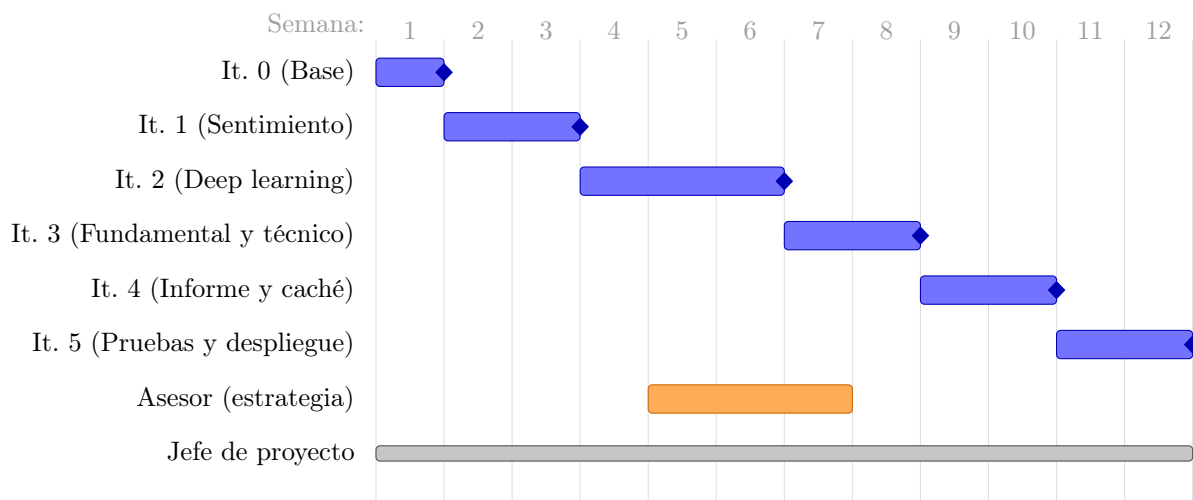


Figura 2.1: Diagrama de Gantt del proyecto. Las barras azules son las iteraciones del programador, ejecutadas de forma secuencial por el límite de una tarea en curso; el rombo marca el hito de cierre de cada iteración. La barra naranja es la definición de la estrategia por el asesor financiero y la gris, la supervisión del jefe de proyecto.

Las dependencias condicionan el orden. La iteración 1 necesita la infraestructura de la iteración 0 y reutiliza la investigación previa. Las iteraciones 3 y 4 dependen del servicio LLM centralizado que entrega la iteración 1, y la iteración 3 requiere además la estrategia del asesor y el módulo compartido de datos de mercado. La iteración 4

no puede sintetizar el informe hasta que los cuatro módulos están disponibles, por lo que sucede a las tres anteriores. La iteración 5 cierra el proyecto una vez integrada la funcionalidad completa.

Hito	Iteración	Entregable	Semana	Depende de
H0	Fase previa	Investigación y prototipo CLI de análisis de sentimiento	Cuatr. anterior	–
H1	Iteración 0	Infraestructura y canalización de integración continua operativas	Semana 1	H0
H2	Iteración 1	Módulo de análisis de sentimiento GraphRAG y servicio LLM	Semana 3	H1
H3	Iteración 2	Modelo GRU por <i>ticker</i> y reentrenamiento automático	Semana 6	H1
H4	Iteración 3	Módulos fundamental y técnico integrados en la API	Semana 8	H1, H2
H5	Iteración 4	Informe consolidado en español e interfaz completa	Semana 10	H2, H3, H4
H6	Iteración 5	Sistema probado y desplegado en producción	Semana 12	H5

Tabla 2.8: Hitos del proyecto, entregables, semana de cierre y dependencias.

2.5. Gestión de recursos

La gestión de recursos especifica los medios necesarios para ejecutar el plan de trabajo y los reparte entre las tareas. Se distinguen tres tipos: personales, materiales y de software y servicios.

2.5.1. Especificación de recursos

Los recursos personales son los tres perfiles del equipo descritos en la Tabla 2.2: el jefe de proyecto (10 h), el programador (150 h) y el asesor financiero (20 h).

En cuanto a los recursos materiales, el proyecto emplea dos equipos portátiles, cuyas características y uso recoge la Tabla 2.9. El MSI Katana incorpora una GPU NVIDIA

RTX 4060, imprescindible para la optimización de hiperparámetros y el entrenamiento local de los modelos GRU, y para ejecutar Ollama sin conexión.

Equipo	Características	Uso principal
MSI Katana 15 B13VFK	Intel i7-13620H, 32 GB RAM, 1 TB SSD, GPU NVIDIA RTX 4060 8 GB	Desarrollo y entrenamiento local de los modelos
Huawei Matebook D14	AMD Ryzen 5 3500U, 8 GB RAM, 512 GB SSD	Redacción de la memoria e investigación

Tabla 2.9: Recursos materiales del proyecto.

El desarrollo se apoya, además, en un conjunto de servicios externos y herramientas, casi todos en su nivel gratuito, que resume la Tabla 2.10. La única dependencia de pago es OpenAI, que proporciona el LLM de razonamiento y los embeddings del grafo de conocimiento. El resto del stack de desarrollo (FastAPI, React, PyTorch y uv, entre otros) es software libre.

Recurso	Función	Modalidad
OpenAI	LLM de razonamiento (GPT-5.4-mini) y embeddings (text-embedding-3-large)	De pago
Ollama	LLM local para desarrollo y ejecución sin conexión	Gratuito
Neo4j AuraDB	Grafo de conocimiento del módulo de sentimiento	Gratuito
Finnhub	Noticias (fuente primaria), datos de mercado y validación de símbolos	Gratuito
NewsAPI	Noticias (fuente de reserva)	Gratuito
yfinance	Series de precios de cierre (EOD)	Gratuito
Render	Alojamiento del frontend y el backend	Gratuito
GitHub	Repositorio, integración continua y tablero Kanban	Gratuito

Tabla 2.10: Recursos de software y servicios.

2.5.2. Asignación de recursos

La asignación vincula cada recurso con las tareas donde se emplea, siguiendo el reparto de esfuerzo de la Tabla 2.7. El programador interviene en todas las iteraciones; el asesor financiero se concentra en la estrategia de los análisis técnico y fundamental

que integra la iteración 3; y el jefe de proyecto supervisa en las transiciones entre iteraciones. Ambos portátiles pertenecen al programador: el MSI como equipo principal de desarrollo y entrenamiento, y el Huawei como equipo secundario para la redacción de la memoria y la investigación. La Tabla 2.11 recoge la asignación completa.

Recurso	Asignación
Programador (150 h)	Todas las iteraciones: desarrollo, integración y pruebas
Asesor financiero (20 h)	Iteración 3: estrategia de los análisis técnico y fundamental
Jefe de proyecto (10 h)	Transiciones entre iteraciones: planificación y seguimiento
MSI Katana (RTX 4060)	Programador: desarrollo y entrenamiento local de los modelos
Huawei Matebook D14	Programador: redacción de la memoria e investigación
OpenAI (LLM y embeddings)	Iteraciones 1, 3 y 4; embeddings obligatorios en todo el sistema
Neo4j AuraDB	Iteración 1: grafo de conocimiento
Finnhub y NewsAPI	Iteración 1: noticias; validación de símbolos y datos de mercado
yfinance	Iteraciones 2 y 3: series de precios de cierre
Render	Iteración 5: despliegue del frontend y el backend
GitHub	Todo el proyecto: repositorio, integración continua y Kanban

Tabla 2.11: Asignación de recursos a tareas e iteraciones.

Dos condicionantes marcan esta asignación. El entrenamiento y la optimización de los modelos exigen la GPU del MSI, pero el entorno de producción (Render en su nivel gratuito) no dispone de GPU, por lo que la inferencia se ejecuta en CPU apoyándose en la caché LRU de modelos. Por otro lado, aunque Ollama cubre el LLM durante el desarrollo, los embeddings dependen siempre de OpenAI, de modo que una clave de OpenAI con crédito es un recurso necesario incluso en local.

2.6. Gestión de riesgos

La gestión de riesgos identifica las amenazas que pueden comprometer los objetivos del proyecto y define cómo responder a ellas. Los riesgos de carácter legal y normativo se tratan aparte, en el apartado de legislación y normativa.

2.6.1. Identificación de riesgos

La Tabla 2.12 recoge los riesgos identificados, agrupados por categoría. Buena parte proceden de las restricciones del sistema: el despliegue sobre infraestructura gratuita sin GPU, las cuotas de las fuentes externas y las limitaciones propias de los modelos de aprendizaje profundo.

ID	Riesgo	Categoría
R1	Arranque en frío de Render tras un periodo de inactividad, con demora de 30 a 60 segundos	Infraestructura
R2	Inferencia del modelo GRU en CPU, al no disponer de GPU en producción	Rendimiento
R3	Valor predictivo escaso del modelo GRU, que apenas supera al azar y en ocasiones queda por debajo	Modelo
R4	Falta de validación del modelo GRU fuera del universo Nasdaq sobre el que se optimizó	Modelo
R5	Alucinaciones del LLM o explicaciones financieras incorrectas	Calidad
R6	Cuotas y cobertura limitadas de las fuentes de noticias (Finnhub y NewsAPI)	Proveedores
R7	Coste y dependencia de OpenAI para el LLM y, de forma obligatoria, para los embeddings	Proveedores
R8	Rotura de fuentes de datos no oficiales (yfinance) o límite de 512 MB del <i>free tier</i> de Neo4j	Proveedores
R9	Plazo ajustado con un único desarrollador, que concentra el conocimiento del proyecto	Planificación
R10	Dependencia del colaborador de Finanzas para los motores de análisis técnico y fundamental	Equipo
R11	Vulnerabilidades de seguridad, como XSS desde la salida del LLM o abuso de los <i>endpoints</i>	Seguridad

Tabla 2.12: Riesgos identificados en el proyecto.

2.6.2. Análisis de riesgos

Cada riesgo se valora por su probabilidad y su impacto, de donde resulta una severidad que ordena la prioridad de la respuesta. La Tabla 2.13 resume esa valoración y el plan de mitigación previsto.

ID	Prob.	Impacto	Severidad	Respuesta prevista
R1	Alta	Medio	Media	Petición de calentamiento antes de la demostración y aviso del tiempo estimado al usuario.
R2	Media	Medio	Media	Caché LRU y carga perezosa de modelos, con seguimiento del objetivo de menos de 5 segundos.
R3	Alta	Alto	Alta	Presentar la predicción como una señal más del informe holístico, nunca como recomendación, comunicar las métricas con transparencia y mantener el aviso legal.
R4	Media	Medio	Media	Acotar el módulo al universo Nasdaq y avisar al usuario; el resto de análisis permanece disponible.
R5	Media	Alto	Alta	Anclar los <i>prompts</i> en datos reales del grafo y las métricas, reducir la temperatura de los modelos, escapar la salida del LLM y mantener el aviso legal permanente.
R6	Media	Medio	Media	Cadena de reserva de Finnhub a NewsAPI y caché con TTL de 30 minutos.
R7	Media	Medio	Media	Construir el grafo en un proceso por lotes con coste acotado, apoyarse en caché y usar Ollama en desarrollo.
R8	Media	Medio	Media	Aislar la fuente tras un módulo compartido de datos de mercado y acotar los conjuntos al límite de Neo4j.
R9	Media	Alto	Alta	Trabajar con incrementos desplegados e iterativos, limitar el trabajo en curso y documentar las decisiones en ADR para reducir la dependencia de una sola persona.
R10	Media	Medio	Media	Definir una interfaz de integración clara, usar dobles de prueba durante el desarrollo e integrar en la iteración 3 con margen.
R11	Baja	Alto	Media	Sanear las entradas, escapar la salida del LLM, limitar peticiones por IP, configurar CORS y HTTPS y analizar el código con CodeQL.

Tabla 2.13: Análisis y mitigación de los riesgos.

Los riesgos más severos comparten una misma respuesta de fondo: el informe no descansa en un único módulo. Que la predicción del GRU apenas supere al azar (R3) o que el LLM pueda equivocarse (R5) se mitiga presentando cada análisis como una señal dentro de un informe de cuatro perspectivas, acompañado siempre del aviso legal sobre su carácter informativo. El plazo ajustado con un solo desarrollador (R9) se contiene con el desarrollo iterativo, que mantiene el sistema desplegable en todo momento y prioriza el núcleo funcional.

2.7. Legislación y normativa

FinSight es una aplicación web de uso público que integra sistemas de inteligencia artificial y proporciona información de carácter financiero. Su naturaleza hace que esté sujeta a un conjunto de disposiciones legales y normativas técnicas. A continuación, se describen los marcos normativos que aplican organizados por ámbito temático.

2.7.1. Ley de Servicios de la Sociedad de la Información y de Comercio Electrónico

La Ley 34/2002, de 11 de julio, de Servicios de la Sociedad de la Información y de Comercio Electrónico (LSSI-CE) [20] es la normativa española de referencia para cualquier servicio prestado por medios electrónicos con carácter público, exista o no un modelo de negocio asociado. FinSight, como plataforma web accesible a través de Internet, queda sujeta a las obligaciones que esta ley impone a los prestadores de servicios de la sociedad de la información (PSSI).

El artículo 10 de la LSSI-CE exige que el prestador del servicio facilite de forma permanente, fácil, directa y gratuita determinados datos identificativos, entre ellos la denominación o el nombre, el domicilio, la dirección de correo electrónico y, en su caso, el número de identificación fiscal. Esta información debe figurar en un aviso legal accesible desde cualquier punto de la interfaz, y su omisión se considera una infracción leve en virtud del artículo 38 de la misma ley.

Por otra parte, el artículo 22 regula el uso de dispositivos de almacenamiento y recuperación de datos en el terminal del usuario, lo que comúnmente se conoce como

cookies. Este artículo exige que el usuario preste su consentimiento previo e informado antes de que se instale cualquier *cookie* que no sea estrictamente necesaria para la prestación del servicio. Por consiguiente, la política de *cookies* del sistema debe distinguir con claridad entre las *cookies* técnicas, cuyo despliegue está permitido sin consentimiento, y las *cookies* analíticas o de seguimiento, que sí requieren consentimiento explícito antes de activarse. La Agencia Española de Protección de Datos (AEPD) ha publicado guías técnicas específicas sobre el diseño de mecanismos de consentimiento conformes, cuyas directrices deben tenerse en cuenta al desarrollar el componente de gestión de *cookies* del frontend.

2.7.2. Protección de datos personales: RGPD y LOPDGDD

El Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD) [53], se aplica al sistema tanto si este implementa cuentas de usuario registradas como si no. Las razones, de naturaleza técnica, son las siguientes.

En primer lugar, la dirección IP de origen de cada petición HTTP que queda registrada en los logs del servidor es un dato personal indirecto en el sentido del artículo 4.1 del RGPD [53], ya que, combinada con la información de que dispone el proveedor de acceso a Internet, permite identificar a una persona física. El sistema trata, por tanto, datos personales desde el momento en que registra la actividad de los usuarios, y eso activa todas las obligaciones del reglamento: base jurídica para el tratamiento, información al interesado, medidas de seguridad y ejercicio de derechos.

En segundo lugar, al prestar el servicio a usuarios de la Unión Europea entran en juego los derechos que reconocen los artículos 15 a 22 del RGPD [53]: acceso, rectificación, supresión, limitación del tratamiento, portabilidad y oposición. El sistema debe contar con un canal documentado para atender estas solicitudes y con procedimientos que garanticen su tramitación dentro de los plazos legales.

En tercer lugar, el sistema realiza llamadas a servicios operados por proveedores con sede en Estados Unidos, en concreto OpenAI, la Groq API y NewsAPI. Toda transmisión de datos a un país tercero es una transferencia internacional en el sentido

del Capítulo V del RGPD [53]. Por ahora, la licitud de estas transferencias se ampara en el marco EU-US Data Privacy Framework, adoptado mediante la Decisión de Ejecución (UE) 2023/1795 de la Comisión, de 10 de julio de 2023 [10], y esa base jurídica debe figurar de forma expresa en la política de privacidad del servicio. Además, conviene evaluar qué datos concretos se envían a cada proveedor externo y si es posible reducir esa transmisión al mínimo imprescindible, conforme al principio de minimización del artículo 5.1.c del RGPD [53].

En cuarto lugar, parte del tratamiento se realiza por encargo de terceros. Cuando el módulo de análisis de sentimiento envía los textos de las noticias a la OpenAI API para obtener sus representaciones vectoriales (*embeddings*), esos textos pueden contener referencias a personas físicas identificables, de modo que la transmisión supone un tratamiento de datos personales. Esta situación activa las obligaciones del artículo 28 del RGPD sobre el tratamiento por encargo, que obligan al responsable a formalizar con el encargado un contrato o acto jurídico equivalente que regule las condiciones del tratamiento. Para cubrir este requisito, OpenAI ofrece a los usuarios de su API un Acuerdo de Procesamiento de Datos (*Data Processing Addendum*, DPA), que debe suscribirse antes de enviar a través de la API cualquier contenido que pueda considerarse dato personal. En cuanto a la retención, OpenAI conserva por defecto las entradas y salidas de la API durante treinta días con fines de supervisión de abusos y seguridad, plazo que puede reducirse a cero mediante acuerdo específico en los planes de empresa. Además, OpenAI declara que no utiliza los datos enviados a través de la API para entrenar sus modelos salvo que el usuario lo consienta de forma explícita, lo que es relevante para el principio de limitación de la finalidad del artículo 5.1.b del RGPD [53]. Conviene conservar copia del DPA suscrito con OpenAI como evidencia del cumplimiento de estas obligaciones.

En quinto lugar, el principio de *privacy by design* y *privacy by default*, recogido en el artículo 25 del RGPD [53], exige tener en cuenta la privacidad desde el diseño del sistema y no como una medida reactiva. En FinSight esto se traduce en reducir al mínimo los datos que se guardan en los logs, fijar para esos logs períodos de retención definidos, prescindir de *cookies* de seguimiento en la versión inicial y no incorporar ningún dato personal más allá del estrictamente necesario para operar el servicio.

Por último, el principio de responsabilidad proactiva (*accountability*), establecido en el artículo 5.2 del RGPD [53], obliga al responsable del tratamiento a demostrar de forma activa que cumple. En la práctica, esto supone mantener un registro de actividades de tratamiento, elaborar una política de privacidad accesible y, según la escala del servicio, realizar una evaluación de impacto relativa a la protección de datos (EIPD) cuando el tratamiento presente riesgos elevados para los derechos de los interesados, conforme al artículo 35 del RGPD [53].

La Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [22], complementa el RGPD dentro del ordenamiento jurídico español. En el caso de FinSight, su relevancia práctica se centra en dos puntos: la obligación de publicar una política de privacidad acorde con sus disposiciones y la correcta gestión de los datos de posibles usuarios menores de catorce años, para los que el artículo 7 de la LOPDGDD exige el consentimiento del titular de la patria potestad o tutela [22].

2.7.3. Reglamento de Inteligencia Artificial (AI Act)

El Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, relativo a la inteligencia artificial (AI Act) [55], es el primer marco jurídico integral que regula los sistemas de inteligencia artificial en la Unión Europea, y se aplica directamente a FinSight, ya que el sistema integra modelos de lenguaje de gran escala (LLM) a través de OpenAI, la Groq API u Ollama, y modelos de aprendizaje profundo (GRU) para la predicción de tendencias de precio.

Según la clasificación de riesgo del Título III del AI Act [55], el sistema se sitúa en la categoría de riesgo limitado. El motivo es que no toma decisiones autónomas que afecten a derechos fundamentales, no evalúa la solvencia de personas físicas en el sentido del Anexo III [55], no elabora perfiles individuales de los usuarios y no se utiliza en ninguno de los contextos de alto riesgo que enumera el reglamento, como la selección de personal, la administración pública o el diagnóstico médico.

No obstante, dado que FinSight genera contenido informativo de carácter financiero mediante IA y lo dirige al público general, el artículo 50 del AI Act le impone una

obligación de transparencia [55]: el sistema debe informar a los usuarios, de forma clara e inequívoca, de que el contenido que reciben ha sido generado total o parcialmente por un sistema de inteligencia artificial. Esta obligación ya está recogida en los requisitos RF-06, RF-37 y RF-38 del documento SRS del proyecto.

Por último, de cara a una puesta en producción real, el AI Act también establece que los proveedores de modelos de IA de propósito general deben mantener documentación técnica suficiente sobre los modelos que distribuyen [55]. Esa obligación recae principalmente sobre el proveedor del modelo, OpenAI, Meta Platforms a través de la Groq API u Ollama; sin embargo, quien integra el servicio debe llevar un registro de los modelos empleados, sus versiones y las finalidades para las que se utilizan, como parte de la documentación técnica del sistema.

2.7.4. Directiva sobre mercados de instrumentos financieros (MiFID II)

La Directiva 2014/65/UE del Parlamento Europeo y del Consejo, relativa a los mercados de instrumentos financieros (MiFID II) [50], junto con su normativa de desarrollo [9], se aplica en la medida en que FinSight genera y difunde análisis automáticos sobre instrumentos financieros cotizados accesibles al público general.

La plataforma no presta servicios de inversión regulados en el sentido del Anexo I, Sección A de MiFID II [50]: no ejecuta órdenes por cuenta de clientes, no gestiona carteras ni actúa como asesor de inversiones. Tampoco constituye investigación en materia de inversiones en el sentido del artículo 36 del Reglamento Delegado (UE) 2017/565 [9]. No obstante, la difusión pública de análisis automatizados sobre valores, que puede influir en las decisiones de inversión de usuarios indeterminados, sí somete al sistema a las disposiciones de MiFID II sobre la información financiera general dirigida al público [50].

Por consiguiente, el sistema debe garantizar que toda la información analítica presentada al usuario vaya acompañada de un aviso legal explícito, permanente y visualmente destacado que indique que el contenido es de carácter exclusivamente informativo, ha sido generado por sistemas de inteligencia artificial y no constituye asesoramiento fi-

nanciero, recomendación de inversión ni oferta de compra o venta de valores. Además, los términos y condiciones del servicio deben delimitar de forma expresa la responsabilidad del prestador respecto a las decisiones que el usuario adopte a partir del contenido analítico recibido. Estas exigencias quedan recogidas en los requisitos RNF-31, RNF-32 y RF-06 del documento SRS.

2.7.5. Normativa de accesibilidad web

El Real Decreto 1112/2018, de 7 de septiembre, sobre accesibilidad de los sitios web y aplicaciones para dispositivos móviles del sector público [42], transpone la Directiva (UE) 2016/2102 al ordenamiento español [52] y fija el nivel AA de las Pautas de Accesibilidad para el Contenido Web (WCAG) versión 2.1 como estándar de conformidad exigible [62]. Aunque su ámbito de aplicación directo se limita al sector público, el nivel WCAG 2.1 AA se ha convertido en la referencia de facto para cualquier servicio web de acceso público en España. Su exigibilidad general se apoya, además, en el principio de no discriminación que reconoce el Real Decreto Legislativo 1/2013, de 29 de noviembre, Texto Refundido de la Ley General de derechos de las personas con discapacidad y de su inclusión social [43].

Los criterios de conformidad WCAG 2.1 AA que el sistema debe satisfacer se concretan en los requisitos no funcionales RNF-14 a RNF-19 del SRS [62]. Entre ellos están los ratios de contraste de color iguales o superiores a 4,5:1 para el texto normal y 3:1 para el texto de tamaño grande o los componentes de interfaz, la navegación completa por teclado con indicador de foco visible, los textos alternativos para todo el contenido no textual informativo, una estructura HTML semántica con un uso correcto de los elementos `h1` a `h6` y de los *landmark roles* de ARIA [63], y los mensajes de progreso y error perceptibles mediante tecnologías de asistencia.

2.7.6. Licencias de software de terceros

El uso de componentes de software de terceros distribuidos bajo licencias de código abierto genera obligaciones jurídicas derivadas del derecho de autor, y el proyecto debe cumplirlas tanto si tiene carácter académico como comercial. Por defecto, todo código

fuente está protegido por copyright, y su uso sin una licencia expresa está prohibido conforme al régimen general de propiedad intelectual aplicable al software [37].

El *stack* tecnológico de FinSight incorpora componentes bajo tres licencias: MIT (React, Vite, FastAPI, Tailwind CSS), BSD-3-Clause (PyTorch) y Apache 2.0 (varias dependencias del ecosistema Python) [4, 45, 46]. Las licencias MIT y BSD-3-Clause son permisivas y permiten usar, modificar e integrar el código en proyectos propios, incluidos los de carácter comercial, con la única condición de conservar el aviso de copyright original en el código fuente o en la documentación distribuida [45, 46]. La licencia Apache 2.0, también permisiva, añade una cláusula de patentes que protege al usuario frente a reclamaciones de patentes por parte del autor original [4].

El modelo Llama 3.1 de Meta Platforms se distribuye bajo la Llama 3 Community License, que permite su uso en aplicaciones de terceros con ciertas restricciones sobre el uso comercial a partir de determinados umbrales de usuarios activos mensuales y con requisitos de atribución [35]. Como el sistema accede al modelo únicamente a través de la Groq API, es el contrato de servicio de Groq el que regula esta relación [18] y traslada al proveedor las obligaciones de cumplimiento derivadas de la licencia del modelo base. Aun así, el integrador debe comprobar que el volumen de uso del servicio no supera los umbrales que activarían obligaciones adicionales por dicha licencia [35].

El uso de la OpenAI API se rige por un régimen jurídico distinto del de las licencias de código abierto anteriores. Los modelos de lenguaje y de generación de *embeddings* de OpenAI no se distribuyen como *open source*: su acceso queda sujeto a los Términos de Servicio de OpenAI y a las Políticas de Uso Aceptable asociadas, cuyo cumplimiento es un requisito contractual para utilizar el servicio [47, 49]. En FinSight, la OpenAI API se emplea en dos ámbitos: como proveedor LLM alternativo para la generación de análisis en lenguaje natural y como servicio de generación de *embeddings* semánticos para el módulo GraphRAG de análisis de sentimiento. Las implicaciones de este último uso en materia de protección de datos se tratan en la subsección correspondiente al RGPD.

Por último, se incluye en el repositorio del proyecto un fichero **LICENSES** que documenta las licencias de todas las dependencias principales junto con sus avisos de copyright correspondientes.

2.7.7. Normativa de aplicación indirecta o condicionada al crecimiento del servicio

Algunos marcos normativos se aplican al proyecto solo de forma indirecta o potencial en su configuración actual, pero conviene tenerlos en cuenta a medida que la plataforma crezca o cambie su modelo de prestación.

El **Cyber Resilience Act** (Reglamento (UE) 2024/2847), en vigor desde diciembre de 2024 con períodos de transición progresivos, establece requisitos de seguridad por diseño para los productos con elementos digitales que se comercializan en el mercado interior europeo [56]. Su ámbito principal son el software y el hardware distribuidos como producto, mientras que la situación de los servicios de software como servicio (SaaS) tiene una delimitación regulatoria todavía en evolución. Aun así, el principio de seguridad desde el diseño (*security by design*) que recoge el reglamento, en particular la prohibición de distribuir productos con vulnerabilidades conocidas sin parchear y la obligación de proporcionar actualizaciones de seguridad, sirve al sistema como referencia de buenas prácticas de ingeniería [56].

La **Directiva NIS2** (Directiva (UE) 2022/2555) se dirige a los operadores de servicios esenciales e importantes en sectores críticos [54]. FinSight no opera en ninguno de los sectores esenciales que enumera el Anexo I de la directiva ni alcanza, en su dimensión actual, los umbrales de tamaño empresarial fijados para los sectores del Anexo II [54]. Sin embargo, las medidas técnicas y organizativas que NIS2 impone, como la gestión de riesgos de ciberseguridad, la autenticación robusta, el cifrado, los planes de respuesta a incidentes y la auditoría de la cadena de suministro, son el estándar de diligencia debida que cualquier servicio web público de acceso general debería observar, al margen de que le resulte formalmente obligatorio [54].

Si la plataforma evolucionara hacia un modelo de negocio con suscripción de pago, sería plenamente aplicable el **Texto Refundido de la Ley General para la**

Defensa de los Consumidores y Usuarios (Real Decreto Legislativo 1/2007, de 16 de noviembre), que regula los derechos de los consumidores en la contratación de servicios a distancia [41]. Entre esos derechos están el de desistimiento en un plazo de catorce días naturales, la obligación de ofrecer información precontractual clara y la transparencia en la comunicación de precios y condiciones del servicio [41].

2.7.8. Normativa no aplicable: justificación de exclusiones

El **Esquema Nacional de Seguridad** (Real Decreto 311/2022, de 3 de mayo) establece los requisitos de seguridad para los sistemas de información del sector público y de las entidades privadas que le prestan servicios tecnológicos [36]. Como FinSight no presta servicios a ningún organismo de la Administración Pública, este esquema no le obliga. Sus cinco dimensiones de seguridad, que son la confidencialidad, la integridad, la trazabilidad, la autenticidad y la disponibilidad, sí se han tomado como referencia de buenas prácticas en el diseño del sistema [36].

La **Ley Orgánica 7/2021**, de 26 de mayo, de protección de datos personales tratados para fines de prevención, detección, investigación y enjuiciamiento de infracciones penales, no se aplica al sistema, ya que FinSight no trata datos con ninguna de esas finalidades [25].

La **Ley 25/2007, de 18 de octubre, de conservación de datos relativos a las comunicaciones electrónicas y a las redes públicas de comunicaciones** obliga a conservar los metadatos de tráfico de las comunicaciones, pero dirige esa obligación a los operadores de comunicaciones electrónicas y a los proveedores de redes públicas, como los operadores de acceso a Internet [21]. FinSight no presta servicios de comunicaciones electrónicas ni explota una red pública, así que no le corresponde ese deber de conservación. Conviene aclarar este punto porque el sistema sí registra direcciones IP en los logs del servidor: ese tratamiento se rige por el RGPD, ya analizado, y no por la Ley 25/2007 [21].

El **Real Decreto-ley 19/2018, de 23 de noviembre, de servicios de pago y otras medidas urgentes en materia financiera**, que traspone al ordenamiento español la Directiva (UE) 2015/2366 (PSD2), regula la actividad de los proveedores

de servicios de pago [23,51]. FinSight no procesa pagos ni mueve fondos, de modo que esta norma no le afecta. Incluso en el escenario futuro de un modelo con suscripción de pago, el cobro se delegaría en una pasarela de pago externa, sobre la que recaerían las obligaciones derivadas de esta normativa, y no sobre la plataforma [23].

Las disposiciones sobre **facturación y registro de operaciones económicas** tampoco afectan a FinSight en ninguna de sus iteraciones, porque la plataforma no procesa transacciones económicas, no genera registros de facturación ni emite documentos con trascendencia tributaria. En concreto, quedan fuera los siguientes marcos normativos:

- **Ley 11/2021, de 9 de julio, de medidas de prevención y lucha contra el fraude fiscal** (*Ley Antifraude*) [24], y su desarrollo reglamentario mediante el Real Decreto 1007/2023 [40] y la Orden HAC/1177/2024 [39], que regulan el sistema VERI*FACTU. Este marco busca garantizar la integridad e inalterabilidad de los registros de facturación, ya sea remitiendo los datos a la Agencia Estatal de Administración Tributaria (AEAT) o empleando sistemas informáticos que cumplan los requisitos técnicos establecidos. No se aplica a FinSight por dos motivos: la plataforma no realiza operaciones sujetas a facturación y, además, la obligación de usar VERI*FACTU en el software de facturación no entra en vigor hasta el 1 de enero de 2027.
- **Ley 18/2022, de 28 de septiembre, de creación y crecimiento de empresas** (*Ley Crea y Crece*) [26], que obliga a emitir factura electrónica en las relaciones comerciales entre empresarios y profesionales (B2B). Esta norma queda fuera porque FinSight no mantiene relaciones comerciales de facturación con otras entidades en el ejercicio de su actividad.
- **Digitalización certificada de facturas en papel** (Orden EHA/962/2007, de 10 de abril [38], y Resolución de 24 de octubre de 2007 del Departamento de Informática Tributaria de la AEAT [1]), que regula la sustitución de documentos en papel por copias digitales con plena validez tributaria, previa homologación del sistema de digitalización. Como la plataforma no genera ni gestiona documentos tributarios en papel, esta normativa tampoco le resulta aplicable.

2.7.9. Resumen de disposiciones normativas

La tabla 2.14 presenta un resumen de las disposiciones normativas identificadas, su ámbito de aplicación y el grado de exigibilidad para el sistema en su versión de lanzamiento.

Tabla 2.14: Resumen de disposiciones normativas aplicables a FinSight.

Normativa	Relevancia para FinSight	Exigibilidad
LSSI-CE (Ley 34/2002) [20]	Aviso legal, identificación del prestador, política de <i>cookies</i>	Directa y obligatoria
RGPD (Regl. UE 2016/679) [53]	Tratamiento de IPs en logs, derechos de usuarios, transferencias internacionales, <i>privacy by design</i>	Directa y obligatoria
LOPDGDD (LO 3/2018) [22]	Adaptación española del RGPD; consentimiento de menores; política de privacidad	Directa y obligatoria
AI Act (Regl. UE 2024/1689) [55]	Transparencia sobre contenido generado por IA; clasificación de riesgo limitado	Directa y obligatoria
MiFID II (Dir. 2014/65/UE) [50]	Disclaimer financiero; no asesoramiento; atribución de contenido analítico	Directa y obligatoria
WCAG 2.1 AA [62] / RD 1112/2018 [42]	Accesibilidad de la interfaz web	Directa y obligatoria

Continúa en la página siguiente

Normativa	Relevancia para FinSight	Exigibilidad
Licencias <i>open source</i> (MIT, BSD, Apache 2.0, Llama 3) [4, 35, 45, 46] y OpenAI ToS + DPA [47, 48]	Atribución de copyright; condiciones de uso comercial; acuerdo de procesamiento de datos para <i>embeddings</i>	Directa y obligatoria
Cyber Resilience Act (Regl. UE 2024/2847) [56]	<i>Security by design</i> ; actualizaciones de seguridad	Indirecta (referencia de buenas prácticas)
NIS2 (Dir. UE 2022/2555) [54]	Medidas técnicas de ciberseguridad	Indirecta (no operador de servicio esencial)
LGDCU (RDL 1/2007) [41]	Derechos del consumidor en contratación a distancia	Condicionada a modelo de pago
ENS (RD 311/2022) [36]	Seguridad en sistemas de información del sector público	No aplicable (no sector público)
LO 7/2021 (datos penales) [25]	Tratamiento de datos con fines de prevención e investigación penal	No aplicable (sin finalidad penal)
Ley 25/2007 (conservación de datos de comunicaciones) [21]	Conservación de metadatos por operadores de comunicaciones electrónicas	No aplicable (no operador ni red pública)
RDL 19/2018 y PSD2 [23, 51]	Servicios de pago y proveedores de servicios de pago	No aplicable (sin procesamiento de pagos)

Continúa en la página siguiente

Normativa	Relevancia para FinSight	Exigibilidad
Ley Antifraude (Ley 11/2021) [24] y VE-RI*FACTU (RD 1007/2023 [40], Orden HAC/1177/2024 [39])	Integridad de registros de facturación	No aplicable (sin transacciones económicas; en vigor desde 2027)
Ley Crea y Crece (Ley 18/2022) [26]	Factura electrónica obligatoria B2B	No aplicable (sin facturación)
Digitalización Certificada (Orden EHA/962/2007 [38])	Copias digitales con validez tributaria	No aplicable (sin documentos tributarios)

Capítulo 3

Solución

Este capítulo abarca el estudio y planteamiento de la solución, pasando por el proceso de desarrollo, el cual incluye no solo la implementación sino también los procesos previos que justifican y respaldan las decisiones tomadas. Termina con la presentación del producto final, es decir, lo que recibe el cliente.

3.1. Descripción de la solución

La solución adoptada ha sido la realización de una aplicación web de acceso público que ofrece análisis de acciones bajo demanda. La aplicación contiene un buscador de acciones, que permitirá encontrar compañías que cotizan públicamente tanto por su nombre como por su ticker. El resultado es una página donde se desglosa y se explica en lenguaje natural el proceso y resultado de cada una de las caras que conforman el prisma del análisis integral.

La aplicación web es solamente la fachada utilizada para presentar las tecnologías que hay detrás, las cuales corresponden a la faceta verdaderamente interesante de este proyecto. El adjetivo integral de este análisis queda respaldado por los distintos puntos de vista desde los que se afronta la situación de una empresa o activo. El informe final que recibe el usuario está compuesto por el análisis de sentimiento de las noticias financieras relacionadas con la compañía seleccionada, la predicción de la tendencia

del precio de un activo basada en los datos históricos de esa acción, y por último, dos enfoques más clásicos, que corresponden con el análisis técnico y fundamental.

La implementación del análisis de sentimiento se ha llevado a cabo siguiendo la técnica GraphRAG. Se aprovecha de la capacidad de los modelos extensos de lenguaje (LLM) de procesar lenguaje natural para analizar el sentimiento de una noticia. Además, se mitigan los posibles errores o alucinaciones de estos modelos mediante la adición de contexto relevante extraído de una base de datos de grafos, de ahí el nombre Graph, grafo en inglés, RAG, *Retrieval-Augmented Generation*. La ventaja de estos sistemas frente a los *Naive RAG* reside en la riqueza de relaciones existentes en la base de datos del contexto, lo que se traduce en proporcionarle contexto más relevante al modelo, mejorando notablemente su rendimiento en tareas de clasificación como la que nos compete [17].

Por otra parte, para la predicción basada en los datos históricos de un activo se emplean redes neuronales. En primer lugar, se realizaron pruebas utilizando datos OHLC, que incluyen el precio de apertura, el más alto, el más bajo, y el precio de cierre de cada sesión bursátil desde que comenzó a cotizar el activo hasta marzo de 2026. Se compararon varios tipos de redes, en busca de la que mejor rendimiento general diera, entrenando cada una de ellas para tres empresas de distinto sector y capitalización. Con este experimento no se busca afinar el rendimiento para una sola empresa, sino encontrar los parámetros que mejor resultado den para el universo de acciones que contemplamos analizar. También se tuvo en cuenta el tiempo de entrenamiento, pues la aplicación web debe ofrecer resultados en un tiempo comedido. Estos datos fueron limpiados, y también se contempló realizar una descomposición del precio de cierre, antes de alimentar las redes neuronales. En este apartado nos encontramos una de las restricciones del proyecto, la limitada cantidad de datos financieros de calidad gratuitos disponible. Una vez se decidió qué modelo ofrecía el mejor rendimiento, habiendo optimizado los parámetros en la medida de lo posible para un universo heterogéneo de empresas, y teniendo en cuenta el tiempo de respuesta, se implementó el pipeline que, bajo demanda del usuario, descarga datos de la API yfinance, con los cuales se entrena la red y se genera la predicción que ve el usuario.

Por último, se implementa el análisis técnico y fundamental. Ambos fueron desarrollados de forma independiente por el asesor financiero colaborador, contemplado en el personal del proyecto. Una vez diseñados e implementados, fueron integrados al sistema completo como módulos, tal y como manda la arquitectura escogida. A continuación se explican brevemente las estrategias adoptadas:

- El análisis fundamental evalúa la “salud” y el valor intrínseco de una empresa dividiendo el estudio en cuatro áreas clave, cada una ponderada con pesos específicos: Valoración (30 %) examina si la empresa está cara o barata comparando cuatro tipos de rentabilidades (EBITDA, beneficios, flujo de caja libre y patrimonio) contra todas las empresas de su sector; Calidad (30 %) mide la consistencia y robustez de los beneficios históricos para evaluar si la empresa es estable; Crecimiento (20 %) analiza si los ingresos y beneficios están aumentando respecto a años anteriores; y Solvencia (20 %) verifica que la empresa no tenga demasiada deuda y pueda pagar sus obligaciones. Todos estos indicadores se normalizan de forma “relativa”, es decir, se comparan con las otras empresas del universo (S&P 500¹ o MSCI World²) y con el sector específico de la empresa (por ejemplo, tecnología o finanzas), de modo que el resultado final, comprendido entre 0 y 10, muestra si la empresa es mejor o peor que su competencia, y no un número absoluto. Si algún área no dispone de suficientes datos, su peso se redistribuye automáticamente entre las demás para que la puntuación siga siendo comparable.
- El análisis técnico estudia el comportamiento reciente del precio y el volumen de la acción para identificar tendencias y señales de compra o venta. Combina cuatro bloques independientes: Momentum (35 %) mide si el precio ha subido más rápido que el de sus competidores en los últimos meses, ajustando por la volatilidad (oscilaciones) de la acción; Tendencia (30 %) verifica si la dirección estructural del precio es alcista (subiendo) o bajista (bajando) mediante técnicas de regresión; Riesgo/Estabilidad (20 %) cuantifica cuán volátil o arriesgada es la acción comparando sus oscilaciones con las del sector; y Confirmación (15 %) busca señales adicionales, como divergencias de volumen, que corroboren las tendencias

¹Cotización del S&P 500: <https://es.investing.com/indices/us-spx-500>

²Cotización del MSCI World: <https://es.investing.com/indices/msci-world>

detectadas. Al igual que en el análisis fundamental, todos estos indicadores se normalizan con respecto al universo y al sector de referencia, transformando el resultado en una puntuación entre 0 y 10. Finalmente, el motor combina ambos análisis en una puntuación final única (60 % fundamental y 40 % técnico), que produce una recomendación de tres niveles: “atractiva” (puntuación ≥ 7), “neutral/seguimiento” (puntuación entre 5 y 7) o “poco atractiva” (puntuación < 5).

El análisis está sujeto a la disponibilidad de datos, cuantos más y de mejor calidad haya, mejor será el resultado de cada uno de los cuatro aspectos que conforman el informe final.

3.2. El proceso de desarrollo

El proceso de desarrollo seguido fue de carácter iterativo e incremental. Partió de una idea inicial que se refinó a partir del estudio del problema, de las tecnologías disponibles y de las necesidades detectadas, y que se plasmó en un documento de especificación de requisitos abierto a revisiones sucesivas conforme avanzaba el trabajo. Sobre ese análisis se definió el diseño del sistema (arquitectura, herramientas y tecnologías empleadas) y se construyó la lógica en módulos desarrollados de forma independiente, entre los que tuvieron especial relevancia los conjuntos de datos utilizados para poblar el grafo de conocimiento del sistema GraphRAG y para entrenar las redes neuronales recurrentes. La construcción del grafo exigió definir previamente las relaciones de interés y un cliente de acceso al modelo de lenguaje, provisto por OpenAI, del que dependen tanto la generación del grafo como la aplicación final. La elección de la arquitectura de red se resolvió de forma experimental, contrastando distintos tipos de redes recurrentes conforme a la literatura revisada; una vez adoptada esa decisión, se implementó la aplicación web con sus módulos y se validó mediante pruebas empíricas y *tests* estandarizados, con el fin de verificar la calidad del producto y el grado de cumplimiento de los objetivos.

3.2.1. Análisis

La fase de análisis tuvo como objetivo transformar la idea inicial de FinSight en un conjunto de necesidades verificables, acotadas y trazables. Para ello se elaboró una Especificación de Requisitos de Software siguiendo la estructura IEEE 830, en la que se documentaron el propósito del sistema, su alcance, las interfaces externas, las restricciones tecnológicas, los supuestos de partida y los requisitos específicos que debían guiar el diseño posterior.

Más allá de enumerar las funcionalidades visibles para el usuario, el análisis sirvió también para delimitar el comportamiento esperado de los módulos internos de inteligencia artificial, el uso de proveedores externos, las condiciones de rendimiento, las obligaciones legales y las restricciones derivadas del despliegue. El cuidado en esta fase era importante porque FinSight integra fuentes heterogéneas de información financiera, modelos de aprendizaje profundo, un grafo de conocimiento, servicios LLM y una interfaz web pública, de modo que cualquier ambigüedad en los requisitos podía arrastrarse hasta la arquitectura y la evaluación final del sistema.

En esta etapa también se fijaron criterios de prioridad para separar los requisitos imprescindibles de los importantes pero condicionados por el calendario y de las mejoras futuras. Así, el desarrollo pudo centrarse primero en las capacidades esenciales: selección de acciones, generación del informe, ejecución de los cuatro módulos de análisis, seguridad básica, accesibilidad y cumplimiento normativo.

Definición de requisitos

Los requisitos del sistema se dividieron en requisitos funcionales, que describen los servicios que FinSight debe ofrecer, y requisitos no funcionales, que establecen condiciones de calidad, restricciones técnicas y obligaciones de cumplimiento. En el documento SRS, cada requisito recibió un código único, **RF-XX** para los funcionales y **RNF-XX** para los no funcionales, y se clasificó con prioridad alta, media o baja según su importancia para la primera iteración del producto.

Los requisitos funcionales se organizaron por módulos, de forma coherente con la arquitectura prevista para la aplicación. A continuación se listan con su código y su nombre corto; el detalle completo de cada uno figura en el documento SRS.

Interfaz y navegación

- RF-01: catálogo de acciones.
- RF-02: búsqueda de acciones.
- RF-03: solicitud de análisis.
- RF-04: indicador de progreso.
- RF-05: forzar actualización.
- RF-06: disclaimer legal permanente.
- RF-07: política de privacidad.
- RF-08: términos de uso.

Análisis de sentimiento

- RF-09: extracción de noticias en tiempo real.
- RF-10: selección por relevancia.
- RF-11: búsqueda de similitud en el grafo.
- RF-12: expansión por vecindad k-hop.
- RF-13: clasificación de sentimiento por LLM.
- RF-14: resultado tripartito de sentimiento.
- RF-15: explicación en lenguaje natural.
- RF-16: listado de noticias influyentes.
- RF-17: caché de análisis de sentimiento.

Predicción de tendencia (Deep Learning)

- RF-18: disponibilidad de modelos entrenados.

- RF-19: carga *lazy* con caché LRU.
- RF-20: construcción de la ventana de entrada en inferencia.
- RF-21: predicción de precio futuro.
- RF-22: determinación de tendencia.
- RF-23: metadatos del modelo en la respuesta.
- RF-24: pipeline de actualización diaria automática.
- RF-25: uso exclusivo de datos reales.
- RF-26: registro del pipeline de actualización.
- RF-27: notificación de soporte parcial.

Análisis fundamental

- RF-28: obtención de datos fundamentales.
- RF-29: análisis en lenguaje natural.
- RF-30: caché de datos fundamentales.

Análisis técnico

- RF-31: obtención de datos para análisis técnico.
- RF-32: cálculo de indicadores técnicos.
- RF-33: señal de tendencia técnica.
- RF-34: caché del análisis técnico.

Generación de informe

- RF-35: síntesis global por LLM.
- RF-36: estructura ordenada del informe.
- RF-37: disclaimer prominente en el informe.
- RF-38: atribución de IA explícita.
- RF-39: elementos visuales basados en datos.

- RF-40: exportación a PDF.

Los requisitos no funcionales se agruparon por categorías de calidad y cumplimiento, con el mismo esquema de codificación.

Rendimiento

- RNF-01: tiempo máximo de generación del informe.
- RNF-02: tiempo de inferencia GRU.
- RNF-03: tiempo de carga de la interfaz.
- RNF-04: reducción de latencia mediante caché.
- RNF-05: gestión de memoria de modelos.

Seguridad

- RNF-06: HTTPS obligatorio en producción.
- RNF-07: protección de claves API.
- RNF-08: rate limiting en la API.
- RNF-09: validación y sanitización de entradas.
- RNF-10: configuración CORS restrictiva.
- RNF-11: protección XSS en el frontend.
- RNF-12: cabeceras de seguridad HTTP.
- RNF-13: dependencias sin vulnerabilidades.

Accesibilidad

- RNF-14: conformidad WCAG 2.1 nivel AA.
- RNF-15: ratios de contraste de color.
- RNF-16: navegación completa por teclado.
- RNF-17: textos alternativos.
- RNF-18: estructura HTML semántica.

- RNF-19: mensajes de estado accesibles.

Mantenibilidad

- RNF-20: arquitectura modular por dominio.
- RNF-21: servicio LLM centralizado (Adaptador).
- RNF-22: configuración externalizada.
- RNF-23: documentación de código.
- RNF-24: registro de logs estructurado.
- RNF-25: código limpio y convenciones.
- RNF-40: interfaz común de módulos de análisis (Estrategia).
- RNF-41: punto de entrada único de informes (Fachada).
- RNF-42: proveedor de datos de mercado abstracto (Adaptador).
- RNF-43: cadena de obtención de noticias (Cadena de Responsabilidad).

Portabilidad

- RNF-26: compatibilidad de navegadores.
- RNF-27: independencia del sistema operativo.
- RNF-28: portabilidad mediante variables de entorno.
- RNF-29: ejecución autónoma del pipeline.
- RNF-30: cold start documentado.

Legal

- RNF-31: política de privacidad (RGPD).
- RNF-32: aviso legal financiero (MiFID II).
- RNF-33: atribución y enlace a fuentes.
- RNF-34: términos y condiciones de uso.
- RNF-35: cookie de preferencia de consentimiento.

- RNF-36: banner de consentimiento granular.
- RNF-37: cookies funcionales de personalización.
- RNF-38: cookies analíticas de medición de uso.
- RNF-39: equivalencia legal del almacenamiento en cliente.

Especificación de requisitos

La especificación de requisitos concretó cómo debía comportarse FinSight ante las principales interacciones del usuario y ante las operaciones internas del sistema. Desde el punto de vista externo, la aplicación se definió como una SPA accesible desde navegadores modernos, orientada inicialmente a escritorio y con todos los textos en español. El usuario no necesita registrarse ni instalar software adicional: accede a la web, busca una empresa, selecciona una acción y solicita el análisis. Durante el procesamiento, la interfaz debe mostrar retroalimentación visual y advertir de que la generación completa puede tardar hasta 60 segundos.

El flujo funcional principal comienza con la selección de una acción del catálogo. Si la acción tiene soporte completo, el sistema ejecuta los cuatro módulos de análisis; si no dispone de modelo GRU entrenado, debe ofrecer soporte parcial y comunicarlo con claridad, sin impedir la consulta del resto de módulos. La generación del informe se plantea como una operación que orquesta el backend: los módulos se ejecutan en paralelo cuando es posible, se recogen sus resultados normalizados y después se invoca al servicio LLM para producir una conclusión global en español.

La estructura del sistema queda condicionada por las interfaces externas especificadas. El frontend React se comunica con el backend FastAPI mediante una API REST bajo el esquema `/api/v1/{recurso}` y con JSON como formato de intercambio. Las llamadas a servicios externos, como NewsAPI, Finnhub, Yahoo Finance, OpenAI o Neo4j AuraDB, se hacen exclusivamente desde el backend para no exponer las credenciales. En producción, toda comunicación cliente-servidor debe emplear HTTPS con certificado TLS válido, y CORS debe aceptar únicamente el dominio autorizado del frontend.

En el módulo GraphRAG, la especificación exige combinar las noticias recientes con el conocimiento histórico almacenado en un grafo. Para cada noticia obtenida se buscan nodos similares mediante embeddings, se expande el contexto hasta una vecindad configurable y se entrega esta información al LLM para obtener una clasificación explicada. Con esta definición, el análisis de sentimiento queda documentado de principio a fin: además de la etiqueta, la salida incluye una puntuación, un nivel de confianza y las noticias que más han influido en el resultado.

En el módulo de Deep Learning, la especificación distingue con claridad entre entrenamiento e inferencia. El entrenamiento se realiza de forma periódica mediante un pipeline diario gestionado con APScheduler, que obtiene datos EOD, recalcula los modelos y registra la fecha y las métricas en ficheros de metadatos. La inferencia, en cambio, se ejecuta bajo demanda y debe reutilizar los modelos ya cargados en memoria mediante una caché LRU para cumplir los requisitos temporales en un servidor de producción sin GPU. La salida del módulo no es un valor numérico aislado: se expresa como una tendencia interpretable que pueda integrarse en el informe final.

Los módulos fundamental y técnico se especifican como análisis complementarios. El primero traduce los indicadores financieros a lenguaje natural mediante el LLM centralizado, mientras que el segundo calcula señales a partir de indicadores clásicos de mercado. Ambos deben apoyarse en caché porque dependen de datos que no requieren actualización continua durante la sesión, lo que reduce la latencia, el consumo de APIs externas y el coste de las llamadas al LLM.

La especificación también establece cómo debe comportarse el sistema ante fallos parciales. Si un proveedor externo no responde, si se alcanza una cuota o si un módulo no puede completarse, la arquitectura debe permitir una degradación elegante: informar del error, conservar los resultados disponibles y evitar que el fallo de un componente inutilice todo el informe. Para ello se definieron contratos comunes entre módulos, abstracciones de proveedores y una fachada de generación de informes que oculta la coreografía interna al endpoint de la API.

Por último, los requisitos legales y de transparencia se integraron en el comportamiento funcional del producto con el mismo rango que cualquier otra funcionalidad.

FinSight debe mostrar de forma visible que el contenido es informativo, que puede haber sido generado por inteligencia artificial y que no constituye asesoramiento financiero ni recomendación de inversión. También debe atribuir las fuentes empleadas, ofrecer la política de privacidad y los términos de uso, y gestionar cualquier almacenamiento en el cliente conforme a la normativa de cookies y consentimiento aplicable.

3.2.2. Diseño del sistema

En la Fig. 3.1 se muestra un esquema del sistema desarrollado. El esquema representa la arquitectura del sistema propuesto para el análisis bursátil, integrando dos fuentes principales de datos. Por un lado, se emplea un conjunto de noticias junto con información de sentimiento, que es procesado mediante un grafo de conocimiento combinado con un modelo de lenguaje (LLM) implementado en Neo4j. Por otro lado, se utiliza un conjunto de datos financieros de acciones, que es analizado mediante una red neuronal GRU para capturar patrones temporales. Ambas salidas convergen en un módulo final de predicción, donde se fusiona la información semántica proveniente de las noticias y la dinámica histórica del mercado para generar la predicción del comportamiento de las acciones. Además, también se integran los dos enfoques clásicos del análisis de acciones, el análisis técnico y el análisis fundamental.

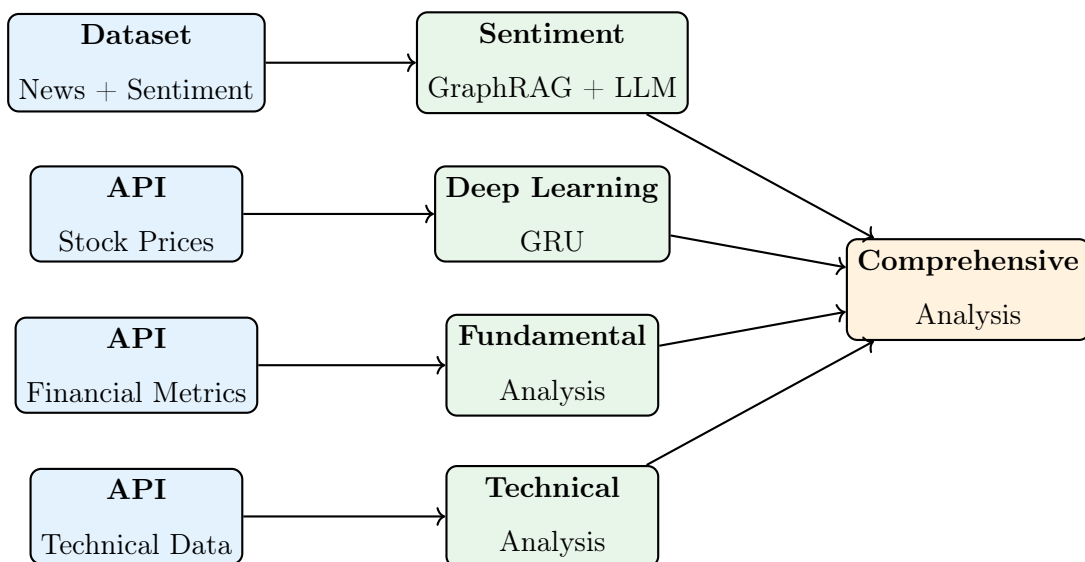


Figura 3.1: Esquema del sistema diseñado.

3.2.3. Dataset

En esta sección se trata la búsqueda, obtención y tratamiento de los datos empleados en la totalidad del proyecto.

Información para la creación del grafo de conocimiento.

El módulo de análisis de sentimiento se apoya en un sistema GraphRAG, de modo que la calidad de sus respuestas depende directamente de la del grafo de conocimiento que lo alimenta y, sobre todo, de cómo se ha asignado el sentimiento que cada noticia atribuye a cada activo. Cuando el usuario consulta una acción, el sistema recupera del grafo las noticias y relaciones más próximas y se las entrega al modelo de lenguaje como contexto. Si ese contexto parte de etiquetas dudosas, el modelo razonará sobre información poco fiable por bueno que sea. Por eso la elección de los datos no se ha tratado como un detalle de implementación, sino como una decisión de diseño: interesa un contexto rico y veraz, etiquetado por expertos o por hechos comprobables, y no por un modelo de lenguaje que puede equivocarse.

Se estudiaron tres conjuntos de datos. El primero, Financial PhraseBank, se empleó en una versión previa e independiente del sistema de análisis de sentimiento, usando también la arquitectura GraphRAG pero con modelos menos potentes en local y un grafo generado de dudosa calidad. Los otros dos, FinEntity y FinMarBa, son los que finalmente nutren el grafo actual. A continuación se describe el origen y la construcción de cada uno, junto con las razones que llevaron a descartar el primero.

Financial PhraseBank. Financial PhraseBank [34] es uno de los conjuntos de referencia para la clasificación de sentimiento financiero. La versión ampliada utilizada contiene 5.842 frases, cada una etiquetada como positiva, negativa o neutra. Las etiquetas las asignó un grupo de trece personas (diez estudiantes de máster y tres investigadores) según su interpretación subjetiva de las condiciones económicas descritas en cada frase. Durante el prototipo inicial el conjunto cumplió su cometido, pero al construir el grafo de conocimiento se hicieron evidentes dos problemas. El primero es que la etiqueta vive a nivel de frase completa, sin distinguir qué empresa concreta provoca el sentimiento, algo poco útil cuando un mismo texto menciona varios activos. El segundo, más grave

para este proyecto, es que las noticias son antiguas y están concentradas en el mercado nórdico: predominan compañías y referencias del norte de Europa, como se aprecia en frases del tipo «*Kone's net sales rose by some 14 % year-on-year in the first nine months of 2008*» (positiva) o «*Circulation revenue has increased by 5 % in Finland and 4 % in Sweden in 2008*» (positiva). Este sesgo regional, dominado por términos como «EUR», «Finland» o «Finnish», lo documentan también los autores de FinMarBa [29], que además ilustran cómo una etiqueta humana puede contradecir la reacción real del mercado. Para un sistema cuyo objetivo es analizar acciones del mercado global, un conjunto así aporta poco contexto aprovechable, de modo que se descartó para la versión final.

FinEntity. FinEntity [60] resuelve la primera limitación: en lugar de etiquetar la frase entera, anota el sentimiento a nivel de entidad. Se construyó a partir de noticias financieras de la base de datos Reuters (Refinitiv) y contiene 979 párrafos con 2.131 entidades anotadas, repartidas de forma equilibrada entre 503 positivas, 498 negativas y 1.130 neutras; alrededor del 60 % de los textos mencionan más de una entidad, lo que refleja el caso habitual de una noticia real. Cada ejemplo lo etiquetaron tres de los doce anotadores, todos ellos estudiantes de último curso de Finanzas o Empresa, y la consistencia se controló con medidas explícitas: un coeficiente de Jaccard de 0,754 sobre los límites de las entidades y un filtro de κ de Fleiss superior a 0,8 sobre el sentimiento, resolviendo los desacuerdos por mayoría. El resultado es un etiquetado de consenso, hecho por personas con criterio financiero y verificado entre sí. La Tabla 3.1 muestra un ejemplo del formato, en el que a cada entidad se le asocia su posición en el texto y su sentimiento. Conviene resaltar un hallazgo de los propios autores: un modelo ajustado sobre estas anotaciones supera a ChatGPT en configuración *zero-shot*, lo que confirma que para esta tarea un etiquetado humano y cuidado rinde mejor que un modelo generativo respondiendo por su cuenta. En el grafo, el sentimiento de FinEntity se traslada tal cual a las relaciones entre noticia y activo, y el modelo de lenguaje solo interviene después para una tarea secundaria, extraer temas y eventos, nunca para decidir el sentimiento.

Texto (fragmento)	Entidad	Sentimiento
<i>Johnson & Johnson shares gained 0.20 % after posting results that beat expectations...</i>	Johnson & Johnson	Positiva
<i>... Siemens is rallying 6 % after a boom in quarterly orders and packaging maker Huhtamaki is also up by 6 %...</i>	Siemens	Positiva
	Huhtamaki	Positiva

Tabla 3.1: Ejemplo del formato de FinEntity: sentimiento anotado por entidad. El segundo caso ilustra un párrafo con varias entidades.

FinMarBa. FinMarBa [29] ataca el segundo problema, el del sesgo y la cobertura, y lo hace cambiando por completo el origen de la etiqueta. Parte de los Bloomberg Market Wraps, los resúmenes diarios que redactan periodistas financieros de Bloomberg, y abarca de 2010 a 2024. De esos resúmenes se extraen titulares y, para cada titular, se identifican los *tickers* afectados; ambas tareas se delegan en GPT-4. La diferencia decisiva está en cómo se asigna el sentimiento: no lo decide ni una persona ni el modelo, sino el propio mercado. Para cada *ticker* se compara la variación de su precio al día siguiente de la noticia con la distribución de sus rendimientos de los cinco años anteriores, y se etiqueta como positivo si supera el percentil 60, negativo si queda por debajo del percentil 30 y neutro en caso contrario. La etiqueta es, por tanto, un hecho comprobable, la reacción observada del mercado, y no una opinión. El conjunto completo descrito por los autores reúne 61.252 titulares; el subconjunto publicado en abierto y empleado aquí contiene 8.142. Su cobertura es global aunque dominada por Estados Unidos, e incluye más de cinco mil *tickers* de renta variable, bonos, materias primas, divisas y criptomonedas. La Tabla 3.2 recoge varias filas reales; nótese el segundo ejemplo, en el que un mismo titular recibe sentimientos opuestos para dos *tickers* (negativo para el dólar, positivo para el índice global) porque sus precios reaccionaron en sentidos contrarios.

Fecha	Titular	Tickers	Sent.
2010-01-04	Oil Prices Surge Above \$81 a Barrel Due to U.S. Cold Weather	CL=F	+1
2010-01-04	Dollar Slumps Amid Worldwide Manufacturing Improvement	UUP ACWI	-1 +1
2010-01-04	S&P 500 Sees 1.6 Percent Increase, Hits 15-Month High	^GSPC	+1

Tabla 3.2: Ejemplo de FinMarBa: a cada titular se le asocian sus *tickers* y un sentimiento por *ticker* derivado de la reacción del precio (+1 positivo, 0 neutro, -1 negativo).

La ventaja de etiquetar por reacción de mercado no es solo conceptual. Frente al sesgo hacia lo neutro del PhraseBank, la distribución de FinMarBa se reparte de forma más realista entre las tres clases (Tabla 3.3) y recoge el ligero sesgo alcista propio de la renta variable a largo plazo. Los autores muestran además que una señal construida sobre estas etiquetas predice mejor el mercado: un mismo modelo BERT alcanza un ratio de Sharpe de 0,30 entrenado con FinMarBa, frente al -0,13 que obtiene entrenado con PhraseBank (el conjunto sobre el que se entrenó el conocido FinBERT [5]). Igual que en FinEntity, en nuestro grafo el sentimiento y la variación de precio de cada noticia se cargan directamente desde estos datos estructurados, sin pasar por el modelo de lenguaje.

Clase	PhraseBank (%)	FinMarBa (%)
Positiva	28,13	42,11
Negativa	12,46	31,43
Neutra	59,41	26,45

Tabla 3.3: Distribución de etiquetas en Financial PhraseBank y FinMarBa. La etiqueta de mercado reparte mejor las clases y refleja el sesgo alcista de la renta variable [29].

Tratamiento e integración. Los dos conjuntos finales son complementarios. FinEntity aporta granularidad por entidad y párrafos con varios activos a la vez; FinMarBa aporta amplitud, actualidad hasta 2024, cobertura global y, sobre todo, etiquetas ancla-

das en el comportamiento real del mercado. Antes de entrar en el grafo, cada conjunto pasa por su propio cargador, que normaliza el formato (los vanos y etiquetas de FinEntity, los diccionarios por *ticker* de FinMarBa) y traduce el sentimiento a la escala común $\{+1, 0, -1\}$. Los nombres de entidad y los *tickers* se resuelven contra yfinance para fijar el identificador y el tipo de activo; el texto de cada noticia se convierte en un vector con `text-embedding-3-large` (3.072 dimensiones) para la búsqueda semántica, y se evita la duplicación mediante un identificador derivado del propio texto. Con todo ello se construyen los nodos de activo, noticia, tema y evento, y las relaciones de mención que portan el sentimiento. El grafo resultante ofrece así el contexto que persigue el sistema: una base etiquetada por consenso de expertos o por hechos de mercado, lista para entregarse al modelo en el momento de la consulta.

Series de precios para el entrenamiento y optimización del módulo de *deep learning*

El módulo de *deep learning* predice la tendencia a corto plazo del precio de una acción, así que su materia prima son series históricas de cotización diaria de cierre (*end of day*). A diferencia del módulo de sentimiento, donde el problema estaba en cómo se asignaban las etiquetas, aquí el reto es doble: reunir un histórico amplio y limpio, y construir con él un protocolo experimental sin fuga de información que permita decidir con honestidad qué modelo usar y con qué parámetros.

Origen y composición. Los datos provienen del conjunto público *Nasdaq Daily Stock Prices*, publicado en Kaggle [59]. Reúne 3.347 ficheros CSV, uno por cada empresa del Nasdaq, con el mismo esquema: `ticker`, `date`, `open`, `high`, `low`, `close`. Cada fichero recoge el precio diario de una compañía, ya ajustado por *splits* y dividendos (el valor de Apple en 1980, 0,13 USD, refleja ese ajuste). El ejemplo de Apple (Tabla 3.4) abarca desde su salida a bolsa, el 12 de diciembre de 1980, hasta marzo de 2026, con 11.385 sesiones. Como cada modelo se entrena por separado para un único *ticker*, los ficheros no se fusionan entre sí: no existe un dataset combinado, sino 3.347 series independientes. Este formato coincide a propósito con el de producción, donde la aplicación descarga ese mismo histórico OHLC en vivo desde yfinance, también con

ajuste por *splits* y dividendos, de modo que el dato con el que se experimenta y el que se usa en explotación son equivalentes.

date	open	high	low	close
1980-12-12	0,13	0,13	0,13	0,13
...				
2026-03-25	254,10	255,00	251,60	252,62
2026-03-26	252,12	257,00	250,77	252,89
2026-03-27	253,90	255,49	248,07	248,80

Tabla 3.4: Muestra del fichero `AAPL.csv`: precio diario ajustado en formato OHLC. La columna `ticker` se omite por ser constante dentro de cada fichero.

Subconjunto experimental. De las 3.347 empresas disponibles, la investigación no necesitó todas. La comparación de modelos (*benchmark*) se hizo sobre ocho *tickers* elegidos para cubrir regímenes distintos de volatilidad e histórico, y la optimización de hiperparámetros se restringió a tres de ellos. La Tabla 3.5 recoge el subconjunto junto al error del modelo *naive* de referencia para cada serie, que da una idea de lo difícil que es predecir cada acción. La elección de un conjunto pequeño pero diverso responde a un motivo práctico: la optimización es costosa y, una vez fijada la receta, el modelo se reentrena por separado para cada *ticker* en producción, así que basta con que el subconjunto sea representativo.

Limpieza. Cada serie pasa por una limpieza mínima antes de usarse. Se ordenan las filas por fecha, se eliminan los valores ausentes y las fechas duplicadas, y se descartan las filas incoherentes con la definición de una vela (*candlestick*): aquellas en las que el mínimo no es el menor de los precios, el máximo no es el mayor, o algún precio no es positivo. No hay anotación manual de ningún tipo, a diferencia del módulo de sentimiento: la etiqueta no la asigna nadie, se deriva del propio precio futuro, como se ve más adelante.

```

1 df = df.sort_values("date").set_index("date")[FEATURES].dropna()
2 df = df[~df.index.duplicated(keep="first")]
3 valid = (

```


Ticker	Uso	RMSE <i>naive</i> (retorno 10 d, %)
AAPL	<i>benchmark + tuning</i>	5,76
NVDA	<i>benchmark + tuning</i>	13,71
PEP	<i>benchmark + tuning</i>	3,71
TSLA	<i>benchmark</i>	11,98
MSFT	<i>benchmark</i>	4,83
ABNB	<i>benchmark</i>	6,34
DDOG	<i>benchmark</i>	10,17
INTC	<i>benchmark</i>	8,73

Tabla 3.5: Subconjunto de *tickers* usado en la fase de investigación y el RMSE del *naive* (paseo aleatorio) sobre el retorno a 10 días, expresado en puntos porcentuales.

```

4 (df["low"] <= df[["open", "close"]].min(axis=1))
5 & (df["high"] >= df[["open", "close"]].max(axis=1))
6 & (df > 0).all(axis=1)
7 )
8 return df[valid]
```

Código 3.1: Limpieza básica de una serie OHLC antes de construir las ventanas de entrenamiento.

Objetivo y ventanas temporales. El modelo no predice el precio, sino el rendimiento acumulado a diez días, esto es, la variación porcentual del cierre entre el día t y el día $t + 10$. Trabajar con un rendimiento, y no con un nivel de precio, tiene una ventaja: es una magnitud adimensional y comparable entre empresas, y ahorra deshacer cualquier escalado al final. El histórico se recorre con una ventana deslizante de paso uno [15]: cada muestra toma la ventana $[t - \text{lookback} + 1, t]$ como entrada y el rendimiento de t a $t + 10$ como objetivo. La longitud de la ventana (*lookback*) resultó ser el hiperparámetro más determinante de todo el estudio, y quedó fijada en 24 sesiones por ser el valor que mejor rendimiento ofreció.

```

1 def future_return(close, horizon):
2     # variacion porcentual entre t y t+horizon
3     return (close[horizon:] - close[:-horizon]) / close[:-horizon] * 100.0
```

```
4
5 for t in range(t_start, len(close) - horizon):
6     window = features[t - lookback + 1 : t + 1].astype(np.float64)
7     mu = window.mean(axis=0)
8     sd = window.std(axis=0)
9     sd[sd < 1e-8] = 1.0
10    xs.append((window - mu) / sd)    # tipificación con estadísticos de la propia
    ventana
```

Código 3.2: Construcción del objetivo a diez días y tipificación causal de cada ventana temporal.

Escalado. El escalado es el punto donde es más fácil introducir fuga de información, y por eso se trató con cuidado. En lugar de normalizar toda la serie de una vez con estadísticos globales (práctica habitual, pero que filtra información del futuro hacia el pasado), cada ventana se tipifica con su propia media y desviación típica, calculadas solo con los datos que contiene. Así, cada muestra depende únicamente de lo conocido hasta su día t , y es causal por construcción. Como el objetivo es un rendimiento, no hace falta ninguna transformación inversa para interpretar la predicción. Durante la investigación se probaron además dos conjuntos de variables de entrada: los niveles brutos de apertura, máximo y mínimo, frente a un conjunto de variables intradía derivadas y causales (rango, cuerpo y hueco de apertura), pensadas para aportar la información que el máximo, el mínimo y la apertura apenas añaden sobre el cierre, ya que los cuatro precios están correlacionados en torno al 99% de un día para otro. El *benchmark* concluyó que ese conjunto auxiliar era irrelevante: casi toda la señal la portaba la propia ventana del cierre.

Partición sin fuga. La partición es cronológica, 70/15/15, y no aleatoria: mezclar fechas en una serie temporal dejaría días futuros en el conjunto de entrenamiento. Además, como las muestras se solapan (paso uno) y el objetivo abarca diez días, las últimas muestras de cada bloque comparten su ventana de objetivo con las primeras del siguiente. Para cortar ese solape se descartan diez muestras (el tamaño del horizonte) en cada frontera.

```
1 def split_bounds(n, train_frac=0.70, val_frac=0.15, gap=10):
```

```
2     i1 = int(n * train_frac)
3     i2 = int(n * (train_frac + val_frac))
4     return slice(0, i1), slice(i1 + gap, i2), slice(i2 + gap, n)
```

Código 3.3: Partición cronológica con separación entre bloques para evitar solape del horizonte de predicción.

Decisiones que sostuvo este protocolo. Sobre esta base se desarrolló todo el estudio comparativo. Cada modelo se evaluó frente a un *naive* de rendimiento nulo (un paseo aleatorio, *random walk*), midiendo la calidad como el cociente entre su RMSE de validación y el del *naive*: por debajo de 1,0 se bate al paseo aleatorio. Con ese criterio se compararon, entre otros, LSTM, GRU y un *Transformer*, con y sin descomposición VMD, y después se afinó la red elegida con Optuna³ (muestreador TPE y poda Hyperband, dos semillas por configuración). La literatura atribuye grandes mejoras a las arquitecturas con VMD [31, 33, 58, 66] y al ajuste de hiperparámetros mediante algoritmos como el *Sparrow Search* [14, 32], frente a los modelos recurrentes clásicos para predicción bursátil y a la tendencia entendida como problema de clasificación [2, 57, 64, 65]. Sin embargo, la mayoría de esos trabajos descomponen la serie entera de una sola vez, con lo que filtran información futura hacia las muestras pasadas; al replicar el experimento con una descomposición causal, esa ventaja de la VMD desaparece. La conclusión, detallada en la sección dedicada a la arquitectura del módulo, fue emplear una GRU sencilla sin VMD. Conviene no exagerar el alcance: el rendimiento a diez días es solo marginalmente más predecible que un paseo aleatorio (la mejor puntuación de validación rondó 0,98), algo coherente con lo que la literatura advierte sobre la escasa señal de los retornos a corto plazo [27], y que justifica la banda neutral del clasificador de tendencia y el aviso legal del informe.

Información para inferencia

Las secciones anteriores describen los datos con los que se construye el sistema fuera de línea: el grafo de conocimiento del módulo de sentimiento y las series con las que se entrenan las redes GRU. En explotación, en cambio, los cuatro módulos no parten de esos conjuntos, sino que recuperan datos frescos en el momento de la consulta. Esta

³Web de Optuna: <https://optuna.org/>

sección recoge de dónde sale cada uno de esos datos y cómo se evita saturar las APIs externas.

El flujo lo inicia siempre el usuario desde la web. La interfaz no le pide pegar datos ni elegir un proveedor: solo introduce el *ticker* de la acción que quiere analizar en el buscador, y es el sistema el que fija una fuente concreta para cada módulo. Antes de lanzar ningún análisis, el *ticker* se valida en dos pasos. Primero, de forma sintáctica, comprobando que son entre dos y cinco caracteres alfanuméricos en mayúscula. Después, contra el catálogo de símbolos de Finnhub (`symbol_lookup`), exigiendo una coincidencia exacta, de modo que un símbolo inexistente se rechaza antes de buscar noticias. Esta segunda comprobación es *fail-open*: si la propia consulta de validación falla, el análisis continúa, porque la validación protege la calidad del resultado, no su disponibilidad. Superada la validación, los cuatro módulos se ejecutan de forma concurrente y cada uno descarga lo que necesita.

Noticias (módulo de sentimiento). Las noticias se obtienen de la API de Finnhub⁴, en concreto del endpoint `company_news`, que devuelve la actualidad de los últimos siete días para un *ticker* concreto, ordenada de más a menos reciente y sin retardo de publicación. Su límite es la cobertura: en el plan gratuito Finnhub solo cubre compañías norteamericanas. Por eso, cuando Finnhub no devuelve noticias para un valor, la petición cae por una cadena de respaldo hacia NewsAPI⁵, de cobertura universal aunque con un retardo de 24 horas en el plan gratuito. La variable de entorno `NEWS_PROVIDER` elige la cabeza de la cadena (Finnhub por defecto) y el otro proveedor actúa siempre como reserva; el detalle de esta decisión queda recogido en la ADR-0005. El cliente de Finnhub es síncrono, así que la llamada se delega a un hilo de trabajo para no bloquear el bucle asíncrono.

```
1 raw = await asyncio.to_thread(  
2     self._client.company_news,  
3     ticker,  
4     _from=(today - timedelta(days=7)).isoformat(),  
5     to=today.isoformat(),  
6 )
```

⁴Documentación de Finnhub: <https://finnhub.io/docs/api>

⁵Documentación de NewsAPI: <https://newsapi.org/docs>

Código 3.4: Consulta de noticias recientes a Finnhub.

Precios (deep learning, técnico y fundamental). Los tres módulos cuantitativos comparten una única fuente de precios: `yfinance`⁶, siempre con `auto_adjust=True`, es decir, con cotizaciones ya ajustadas por *splits* y dividendos. Para garantizar que todos descargan el dato de la misma manera, el acceso se centraliza en una sola función, `get_price_history`, que el resto de la aplicación reutiliza. A partir de ese histórico de cierre, el módulo de *deep learning* carga el modelo GRU entrenado para ese *ticker* (con carga perezosa y una caché LRU de hasta diez modelos) y le pasa la ventana más reciente para predecir el rendimiento a diez días; el módulo técnico calcula sus indicadores sobre los cierres recientes y se apoya en un universo de referencia (el S&P 500, cuya composición se obtiene de Wikipedia) para normalizar las señales; y el módulo fundamental usa los estados financieros que expone `yfinance` (`info`, `income_stmt`, `balance_sheet` y `cash_flow`) para las magnitudes *TTM* y el histórico anual. Que los tres beban de la misma fuente y con el mismo ajuste evita incoherencias entre lo que predice una red y lo que muestran los indicadores.

```
1 def get_price_history(ticker: str, period: str = "3y") -> pd.DataFrame:
2     data = yf.download(ticker, period=period, auto_adjust=True, progress=False)
3     if data.empty:
4         raise ValueError(f"No price data found for '{ticker}'.")
5     if isinstance(data.columns, pd.MultiIndex): # aplanar (campo, ticker)
6         data.columns = data.columns.droplevel(1)
7     return data.dropna(subset=["Close"])
```

Código 3.5: Fuente única de precios de cierre ajustados, compartida por los tres módulos cuantitativos.

Caché. Todas estas fuentes son externas y, en su mayoría, de plan gratuito con límites estrictos (Finnhub admite 60 peticiones por minuto, NewsAPI solo 100 al día). Además, el alojamiento gratuito de Render entra en reposo tras unos minutos de inactividad y arranca en frío. Por ambos motivos cada módulo cachea sus resultados con

⁶Biblioteca `yfinance`: <https://github.com/ranaroussi/yfinance>

un tiempo de vida (TTL) configurable: 30 minutos para el sentimiento, y un día para el fundamental y el técnico, ajustado a la frecuencia con la que cambia cada dato (las noticias se renuevan a menudo, los estados financieros no). La caché se implementa con `cachetools.TTLCache`, indexada por *ticker* (o por la pareja universo y *ticker* en el caso fundamental), de forma que una segunda consulta sobre el mismo valor responde al instante sin volver a llamar a la API. A esto se suman cachés en memoria para las operaciones más caras, como la descarga del universo completo de precios (de 30 a 60 segundos) o la lista del S&P 500, y la reutilización de clientes y servicios como *singletons* mediante `lru_cache`. El resultado es un sistema que se mantiene dentro de las cuotas gratuitas y responde con rapidez en las consultas repetidas.

```
1 self._cache: TTLCache[str, SentimentResult] = TTLCache(  
2     maxsize=256, ttl=cache_ttl # cache_ttl_sentiment = 1800 s (30 min)  
3 )  
4 # ...  
5 if not force_refresh and ticker in self._cache:  
6     logger.info("Sentiment cache hit for %s", ticker)  
7     return self._cache[ticker]
```

Código 3.6: Caché por *ticker* con TTL y atajo en caso de acierto del módulo de sentimiento.

En conjunto, la inferencia se nutre de datos bajo demanda y específicos de cada módulo: noticias de Finnhub (con NewsAPI de reserva) para el sentimiento, y precios y estados financieros de yfinance para los otros tres. Esos datos en vivo, validados y cacheados, complementan a los conjuntos fuera de línea descritos antes, que son los que dieron forma al grafo y a los modelos que ahora se consultan.

3.2.4. Grafo de conocimiento

Los dos conjuntos que nutren el grafo, FinEntity y FinMarBa, ya se describieron en la subsección dedicada a los datos, junto con el origen de su sentimiento y la razón de descartar Financial PhraseBank. Esta subsección da el paso siguiente: cómo esos datos se convierten en un grafo de conocimiento sobre Neo4j y cómo se consulta después en tiempo real. La técnica sobre la que se apoya es GraphRAG, que combina la recuperación semántica propia de RAG [30] con la expansión del vecindario que ofrece

un grafo [12]. El interés de trabajar sobre un grafo es que, al recuperar una noticia parecida a la que se analiza, no llega sola: arrastra el activo al que afecta, el sentimiento anotado sobre esa mención, el tema bajo el que cae y el evento que la origina. El contexto que recibe el modelo es relacional, no una lista de fragmentos sueltos.

Modelo del grafo: nodos y relaciones

El grafo se organiza en torno a cuatro tipos de nodo y cuatro tipos de relación, que la figura 3.2 resume. Los nodos son:

News una noticia o titular. Guarda el texto, el título (si lo hay), la fecha, la fuente (`finentity` o `finmarba`) y su *embedding* semántico.

Asset un activo financiero identificado por su `ticker`. Guarda su nombre y su tipo (`STOCK`, `ETF`, `INDEX`, `CURRENCY`, etc.).

Topic un tema, tomado de un vocabulario controlado de 15 valores (resultados, fusiones, regulación, dividendos, macroeconomía, etc.).

Event un evento financiero concreto extraído del texto, con un tipo, una descripción y una fecha cuando puede determinarse.

Y las relaciones que los conectan:

(**News**)-[:`MENTIONS`]->(Asset) la noticia menciona el activo. Es la relación que carga el sentimiento: lleva `sentiment_label`, `sentiment_score` y, en el caso de FinMarBa, también `pct_change`.

(**News**)-[:`TAGGED`]->(Topic) la noticia trata sobre ese tema.

(**News**)-[:`DESCRIBES`]->(Event) la noticia describe ese evento.

(**Event**)-[:`AFFECTS`]->(Asset) el evento afecta a ese activo. Es el salto que enlaza dos noticias distintas a través de un mismo evento o activo, y el que da valor a la expansión de vecindario.

El esquema se materializa en Neo4j con restricciones de unicidad sobre la clave de cada nodo y con un índice vectorial sobre el *embedding* de las noticias. El índice usa distancia coseno y 3.072 dimensiones, que es lo que produce el modelo de *embeddings*

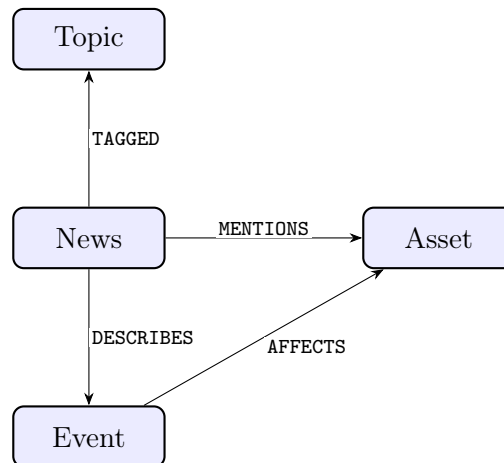


Figura 3.2: Esquema del grafo de conocimiento. El nodo **News** es el centro: referencia activos (con sentimiento), temas y eventos; los eventos, a su vez, vuelven a apuntar a los activos que impactan.

empleado (`text-embedding-3-large`), y es el que hace posible la búsqueda semántica en tiempo de consulta.

```

1  _EMBEDDING_DIMS = 3072  # text-embedding-3-large
2
3  _CONSTRAINTS = [
4      "CREATE CONSTRAINT asset_ticker IF NOT EXISTS FOR (a:Asset) REQUIRE a.ticker IS
5      UNIQUE",
6      "CREATE CONSTRAINT news_id      IF NOT EXISTS FOR (n:News)  REQUIRE n.id      IS
7      UNIQUE",
8      "CREATE CONSTRAINT topic_name   IF NOT EXISTS FOR (t:Topic) REQUIRE t.name   IS
9      UNIQUE",
10     "CREATE CONSTRAINT event_id      IF NOT EXISTS FOR (e:Event) REQUIRE e.id      IS
11     UNIQUE",
12 ]
13
14 _VECTOR_INDEX = (
15     "CREATE VECTOR INDEX news_embedding IF NOT EXISTS "
16     "FOR (n:News) ON n.embedding "
17     "OPTIONS {indexConfig: {"
18     "'vector.dimensions': 3072, "
19     "'vector.similarity_function': 'cosine'}}"
20 )

```

Código 3.7: Restricciones e índice vectorial (`schema.py`).

Construcción del grafo

El grafo se puebla con un proceso batch que se ejecuta una sola vez, fuera de línea, y que no forma parte del camino de una petición del usuario. Por cada registro de los datasets el pipeline hace cinco cosas: lo carga y normaliza, genera su *embedding*, extrae eventos y temas con el modelo de lenguaje, resuelve las entidades a tickers reales, y escribe el nodo de noticia junto con sus relaciones. Todo es idempotente, de modo que reejecutar el proceso no duplica nada.

Carga y normalización. Cada conjunto entra por su propio cargador. El de FinEntity recorre el JSON y extrae, por registro, el texto y la lista de anotaciones de entidad con su etiqueta; el de FinMarBa reconstruye por fila el titular, la fecha, los tickers y los diccionarios de sentimiento y cambio porcentual por ticker. El detalle a resaltar en FinMarBa es que la etiqueta y el cambio porcentual salen de datos estructurados, nunca del LLM.

```
1 _SENTIMENT_MAP = {1: "Positive", 0: "Neutral", -1: "Negative"}
2
3 # El sentimiento y el cambio porcentual proceden del dataset, no del modelo.
4 records.append(FinMarBaRecord(
5     title=str(row["Title"]),
6     date=str(row["Date"]),
7     tickers=tickers,
8     sentiment={t: _SENTIMENT_MAP.get(v, "Neutral") for t, v in sent_raw.items()},
9     sentiment_score={t: int(v) for t, v in sent_raw.items()},
10    pct_change={t: float(v) for t, v in pct_raw.items()},
11 ))
```

Código 3.8: Normalización de FinMarBa: sentimiento derivado del mercado y cambio porcentual por ticker (`loaders/finmarba.py`).

Identidad determinista de las noticias. Cada noticia recibe un identificador derivado del propio texto mediante un hash SHA-256 normalizado. Así, dos textos idénticos, aunque vengan de datasets distintos, producen el mismo nodo y se deduplican solos.

```
1 def news_id(text: str) -> str:
2     normalised = text.lower().strip()
3     return hashlib.sha256(normalised.encode()).hexdigest()[:32]
```

Código 3.9: Identificador determinista de una noticia (`neo4j_client.py`).

Embeddings semánticos. Los textos se transforman en vectores con `text-embedding-3-large` de OpenAI⁷. La generación se hace por lotes de 512 textos, con reintentos y espera exponencial para absorber errores transitorios de la API. El mismo módulo de *embeddings* se comparte entre la construcción offline y la recuperación en tiempo real, de modo que el vector de una consulta vive en el mismo espacio que los del grafo.

```
1 async def embed_texts(texts: list[str]) -> list[list[float]]:
2     settings = get_settings()
3     client = AsyncOpenAI(api_key=settings.openai_api_key)
4     model = settings.openai_embedding_model      # text-embedding-3-large
5     results: list[list[float]] = []
6     for i in range(0, len(texts), _BATCH_SIZE): # _BATCH_SIZE = 512
7         batch = texts[i : i + _BATCH_SIZE]
8         results.extend(await _embed_batch(client, batch, model))
9     return results
```

Código 3.10: Generación de embeddings por lotes (`embeddings.py`).

Extracción de eventos y temas con el LLM. Aquí interviene el modelo de lenguaje, y solo aquí. Se usa `gpt-5.4-mini` para leer cada texto y devolver, en JSON estricto, los eventos que describe y los temas que le corresponden. El modelo no realiza clasificación de sentimiento ni genera etiquetas arbitrarias. La salida se restringe a un vocabulario controlado formado por 15 temas y 10 tipos de eventos. Durante el proceso de validación y parsing de la respuesta, cualquier categoría que no pertenezca a dicho

⁷Documentación de `text-embedding-3-large`: <https://developers.openai.com/api/docs/models/text-embedding-3-large>

vocabulario es descartada. Esa restricción explica que el grafo tenga exactamente 15 nodos `Topic`, uno por valor permitido. La extracción se hace en lotes con concurrencia acotada (un semáforo limita a tres las llamadas simultáneas) y, si una respuesta no es JSON válido, ese registro se salta sin finalizar de forma temprana el proceso. Delegar en un LLM la extracción de estructura y el mapeo de entidades a tickers, con validación posterior, sigue la línea de trabajos recientes sobre procesamiento de noticias financieras [11].

```

1 _SYSTEM_PROMPT = f"""You are a financial news analyser. Given a news text, extract:
2 1. Financial events described (if any). Each event must have:
3     - type: one of {EVENT_TYPES}
4     - description: one concise sentence
5     - date: ISO date string if determinable, otherwise null
6 2. Relevant topics from this controlled list: {TOPICS}
7 Respond ONLY with valid JSON ..."""
8
9 async def extract(text: str, llm_service) -> ExtractionResult:
10     raw = await llm_service.complete(system_prompt=_SYSTEM_PROMPT, user_prompt=text)
11     data = json.loads(raw)
12     events = [ExtractedEvent(e.get("type"), e.get("description", ""), e.get("date"))
13               for e in data.get("events", []) if e.get("type") in EVENT_TYPES]
14     topics = [t for t in data.get("topics", []) if t in TOPICS]
15     return ExtractionResult(events=events, topics=topics)

```

Código 3.11: Prompt de sistema y parseo defensivo de la extracción (`llm_extractor.py`).

Resolución de entidades a tickers. Las anotaciones de `FinEntity` vienen como nombres (“Johnson & Johnson”), no como tickers, y se necesita un ticker canónico para unificar el activo en el grafo. La resolución intenta primero un patrón de ticker embebido al estilo Reuters dentro del propio texto (por ejemplo `<JNJ.N>`) y, si no lo encuentra, recurre a la búsqueda de `yfinance`, que además aporta el nombre largo y el tipo de activo. Todo se cachea en memoria para no repetir llamadas. Si un nombre no se puede resolver no se descarta: el activo se guarda con su nombre como clave de respaldo, de modo que el registro sigue entrando en el grafo aunque ese activo no se pueda deduplicar entre datasets.

```

1 _REUTERS_RE = re.compile(r"<([A-Z0-9~=]+)\.([A-Z]+>")

```

```

2
3 def resolve_from_name(entity_name: str, text: str) -> tuple[str | None, str, str]:
4     if entity_name in _cache:
5         return _cache[entity_name]
6     match = _REUTERS_RE.search(text)           # 1. patron Reuters en el texto
7     if match:
8         ticker = match.group(1)
9         name, asset_type = _yf_info(ticker)
10        return _cache.setdefault(entity_name, (ticker, name, asset_type))
11    hits = yf.Search(entity_name, max_results=1).quotes # 2. busqueda yfinance
12    if hits:
13        ticker = hits[0].get("symbol") or None
14        name = hits[0].get("longname") or hits[0].get("shortname") or entity_name
15        asset_type = _QUOTE_TYPE_MAP.get(hits[0].get("quoteType", "EQUITY"),
16        "STOCK")
17        return _cache.setdefault(entity_name, (ticker, name, asset_type))
18    return _cache.setdefault(entity_name, (None, entity_name, "STOCK"))

```

Código 3.12: Resolución de nombre a ticker con patrón Reuters y yfinance (entity_resolver.py).

Escritura idempotente en Neo4j. La persistencia se apoya por completo en la cláusula MERGE de Cypher, que crea el nodo o la relación solo si no existía. Combinada con las restricciones de unicidad del esquema, garantiza que reejecutar el pipeline no genere duplicados. El sentimiento se escribe sobre la relación MENTIONS, y el cambio porcentual solo se añade cuando existe, es decir, para los registros de FinMarBa.

```

1 async def merge_news(session, node_id, text, title, embedding, published_at,
2     source):
3     await session.run("""
4         MERGE (n:News {id: $id})
5         SET n.text = $text, n.title = $title, n.embedding = $embedding,
6             n.published_at = $published_at, n.source = $source
7     """, id=node_id, text=text, title=title, embedding=embedding,
8         published_at=published_at, source=source)
9
10 async def merge_mentions(session, news_id, ticker, entity_name,
11     sentiment_label, sentiment_score, pct_change=None):

```

```
11     asset_key = ticker if ticker else entity_name
12     query = """
13         MATCH (n:News {id: $news_id})
14         MATCH (a:Asset {ticker: $asset_key})
15         MERGE (n)-[r:MENTIONS]->(a)
16         SET r.sentiment_label = $sentiment_label, r.sentiment_score =
17         $sentiment_score
18     """
19     if pct_change is not None:                # solo FinMarBa
20         query += ", r.pct_change = $pct_change"
21     await session.run(query, ...)
```

Código 3.13: MERGE de la noticia y de la relación MENTIONS con su sentimiento (neo4j_client.py).

El orquestador recorre cada registro, resuelve sus activos, escribe la noticia y teje las relaciones. Conviene fijarse en que el sentimiento de MENTIONS viene siempre de la fuente (anotación humana en FinEntity, reacción del mercado en FinMarBa), mientras que los nodos Topic y Event son los únicos que produce el LLM. Cada evento se crea con un identificador propio y se enlaza con la noticia que lo describe y con los activos que menciona esa noticia.

```
1  # Temas
2  for topic in extraction.topics:
3      await neo.merge_topic(session, topic)
4      await neo.merge_tagged(session, node_id, topic)
5
6  # Eventos: 1 evento -> 1 noticia que lo describe -> N activos afectados
7  for event in extraction.events:
8      event_id = neo.new_event_id()
9      await neo.merge_event(session, event_id, event.type, event.description,
10                             event.date)
11      await neo.merge_describes(session, node_id, event_id)
12      for ticker, name in asset_tuples:
13          await neo.merge_affects(session, event_id, ticker, name)
```

Código 3.14: Tejido de temas y eventos por registro (extracto del orquestador).

El grafo resultante

Tras procesar ambos conjuntos, el grafo final contiene 14.932 nodos y 36.889 relaciones. El cuadro 3.6 recoge el desglose por tipo.

Categoría	Tipo	Recuento
Nodos	News	9.066
	Event	4.479
	Asset	1.372
	Topic	15
	Total	14.932
Relaciones	TAGGED	14.491
	MENTIONS	12.091
	AFFECTS	5.828
	DESCRIBES	4.479
	Total	36.889

Tabla 3.6: Composición del grafo de conocimiento por tipo de nodo y de relación.

Los números dejan ver algunas propiedades del diseño. Los 15 nodos **Topic** coinciden exactamente con el tamaño del vocabulario controlado, prueba de que la restricción del *prompt* funciona y no introduce temas espurios. El recuento de **DESCRIBES** (4.479) es idéntico al de nodos **Event**, lo que confirma que cada evento se crea ligado a una única noticia. Y el número de nodos **News** (9.066) encaja con la suma de ambos conjuntos una vez descontados los textos idénticos: los 979 párrafos de FinEntity y los 8.142 titulares del subconjunto de FinMarBa empleado. El tamaño total, por lo demás, cabe con holgura en el nivel gratuito de Neo4j AuraDB (512 MB), una de las restricciones de infraestructura del proyecto.

Como muestra de la cobertura del grafo, la Tabla 3.7 lista los diez activos más mencionados. El perfil es coherente con el origen macro de FinMarBa, construido sobre resúmenes globales de mercado: dominan índices y ETFs de gran capitalización (el S&P 500 en sus formas **SPY** y **^GSPC**, **ACWI** para renta variable global), junto con referencias

de materias primas, tipos y divisas (petróleo CL=F, bono a 10 años ^TNX, dólar UUP, euro FXE, oro GLD). Es el universo que cabe esperar de una fuente centrada en el pulso general del mercado más que en compañías individuales.

Ticker	Nombre	Menciones
SPY	State Street SPDR S&P 500 ETF Trust	1.219
^GSPC	S&P 500	1.131
ACWI	iShares MSCI ACWI ETF	542
CL=F	Crude Oil (futuro)	476
^TNX	CBOE Interest Rate 10 Year T Note	347
EEM	iShares MSCI Emerging Markets ETF	318
UUP	Invesco DB US Dollar Index Bullish Fund	293
FXE	Invesco CurrencyShares Euro Currency Trust	243
EWG	iShares MSCI Germany ETF	222
GLD	SPDR Gold Shares	219

Tabla 3.7: Diez activos con más menciones en el grafo.

Para ver cómo encajan estas piezas en un caso concreto, la Figura 3.3 muestra un subgrafo de ejemplo alrededor de una noticia: esta menciona varios activos, se etiqueta con un tema y describe un evento que, a su vez, afecta a esos mismos activos. Esa conexión cruzada entre la noticia, el evento y los activos que ambos tocan es la que aprovecha después la recuperación GraphRAG.

Recuperación GraphRAG en tiempo de consulta

Una vez construido, el grafo se consulta cada vez que llega una petición de análisis. El texto de cada noticia reciente sobre el ticker pedido se convierte en un *embedding* y se buscan en el índice vectorial `news_embedding` las noticias históricas más parecidas. Hasta aquí es RAG clásico. La diferencia está en lo que ocurre después: en lugar de devolver solo el texto de esas noticias, se expande su vecindario en el grafo para arrastrar el contexto relacional. Esto se implementa con `VectorCypherRetriever` de la librería

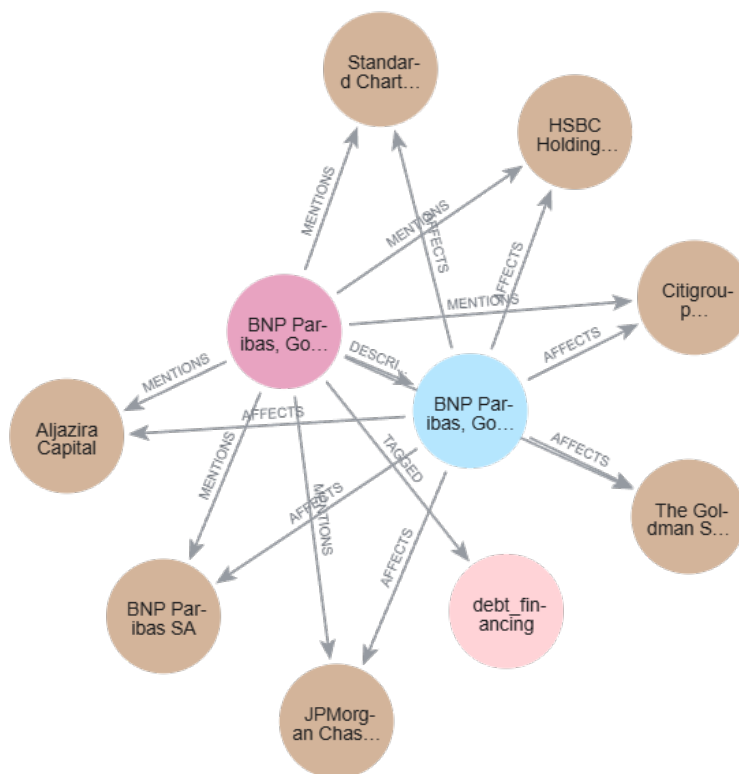


Figura 3.3: Subgrafo de ejemplo visualizado en Neo4j Browser: una noticia (News) menciona (MENTIONS) varios activos, se etiqueta (TAGGED) con temas y describe (DESCRIBES) un evento que, a su vez, afecta (AFFECTS) a esos mismos activos.

neo4j-graphrag⁸, que ejecuta una consulta Cypher a continuación de la búsqueda vectorial.

La profundidad de esa expansión es configurable con la variable `GRAPH_HOP_DEPTH`, fijada a 2. A profundidad 1 se recoge el vecindario directo de cada noticia (los activos que menciona con su sentimiento, los temas y los eventos). A profundidad 2 se añade el salto `Event -> AFFECTS -> Asset`, que trae también los activos afectados por esos eventos aunque la noticia no los mencionara de forma directa. Ese segundo salto es justamente lo que aporta el grafo frente a un almacén vectorial plano.

```
1 def _build_retrieval_query(hop_depth: int) -> str:
2     legs = [
3         "OPTIONAL MATCH (node)-[m:MENTIONS]->(a:Asset)",
4         "OPTIONAL MATCH (node)-[:TAGGED]->(t:Topic)",
5         "OPTIONAL MATCH (node)-[:DESCRIBES]->(e:Event)",
6     ]
7     affected_column = "[] AS affected_assets"
8     if hop_depth >= 2:                                     # segundo salto
9         legs.append("OPTIONAL MATCH (e)-[:AFFECTS]->(ea:Asset)")
10        affected_column = "collect(DISTINCT ea.name) AS affected_assets"
11    return f"""
12        {chr(10).join(legs)}
13        RETURN node.text AS text, node.title AS title, score,
14            collect(DISTINCT {{ ticker: a.ticker, name: a.name,
15                sentiment_label: m.sentiment_label,
16                sentiment_score: m.sentiment_score }}) AS mentions,
17            collect(DISTINCT t.name) AS topics,
18            collect(DISTINCT {{ type: e.type, description: e.description,
19                date: e.date }}) AS events,
20            {affected_column}
21    """
```

Código 3.15: Consulta de recuperación tras la búsqueda vectorial, parametrizada por la profundidad de salto (`graph_retriever.py`).

⁸Documentación Neo4j GraphRAG for Python: <https://neo4j.com/docs/neo4j-graphrag-python/>

En la práctica, el servicio de sentimiento lanza una recuperación por cada noticia reciente, recupera las tres más similares de cada una, deduplica los resultados por el texto de la noticia (quedándose con la mayor similitud cuando una misma noticia aparece por varias vías), los ordena y se queda como mucho con las diez de mayor similitud. Como la librería `neo4j-graphrag` es síncrona, esa llamada se ejecuta en un hilo aparte para no bloquear el bucle de eventos de FastAPI. El resultado, ya con activos, sentimientos, temas y eventos, se serializa como contexto histórico y se entrega al LLM junto con las noticias frescas, con la instrucción explícita de tratar lo reciente como señal principal y usar lo histórico solo para calibrar. La salida final del módulo, en español y con su explicación, se apoya así en un contexto que no es una lista de textos sueltos sino una porción del propio grafo. La Figura 3.4 resume este flujo completo, desde las noticias recientes hasta la respuesta en español.

3.2.5. Servicio LLM centralizado

Los modelos de lenguaje aparecen en dos momentos distintos de FinSight: en la construcción del grafo de conocimiento, para extraer temas y eventos, y en la aplicación en marcha, donde clasifican el sentimiento, traducen las métricas fundamentales a lenguaje natural, comentan la señal técnica y redactan la conclusión final del informe. Si cada uno de esos usos hablara directamente con la API de un proveedor concreto, cambiar de proveedor obligaría a modificar el código de todos los módulos. Para evitarlo, todo el acceso al LLM pasa por una única abstracción, siguiendo el patrón Adaptador [16]: los módulos dependen de una interfaz común y no del SDK concreto que hay debajo, de modo que sustituir un proveedor por otro no cambia ni una línea de la lógica de negocio.

Sobre esa interfaz se ofrecen dos proveedores. Ollama⁹ ejecuta modelos abiertos en local (por defecto Llama 3.1 8B) sin coste y sin salir de la máquina, lo que es útil para desarrollar offline. OpenAI es el proveedor externo de pago (`gpt-5.4-mini`), de calidad notablemente superior. Ambos son intercambiables mediante una variable de entorno, pero conviene ser honesto sobre la diferencia: la calidad del análisis depende de forma directa de la capacidad del modelo empleado, y los modelos gratuitos que

⁹Ollama: <https://ollama.com>

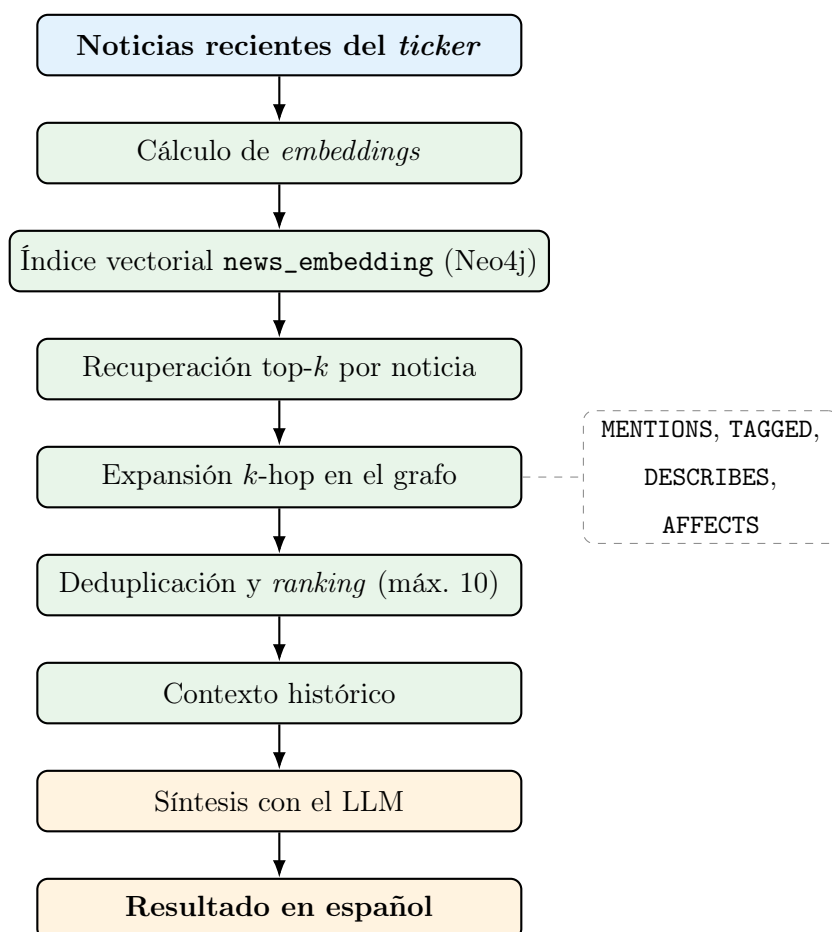


Figura 3.4: Flujo de la recuperación GraphRAG en tiempo de consulta: las noticias recientes del valor se convierten en *embeddings* y se buscan en el índice vectorial *news_embedding*; sobre las más similares se expande su vecindario en el grafo (MENTIONS, TAGGED, DESCRIBES y AFFECTS), se deduplican y ordenan los resultados, y el contexto histórico resultante se entrega al LLM, que produce la respuesta en español.

corren cómodamente en local rinden bastante peor en extracción estructurada y en redacción en español que `gpt-5.4-mini`, por lo que se recomienda OpenAI para obtener resultados fiables. En una fase inicial se valoró añadir un tercer proveedor alojado y gratuito (Groq), pero se descartó porque añadía complejidad al sistema sin aportar ninguna ventaja frente a lo que ya cubren estos dos: el desarrollo local con Ollama y la calidad de OpenAI.

La interfaz común

La interfaz es deliberadamente mínima. Una clase abstracta, `LLMService`, declara un único método asíncrono, `complete`, que recibe un *system prompt* y un *user prompt* y devuelve el texto de la respuesta. Todo lo demás (autenticación, reintentos, forma del cuerpo de la petición) queda encapsulado en cada adaptador concreto. La figura 3.5 resume la jerarquía.

```
1 class LLMService(ABC):
2     """Abstract interface for all LLM providers.
3
4     All modules must call the LLM exclusively through this interface so that
5     swapping providers requires no changes in business logic.
6     """
7
8     @abstractmethod
9     async def complete(self, system_prompt: str, user_prompt: str) -> str:
10         """Send a chat completion request and return the assistant's response."""
```

Código 3.16: Interfaz abstracta del servicio LLM (`base.py`).

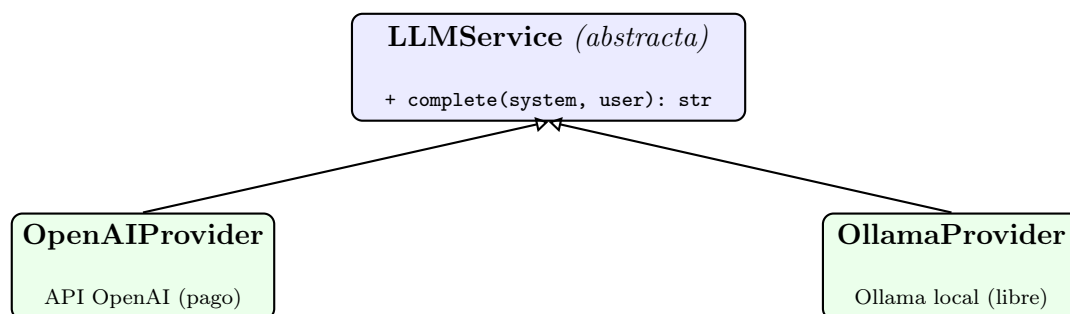


Figura 3.5: Patrón Adaptador aplicado al servicio LLM. Los módulos dependen de la interfaz `LLMService`; cada proveedor la implementa por su cuenta.

Los proveedores

El adaptador de OpenAI envuelve el cliente asíncrono oficial y delega en el endpoint de *chat completions*. Se configura con seis reintentos automáticos para absorber errores transitorios de red o límites de tasa, y devuelve directamente el contenido del primer mensaje de la respuesta.

```
1 class OpenAIProvider(LLMService):
2     """LLM provider backed by the OpenAI API."""
3
4     def __init__(self, api_key: str, model: str) -> None:
5         self._client = AsyncOpenAI(api_key=api_key, max_retries=6)
6         self._model = model
7
8     async def complete(self, system_prompt: str, user_prompt: str) -> str:
9         response = await self._client.chat.completions.create(
10             model=self._model,
11             messages=[
12                 {"role": "system", "content": system_prompt},
13                 {"role": "user", "content": user_prompt},
14             ],
15         )
16         return response.choices[0].message.content or ""
```

Código 3.17: Adaptador de OpenAI (`openai_provider.py`).

El adaptador de Ollama habla con la instancia local por HTTP contra su endpoint `/api/chat`, con *streaming* desactivado para recibir la respuesta completa de una vez y un tiempo de espera holgado, porque la inferencia en CPU es lenta. La forma de la petición y de la respuesta cambia respecto a OpenAI, pero eso queda oculto tras la misma firma de `complete`, que es justo lo que persigue el patrón.

```
1 class OllamaProvider(LLMService):
2     """LLM provider backed by a local Ollama instance."""
3
4     def __init__(self, base_url: str, model: str) -> None:
5         self._base_url = base_url.rstrip("/")
6         self._model = model
7
```

```
8     async def complete(self, system_prompt: str, user_prompt: str) -> str:
9         payload = {
10             "model": self._model, "stream": False,
11             "messages": [
12                 {"role": "system", "content": system_prompt},
13                 {"role": "user", "content": user_prompt},
14             ],
15         }
16         async with httpx.AsyncClient(timeout=120.0) as client:
17             response = await client.post(f"{self._base_url}/api/chat", json=payload)
18             response.raise_for_status()
19             data = response.json()
20             return data["message"]["content"]
```

Código 3.18: Adaptador de Ollama (ollama_provider.py).

Selección del proveedor

Qué adaptador se instancia lo decide una factoría a partir de la variable de entorno `LLM_PROVIDER`, sin que ningún módulo tenga que saberlo. La función está decorada con `lru_cache`, de modo que el proveedor se crea una sola vez y se reutiliza como *singleton* durante toda la vida del proceso, evitando reabrir clientes en cada petición.

```
1 @lru_cache
2 def get_llm_service() -> LLMService:
3     """Return the singleton LLM provider configured via LLM_PROVIDER env var."""
4     settings = get_settings()
5     provider = settings.llm_provider
6
7     if provider == "ollama":
8         return OllamaProvider(base_url=settings.ollama_base_url,
9                                model=settings.ollama_model)
10    if provider == "openai":
11        return OpenAIProvider(api_key=settings.openai_api_key,
12                               model=settings.openai_model)
13    raise ValueError(f"Unknown LLM_PROVIDER: {provider!r}. Must be 'ollama' or 'openai'.")
```

Código 3.19: Factoría con selección por configuración y singleton (factory.py).

Consumo desde los módulos

Los cuatro módulos obtienen el servicio con `get_llm_service()` y lo invocan siempre igual, con `await self._llm.complete(system_prompt, user_prompt)`. La especialización de cada tarea vive por completo en los *prompts*, no en el código de acceso. El procesamiento interno se hace en inglés (noticias, métricas y prompts), pero a los módulos que generan texto para el usuario se les exige de forma explícita que respondan en español, y a los que necesitan una salida estructurada se les pide un JSON con un esquema fijo. Como un modelo puede devolver JSON mal formado, la respuesta se parsea de forma defensiva y se reintenta una vez antes de dar el resultado por inválido, tal y como se ve en el módulo de sentimiento.

```
1 for attempt in range(1, _MAX_LLM_ATTEMPTS + 1):           # _MAX_LLM_ATTEMPTS = 2
2     raw = await self._llm.complete(system_prompt=_SYSTEM_PROMPT,
3     user_prompt=user_prompt)
4     try:
5         return self._parse_response(raw, articles)         # valida y normaliza el JSON
6     except (json.JSONDecodeError, KeyError, TypeError, ValidationError) as exc:
7         last_error = exc                                   # reintenta una vez mas
8 raise SentimentAnalysisError("LLM did not return a valid sentiment result") from
9 last_error
```

Código 3.20: Llamada al LLM con parseo defensivo y reintento (extracto de `sentiment/service.py`).

Una dependencia que el patrón no cubre: los embeddings

El patrón Adaptador desacopla la parte generativa, pero hay una pieza que no se puede intercambiar con la misma libertad: los *embeddings*. Los vectores de las noticias del grafo de conocimiento se calcularon con el modelo `text-embedding-3-large` de OpenAI, y la búsqueda por similitud solo tiene sentido si la consulta se proyecta en ese mismo espacio vectorial, es decir, con el mismo modelo. Un vector generado por otro modelo no es comparable con los que ya están indexados. La consecuencia es que, aunque se elija Ollama como proveedor de generación para no pagar por el LLM, el módulo de sentimiento sigue necesitando la API de OpenAI para embeber la consulta. Aun cuando el coste de los embeddings es muy bajo, esto obliga a disponer de una

cuenta de OpenAI con crédito, que pasa a ser una dependencia crítica del sistema y una restricción de operación más.

A esta restricción se suma otra de despliegue. Ollama necesita cargar el modelo en memoria y, para un modelo de 8.000 millones de parámetros, eso supone varios gigabytes de RAM y, en la práctica, una GPU o una CPU potente para responder en un tiempo razonable. El alojamiento gratuito de Render ofrece 512 MB de RAM, 0,1 de CPU y ningún acelerador, así que ejecutar Ollama sobre esa infraestructura no es viable. En la versión desplegada, por tanto, el proveedor operativo es OpenAI; Ollama queda como opción para el desarrollo local o para un autoalojamiento en una máquina con hardware suficiente. Los dos proveedores son intercambiables por diseño, pero no por ello igual de desplegables en cualquier entorno.

Mitigación propuesta

La dependencia de OpenAI para los embeddings se puede eliminar sin renunciar a la búsqueda semántica, a costa de algo de trabajo y de calidad. La idea es construir un segundo conjunto de embeddings del grafo con un modelo abierto y gratuito, por ejemplo uno servido por el propio Ollama, y almacenarlo en un índice vectorial paralelo en Neo4j junto al actual. En tiempo de consulta, el sistema elegiría un índice u otro en función del proveedor activo: con OpenAI seguiría usando el índice de `text-embedding-3-large`, y con el proveedor libre usaría el índice generado con el modelo gratuito, de modo que consulta y grafo compartirían siempre el mismo espacio vectorial. Así, un despliegue completamente gratuito o autoalojado sería posible sin ninguna cuenta de pago. El coste es doble: hay que recalcular y almacenar los embeddings de todo el grafo con el modelo libre, y hay que asumir que su menor calidad puede degradar la relevancia de las noticias recuperadas. Esta línea no está implementada en la versión actual y queda planteada como trabajo futuro.

3.2.6. Modelo de procesamiento de series temporales

El módulo de *deep learning* debe responder a una pregunta muy concreta: dada la serie de precios de un valor del Nasdaq, ¿en qué dirección se moverá su cotización dentro de diez sesiones? Antes de fijar una arquitectura se revisaron dos familias de

métodos que la literatura aplica a este problema: los modelos econométricos clásicos de series temporales y las redes neuronales profundas. La primera aporta un marco estadístico interpretable y sirve de referencia mínima; la segunda promete capturar las relaciones no lineales del mercado, y es la que dominaba los trabajos recientes que se tomaron como punto de partida. Esta sección recorre ese proceso de investigación, explica por qué se descartaron unas opciones y se retuvieron otras, y describe cómo la red finalmente elegida se integró en el sistema.

Modelos ARIMA para el análisis y predicción de series financieras

El análisis de series temporales es una de las herramientas básicas de la econometría financiera para estudiar la evolución de precios, rendimientos, tipos de interés o tipos de cambio. Su fundamento son los procesos estocásticos, que describen fenómenos cuya evolución depende tanto de factores observables como de componentes aleatorios, y que permiten estimar el comportamiento futuro de una variable en un entorno de incertidumbre. Para examinar la estructura temporal de una serie se recurre a la función de autocorrelación (FAC) y a la función de autocorrelación parcial (FACP), que miden la relación entre el valor actual y sus observaciones pasadas y orientan la elección del modelo.

Entre los modelos clásicos destacan los autorregresivos de medias móviles (ARMA), que explican una variable a partir de sus propios valores históricos y de perturbaciones pasadas. Como muchas series financieras presentan tendencias o cambios estructurales y no son estacionarias, se emplea su extensión ARIMA (*Autoregressive Integrated Moving Average*), formalizada por Box y Jenkins [7], que aplica una diferenciación previa para estacionarizar la serie. Cuando aparecen patrones estacionales se usan variantes como SARIMA, y la configuración concreta se decide analizando la FAC y la FACP junto con criterios de información como AIC o BIC, que equilibran capacidad explicativa y parsimonia.

En este proyecto ARIMA no se descartó por principio, sino que se incorporó como una de las líneas base del estudio comparativo. En concreto se evaluó un ARIMA(1,1,1) con deriva, ajustado de forma *walk-forward*: se estima una vez sobre los datos previos al primer día de predicción y se extiende observación a observación sin reajustar, de

modo que cada pronóstico usa solo información disponible hasta ese día. Sirve como vara de medir lineal frente a las redes neuronales, pero no como modelo de producción: es un método univariante y lineal que difícilmente aprovecha la señal, ya de por sí escasa, de los retornos a diez días. Los resultados de esa comparación se detallan más adelante.

Modelos GARCH para la modelización de la volatilidad financiera

Además de predecir la evolución esperada de una variable, en finanzas importa la incertidumbre asociada a esa evolución, que se refleja en la volatilidad. En los mercados la volatilidad no es constante: alterna periodos de calma con episodios de fuerte incertidumbre, un fenómeno conocido como agrupamiento de volatilidad. Las series financieras suelen estudiarse a través de los rendimientos y no de los precios, porque sus propiedades estadísticas se ajustan mejor a la modelización; aunque su media es relativamente estable, la varianza cambia con el tiempo y genera heterocedasticidad condicional. Para capturarla, Engle propuso los modelos ARCH [13], que describen la varianza condicional como función de los errores pasados, y Bollerslev los generalizó en los modelos GARCH [6], que añaden la influencia de las volatilidades anteriores y ofrecen estimaciones del riesgo que evolucionan con la información disponible.

Los modelos GARCH son habituales en la previsión de volatilidad, la gestión de riesgos, la valoración de derivados y el cálculo de medidas como el Valor en Riesgo (VaR), y complementan de forma natural a ARIMA: uno modela la media de la serie y el otro su varianza. Se estudiaron por completitud, pero quedan fuera del alcance de FinSight. El producto responde por la tendencia del precio a un horizonte fijo, no por su volatilidad, de modo que una estimación de la varianza condicional no alimenta ninguna de las salidas del sistema. Modelar el riesgo con GARCH y trasladarlo al informe se recoge como una posible línea de trabajo futuro.

Redes neuronales recurrentes: modelos LSTM y GRU

El aprendizaje profundo permite modelar relaciones temporales complejas y no lineales que los modelos lineales no capturan. Las redes neuronales recurrentes (RNN) son la base de estos enfoques, ya que incorporan un estado interno que se propaga a

lo largo de la secuencia. Su versión básica, sin embargo, tiene dificultades para aprender dependencias largas por el problema de desvanecimiento del gradiente. Las redes *Long Short-Term Memory* (LSTM), propuestas por Hochreiter y Schmidhuber [19], resuelven esa limitación con una celda de memoria y tres puertas (entrada, olvido y salida) que regulan qué información se conserva y cuál se descarta. Las *Gated Recurrent Unit* (GRU), introducidas por Cho *et al.* [8], simplifican esa estructura a dos puertas (actualización y reinicio), con lo que reducen el número de parámetros y el coste de entrenamiento manteniendo una capacidad predictiva equiparable.

En ambos casos la arquitectura empleada sigue el mismo esquema: la red recorre la ventana de entrada, se toma el estado oculto del último paso temporal como resumen de toda la secuencia y una pequeña cabeza densa lo proyecta a la magnitud objetivo. El problema se plantea como una regresión sobre el retorno porcentual a diez días, no como una clasificación directa de la tendencia, y se entrena minimizando el error cuadrático medio (MSE) con el optimizador Adam y parada temprana sobre la pérdida de validación. La conversión a una etiqueta de tendencia (alcista, bajista o neutral) se hace después, aplicando una banda neutral sobre el retorno predicho. Esta decisión de tratar la tendencia como el signo de una regresión, y no como una clase aprendida directamente, encaja con lo que la literatura observa sobre la escasa separabilidad de las clases a corto plazo [2, 64].

Exploración de arquitecturas híbridas y de atención

La fase inicial de investigación no partió de cero, sino de la réplica de un repositorio público que compara varias arquitecturas sobre precios de diez compañías¹⁰. A partir de él se reprodujeron y midieron modelos progresivamente más sofisticados: LSTM y BiLSTM, combinaciones con capas convolucionales (BiLSTM+CNN, LSTM+CNN), variantes con mecanismos de atención (BiLSTM+Attention) y arquitecturas de tipo *Transformer* basadas en *multi-head attention* [61], además de un conjunto que fusiona varias de ellas (BiLSTM+Attention+GRU+CNN). El interés de las redes BiLSTM está en que procesan la secuencia en los dos sentidos temporales, y el de la atención en

¹⁰Repositorio de referencia: https://github.com/lamhungphu/BLSTM-Attention-GRU-CNN_StockPred

que pondera dinámicamente qué observaciones pesan más en la predicción en lugar de tratar toda la ventana por igual. La misma línea de trabajos combina estas redes con técnicas econométricas como ARIMA y GARCH en modelos híbridos, buscando unir la interpretabilidad estadística con la capacidad de las redes profundas.

Sobre el papel, estas arquitecturas ofrecían las mejores métricas, y otro bloque de la literatura atribuye mejoras notables al preprocesamiento por descomposición variacional (VMD) [31, 33, 58, 66] y al ajuste de hiperparámetros con metaheurísticas como el *Sparrow Search* [14, 32]. Al reproducir esos montajes surgió el problema que reorientó todo el módulo: casi todos predecían el precio del día siguiente sobre una serie escalada y descompuesta de una sola vez, con lo que información del futuro se filtraba hacia las muestras pasadas. Una parte de la ventaja que mostraban esos modelos no era capacidad predictiva, sino fuga de datos. Esto obligó a dejar de perseguir la arquitectura más compleja y a construir un banco de pruebas propio en el que todos los modelos se midieran en condiciones honestas, replicables y equivalentes a las de la inferencia real.

El estudio comparativo causal

El punto de partida de FinSight fue un *notebook* de referencia con la cadena VMD+SSA+LSTM habitual en la literatura; sobre él se rehízo el experimento eliminando toda fuga de información. Los datos y el *pipeline* sin fugas (limpieza de la serie OHLC, objetivo directo a diez días, tipificación causal por ventana y partición cronológica con separación entre bloques) se describen en la sección sobre las series de precios de entrenamiento, de modo que aquí el foco es la comparación entre arquitecturas y sus resultados. Todos los modelos se evaluaron sobre exactamente las mismas muestras y frente a un *naive* de retorno nulo, esto es, un paseo aleatorio; la métrica de referencia es el cociente entre el RMSE de validación del modelo y el del *naive*, de manera que un valor por debajo de 1,0 indica que se bate al paseo aleatorio.

Con ese criterio se compararon las líneas base estadísticas (*naive*, *momentum* y *ARIMA*), las tres redes para el procesamiento de secuencias (*LSTM*, *GRU* y *Transformer*) y sus variantes con VMD, incluida una versión multirrama que asigna una rama recurrente a cada modo de la descomposición. La Tabla 3.8 recoge el RMSE por *ticker*, medido en puntos porcentuales del retorno a diez días y con hiperparámetros fijos. El

primer resultado es sobrio: todas las redes batien al *naive* de media, pero por márgenes pequeños, y solo en los valores más volátiles (ABNB, TSLA, NVDA) la mejora es apreciable; en los estables (MSFT, DDOG, INTC, PEP) el retorno a diez días apenas es más predecible que un paseo aleatorio.

Tabla 3.8: RMSE por *ticker* del retorno a diez días (puntos porcentuales), *pipeline* causal con hiperparámetros fijos. Las variantes VMD usan las características auxiliares OHL. En negrita, el mejor modelo por fila.

<i>Ticker</i>	naive	LSTM	GRU	Transf.	VMD-LSTM	VMD-GRU	VMD-Transf.
AAPL	5,764	5,644	5,675	5,682	5,653	5,672	5,666
NVDA	13,714	13,622	13,572	13,592	13,645	13,630	13,622
TSLA	11,979	11,831	12,233	11,936	11,884	11,890	11,815
MSFT	4,829	4,828	4,834	4,841	4,848	4,842	4,871
ABNB	6,339	5,747	5,964	6,070	5,868	5,930	5,987
DDOG	10,173	10,173	10,178	10,273	10,171	10,166	10,192
PEP	3,711	3,686	3,682	3,681	3,671	3,691	3,683
INTC	8,733	8,840	8,831	8,735	8,719	8,743	8,738
Media	8,155	8,046	8,121	8,101	8,057	8,071	8,072

El segundo resultado es el que decidió la arquitectura. La ventaja que la literatura atribuye a la VMD se desvanece al aplicar la descomposición de forma causal: las diferencias entre cada red y su variante con VMD quedan por debajo del propio ruido de ejecución del entrenamiento (la LSTM sobre GPU no es determinista, y su variabilidad entre repeticiones es mayor que cualquiera de esas diferencias). Entre LSTM y GRU ocurre lo mismo, empatan dentro del margen de ruido. Al no haber una ganancia clara, se aplicó una regla acordada de antemano: en caso de empate gana la GRU, por ser determinista, tener en torno a un 25 % menos de parámetros y entrenar más rápido, algo relevante cuando la producción reentrena a diario y por *ticker* sobre CPU. En cuanto a las características auxiliares, el estudio confirmó que la señal la aporta la descomposición del cierre y que añadir a la ventana los niveles OHL o los rasgos intradía de rango, cuerpo y hueco no cambia el resultado de forma medible.

Optimización de hiperparámetros y arquitectura final

Elegida la GRU, se ajustaron sus hiperparámetros con Optuna bajo el mismo *pipeline* causal, para no reintroducir por la puerta de atrás las fugas que se acababan de eliminar. El estudio usa un muestreador TPE y un podador Hyperband, de modo que cada intento informa su pérdida de validación por época y las configuraciones débiles se detienen pronto; el objetivo es la media, sobre los *tickers* de ajuste (AAPL, NVDA y PEP), del RMSE de validación dividido por el del *naive*, promediado sobre dos semillas para absorber el ruido entre ejecuciones. El espacio de búsqueda cubre la tasa de aprendizaje, la ventana de *lookback*, el tamaño oculto, el número de capas, las unidades densas, el *dropout* y el tamaño de lote; el número de épocas no se busca, ya que todas las pruebas se apoyan en la parada temprana.

La elección de Optuna no fue inmediata. En un principio se valoró optimizar la red con el propio *Sparrow Search Algorithm* (SSA), la metaheurística de enjambre que varios de los trabajos revisados aplican al ajuste de hiperparámetros de redes LSTM con buenos resultados [14,32]. Se descartó por su coste: al ser un método poblacional evalúa una población entera de configuraciones en cada iteración, cada evaluación exige un entrenamiento completo y no incorpora un mecanismo de poda que abandone pronto las candidatas malas. Bajo las restricciones del proyecto, con entrenamiento en CPU y, en la variante con VMD, una descomposición de historial completo por cada nueva ventana, ese presupuesto resultaba difícil de asumir. La combinación de muestreador TPE y podador Hyperband concentra el esfuerzo en las configuraciones prometedoras y detiene pronto el resto, y además Optuna aporta persistencia en SQLite, reproducibilidad y reanudación del estudio sin tener que implementar y calibrar la metaheurística a mano.

Se afinaron en paralelo la GRU sencilla y su versión con VMD para zanjar la duda que el *benchmark* había dejado abierta. La GRU alcanzó un objetivo de 0,9803 frente al 0,9830 de la VMD-GRU: la diferencia es pequeña, pero apunta en contra de la VMD, que además arrastra costes que la GRU no tiene, como un calentamiento de 512 sesiones que deja casi inservibles los valores de historial corto, una descomposición por muestra en el momento de inferencia y más parámetros en el óptimo. Con esto la decisión quedó cerrada: producción emplea la GRU sencilla, sin VMD. El análisis de

importancia de hiperparámetros mostró además que la ventana de *lookback* domina el resultado (importancia de 0,87), muy por encima del resto de parámetros, coherente con la idea de que lo decisivo es cuánta historia reciente ve la red. La Tabla 3.9 resume la comparación final sobre el conjunto de prueba reservado.

Tabla 3.9: GRU ajustada frente a la configuración por defecto y al *naive* sobre el conjunto de prueba de AAPL. RMSE y MAE en puntos porcentuales del retorno a diez días.

Modelo	RMSE	MAE	Acierto direccional	Parámetros
naive	5,7339	4,5387	n/a	0
GRU (por defecto)	5,6245	4,4080	0,6025	15 553
GRU (ajustada)	5,6108	4,3916	0,6025	6 724

La configuración ganadora es, además, más de dos veces menor que la de referencia (6 724 frente a 15 553 parámetros), lo que reduce el coste del reentrenamiento diario sobre CPU sin perder precisión. Esa receta se congela como el contrato que consume el entrenamiento por *ticker* en producción, recogido en el Código 3.21. La Fig. 3.6 muestra el flujo completo de la red, desde la ventana de entrada hasta la etiqueta de tendencia.

```

1 {
2   "architecture": "gru",
3   "use_vmd": false,
4   "horizon": 10,
5   "params": {
6     "lr": 0.00034434073864277764,
7     "lookback": 24,
8     "hidden_size": 37,
9     "num_layers": 1,
10    "dense_units": 50,
11    "dropout": 0.24283099538508776,
12    "batch_size": 16
13  }
14 }
```

Código 3.21: Receta congelada de la GRU (*gru_frozen_config.json*), producto del estudio de Optuna y contrato del entrenamiento por *ticker*.

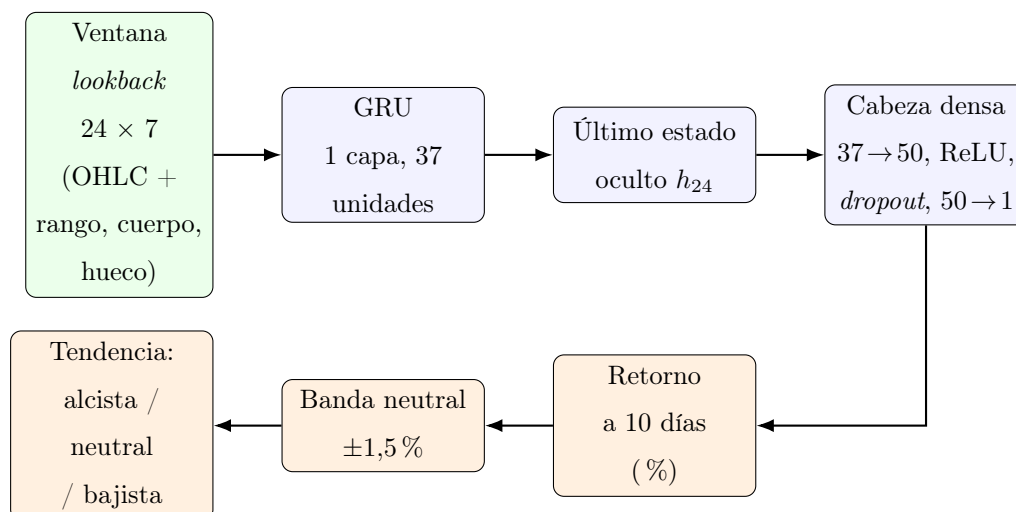


Figura 3.6: Flujo de la red GRU de producción: la ventana tipificada se resume en el último estado oculto, una cabeza densa predice el retorno a diez días y una banda neutral lo traduce a una etiqueta de tendencia.

Integración en el sistema

Llevar la receta al *backend* exige que el entrenamiento y la inferencia apliquen exactamente el mismo preprocesamiento; cualquier discrepancia invalidaría las predicciones. Por eso el pipeline de características es una única fuente de verdad compartida por ambos: a partir de la serie OHLC limpia se calculan tres rasgos intradía causales (rango, cuerpo y hueco), se construye la ventana de las *lookback* sesiones más recientes y se tipifica con su propia media y desviación, de modo que cada muestra depende solo de datos hasta su último día conocido. El modelo es la misma clase `GRURegressor` tanto para ajustar los pesos como para reconstruir la arquitectura antes de cargar el estado entrenado.

En producción los pesos se reajustan a diario y por *ticker*, pero la receta nunca cambia. El reentrenamiento parte en caliente de los pesos existentes cuando los hay, descarga solo el histórico necesario para cubrir el hueco desde la última actualización y lo lanza un trabajo de `APScheduler` embebido en la aplicación. La inferencia carga los modelos de forma perezosa con una caché LRU acotada, toma la ventana más reciente de precios de *yfinance*, ejecuta la pasada hacia delante sobre CPU y obtiene el retorno a diez días. Ese retorno se convierte por último en la etiqueta que ve el usuario aplicando una banda neutral de $\pm 1,5\%$, como recoge el Código 3.22: fuera de la banda

la tendencia es alcista o bajista, y dentro de ella se declara neutral. Este umbral evita presentar como señal lo que es indistinguible de ruido, algo especialmente pertinente dado que el retorno a diez días apenas mejora al paseo aleatorio.

```
1 raw_return = await asyncio.to_thread(self._forward, model, X)
2 current_price = float(ohlcv["close"].iloc[-1])
3 predicted_price = current_price * (1.0 + raw_return / 100.0)
4
5 if raw_return > 1.5:
6     trend = "alcista"
7 elif raw_return < -1.5:
8     trend = "bajista"
9 else:
10    trend = "neutral"
```

Código 3.22: Traducción del retorno predicho a una etiqueta de tendencia mediante la banda neutral (extracto de `deep_learning/service.py`).

Junto a la tendencia, el módulo devuelve el retorno y el precio derivados y las métricas de calidad del modelo (RMSE, MAE y acierto direccional) calculadas sobre el conjunto de prueba durante el último entrenamiento, de forma que el informe pueda acompañar la señal con una idea honesta de su fiabilidad. Conviene no exagerar el alcance de esta predicción: como muestra el estudio comparativo, la mejor puntuación de validación rondó 0,98 y el acierto direccional se mueve alrededor del 0,56, cifras coherentes con lo que la literatura advierte sobre la escasa señal de los retornos a corto plazo [27]. Esa modestia es precisamente la que justifica tanto la banda neutral del clasificador como el aviso legal que acompaña al informe.

3.2.7. Implementación

El proyecto se organiza como un único repositorio con tres subproyectos independientes: el backend en FastAPI, el frontend en React y un entorno de investigación con los cuadernos del módulo de *deep learning*. Cada uno gestiona sus dependencias por separado, con *uv* en los dos de Python y *npm* en el de JavaScript, de modo que los ficheros de bloqueo (`uv.lock` y `package-lock.json`) fijan versiones reproducibles

y el entorno de investigación no arrastra las dependencias pesadas del backend. Esta memoria se redactó en L^AT_EX sobre Overleaf, en paralelo al desarrollo.

El trabajo se llevó a cabo siguiendo un flujo ligero de integración continua. Cada unidad de trabajo nace como un *issue* en GitHub, se desarrolla en una rama propia y se integra mediante un *pull request* que no se fusiona hasta que los controles automáticos están en verde; la planificación se apoya en un tablero *kanban* de GitHub Projects [3] y las decisiones de diseño relevantes quedan registradas como *Architecture Decision Records*. Hay tres flujos de integración continua en GitHub Actions: uno para el backend (análisis estático y formato con Ruff, más la batería de *pytest* con medida de cobertura), otro para el frontend (ESLint, comprobación de tipos con el compilador de TypeScript y *tests* con Vitest) y un tercero de análisis de seguridad con CodeQL sobre el código Python y TypeScript. El despliegue es automático en Render desde la rama principal, con el backend como servicio web y el frontend como sitio estático, ambos declarados en un único fichero `render.yaml`.

El desarrollo se hizo en Visual Studio Code, apoyado en el asistente Claude Code para tareas de programación, cuyo uso se detalla en el apartado dedicado a las herramientas de inteligencia artificial. Los *endpoints* se probaron con peticiones desde la terminal (*curl* y HTTPie) y el grafo de conocimiento se inspeccionó con consultas Cypher desde Neo4j Browser y la consola de AuraDB. El entrenamiento y la optimización de hiperparámetros se ejecutaron en local aprovechando la GPU, una RTX 4060 con CUDA, mientras que en producción la inferencia corre sobre CPU por las limitaciones de la plataforma de despliegue. Para desarrollar sin coste de API se permitió el uso de Ollama con modelos locales, algo que el patrón adaptador hace posible al permitir intercambiar el proveedor del LLM sin tocar la lógica de los módulos.

En el backend, sobre FastAPI y Uvicorn, la configuración se centraliza con *pydantic-settings* y las llamadas HTTP salientes usan *httpx*. El módulo de sentimiento se apoya en el controlador de Neo4j y en *neo4j-graphrag* para la recuperación vectorial, y en el cliente de OpenAI para el LLM y los *embeddings*. Los datos de mercado llegan por *yfinance* y *finnhub-python*, el modelo GRU se implementa en PyTorch (con NumPy, pandas y SciPy en el preprocesamiento), el reentrenamiento diario lo programa APScheduler, la caché de resultados se apoya en *cachetools* y la limitación de peticiones

en *slowapi*. El entorno de investigación añade las piezas que no están incluidas en producción: *Optuna* para la búsqueda de hiperparámetros, *vmcpy* para la descomposición VMD, *statsmodels* para la línea base ARIMA, *scikit-learn* para las utilidades de métrica y partición, y *matplotlib* para las figuras del estudio.

El frontend es una aplicación de página única en React con Vite y TypeScript, con Tailwind CSS para el estilo y React Router para la navegación. Las pruebas se escriben con Vitest y Testing Library, las llamadas a la API se simulan con Mock Service Worker y la accesibilidad se comprueba de forma automática con *vitest-axe* y las reglas *jsx-a11y* de ESLint, en coherencia con el objetivo de cumplir el nivel AA de las WCAG.

Un par de decisiones de implementación tienen consecuencias que conviene señalar. El preprocesamiento del módulo de *deep learning* es una única fuente de verdad compartida por el entrenamiento y la inferencia, para que ambos apliquen exactamente las mismas transformaciones causales y una predicción nunca dependa de datos que en explotación no existirían. Las claves de API nunca viven en el repositorio: se inyectan como variables de entorno y en Render se marcan como secretos. Y para no tener que configurar CORS en producción, el sitio estático reescribe las rutas `/api` hacia el backend, de modo que el navegador siempre habla con el mismo origen.

3.2.8. Métricas

De los cuatro módulos, solo el de *deep learning* produce una salida que puede contrastarse contra un valor real observado, el retorno a diez días que el mercado acaba materializando, y por eso es el único que admite una evaluación cuantitativa por error. Todas las métricas que siguen se calculan sobre ese objetivo, medido en puntos porcentuales del retorno a diez días, y sobre el conjunto de prueba reservado del *pipeline* causal descrito en la sección de las series de precios de entrenamiento. Con la notación habitual, n es el número de muestras, y_i el retorno realmente observado y $\hat{y}_i$ el que predice la red.

El entrenamiento minimiza el error cuadrático medio (MSE), que promedia el cuadrado de la diferencia entre predicción y valor real:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.1)$$

Elevar el error al cuadrado penaliza con más fuerza los fallos grandes que los pequeños, lo que da un gradiente estable al descenso con Adam. Su inconveniente para informar de resultados es que queda en unidades al cuadrado, poco intuitivas, así que para presentar la calidad del modelo se usa su raíz, la raíz del error cuadrático medio (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (3.2)$$

El RMSE recupera las unidades del objetivo, puntos porcentuales de retorno, y es la métrica principal del estudio comparativo: las Tablas 3.8 y 3.9 lo recogen por *ticker* y para la configuración final. Junto a él se reporta el error absoluto medio (MAE), que promedia el valor absoluto del error en lugar de su cuadrado:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.3)$$

Al no elevar al cuadrado, el MAE pesa todos los errores por igual y es menos sensible a los días atípicos. Comparar RMSE y MAE da entonces una idea de cuánto pesan los errores extremos: cuanto más se separa el RMSE del MAE, más dispersos son los fallos del modelo.

Las tres métricas anteriores miden la magnitud del error, pero no si la red acierta la dirección del movimiento, que es lo que finalmente ve el usuario tras aplicar la banda neutral. Para eso se usa el acierto direccional (AD), la proporción de muestras en las que el signo del retorno predicho coincide con el del observado:

$$\text{AD} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\text{sign}(\hat{y}_i) = \text{sign}(y_i)] \quad (3.4)$$

donde $\mathbf{1}[\cdot]$ vale uno cuando la condición se cumple y cero en caso contrario. Un valor de 0,5 equivale a acertar la dirección la mitad de las veces, lo mismo que conseguiría

un lanzamiento de moneda; las cifras obtenidas rondan 0,56, algo por encima del azar pero sin margen para venderlo como una señal fuerte.

Comparar el RMSE entre *tickers* tiene un inconveniente: un valor volátil como NVDA acumula un error en puntos porcentuales mucho mayor que uno estable como PEP por el simple hecho de moverse más, sin que eso diga nada sobre la habilidad del modelo. Para poder agregar resultados de valores con volatilidades distintas se normaliza el RMSE del modelo por el de una referencia ingenua, el *naive* de retorno nulo que siempre predice $\hat{y}_i = 0$:

$$U = \frac{\text{RMSE}_{\text{modelo}}}{\text{RMSE}_{\text{naive}}}, \quad \text{RMSE}_{\text{naive}} = \sqrt{\frac{1}{n} \sum_{i=1}^n y_i^2} \quad (3.5)$$

Como el *naive* predice cero, su error es el propio retorno observado y su RMSE coincide con la raíz del segundo momento de la serie, de modo que U queda adimensional y comparable entre valores. Un cociente por debajo de 1,0 indica que el modelo bate al paseo aleatorio, y uno por encima que no lo mejora. Esta razón, conocida en la literatura de predicción como coeficiente U de Theil, es la que se emplea como objetivo de la optimización con Optuna, promediada sobre los *tickers* de ajuste y sobre dos semillas para absorber el ruido entre ejecuciones. Los valores alcanzados, en torno a 0,98, confirman que la ventaja sobre el azar existe pero es estrecha, en la línea de lo que ya adelantaba el estudio comparativo.

Los otros tres módulos no se prestan a esta clase de medida. El de análisis de sentimiento, el fundamental y el técnico no producen un número que pueda restarse a una verdad de terreno equivalente. El de sentimiento devuelve una etiqueta, una puntuación y una confianza que el LLM infiere sobre las noticias recuperadas del grafo, y el fundamental una explicación en lenguaje natural a partir de las magnitudes financieras; en ninguno de los dos existe un “sentimiento verdadero” ni una “explicación correcta” objetivos contra los que calcular un error. Su calidad se valora de forma cualitativa, atendiendo a la coherencia del texto y a su fidelidad a los datos de partida, y se acota con el aviso legal que acompaña siempre al informe. El módulo técnico es un caso aparte: sus señales salen de indicadores deterministas (RSI, MACD, bandas de Bollinger

y medias móviles), fórmulas cerradas que no aprenden de datos y cuya corrección se comprueba verificando el cálculo, no midiendo un error de predicción.

Al margen de las métricas de modelo, el proyecto se apoya en varias métricas de calidad de software para vigilar el estado del código. La batería de *pytest* se ejecuta con medida de cobertura, de modo que cada cambio informa de qué proporción del *backend* queda ejercitada por las pruebas, y el *frontend* hace lo propio con Vitest. A esto se suman las comprobaciones que deben quedar en verde antes de integrar cualquier rama: el análisis estático y de formato con Ruff y ESLint, la comprobación de tipos con el compilador de TypeScript y el análisis de seguridad con CodeQL. No son métricas del dominio financiero, pero son las que sostienen que el resto de los resultados sea reproducible.

3.2.9. Pruebas

El desarrollo se llevó a cabo siguiendo una variante ligera de desarrollo dirigido por pruebas (TDD). Cada unidad de trabajo empieza por escribir la prueba que describe el comportamiento buscado y comprobar que falla, sigue con la implementación mínima que la hace pasar y termina verificando que ninguna de las pruebas anteriores se ha roto. Este orden no es un formalismo: obliga a fijar el criterio de aceptación antes de tocar el código y deja una red de regresión que crece con el proyecto. La prueba, además, se integra en el flujo descrito en la sección de implementación, ya que un *pull request* no se fusiona hasta que la batería completa pasa en integración continua.

La estrategia se organiza como una pirámide con base ancha de pruebas unitarias, que ejercitan funciones y clases de forma aislada, y una capa más delgada de pruebas de sistema que recorren los *endpoints* de la API tal como los consume el frontend. Todas las pruebas son herméticas: no hay llamadas de red reales, sino dobles de prueba que sustituyen a los proveedores externos, de modo que la suite es determinista y no consume las cuotas de Finnhub, NewsAPI ni OpenAI. La Tabla 3.10 resume su reparto por subproyecto.

En el backend las pruebas unitarias se concentran donde el error sería más silencioso. El caso más representativo es el del preprocesamiento del módulo de *deep learning*:

Tabla 3.10: Reparto de la batería de pruebas por subproyecto y utillaje empleado.

Subproyecto	Ficheros	Casos	Utillaje
<i>Backend</i> (FastAPI)	37	217	<i>pytest, pytest-asyncio, pytest-cov</i>
<i>Frontend</i> (React)	13	50	Vitest, Testing Library, MSW, <i>vitest-axe</i>
Investigación (<i>notebooks</i>)	4	23	<i>pytest</i>
Total	54	290	

como todo el estudio comparativo dependía de eliminar la fuga de información, la propiedad de causalidad se codifica directamente en una prueba. El Código 3.23 construye las ventanas, altera los datos posteriores a un día t_0 y verifica que las muestras cuyo pasado y cuyo objetivo no se han tocado siguen siendo idénticas bit a bit. Si una futura modificación reintrodujera una fuga, esta prueba fallaría de inmediato.

```

1 def test_build_windows_are_causal():
2     features = rng.normal(0.0, 1.0, size=(n, 7))
3     close = 100 + np.cumsum(rng.normal(0, 1, n))
4     lookback, horizon = 24, 10
5     base = pp.build_windows(features, close, dates, lookback, horizon)
6
7     # Alterar los datos estrictamente posteriores a t0; las muestras que
8     # terminan en t0 o antes (y cuyo objetivo t0+horizon no se toca) no cambian.
9     t0 = 40
10    features2, close2 = features.copy(), close.copy()
11    features2[t0 + 1:] += 99.0
12    close2[t0 + horizon + 1:] += 99.0
13    mutated = pp.build_windows(features2, close2, dates, lookback, horizon)
14
15    n_safe = t0 - (lookback - 1) + 1
16    np.testing.assert_array_equal(base.X[:n_safe], mutated.X[:n_safe])
17    np.testing.assert_array_equal(base.y[:n_safe], mutated.y[:n_safe])

```

Código 3.23: Prueba unitaria que fija la causalidad del preprocesamiento: mutar el futuro no altera las muestras ya cerradas
(tests/services/deep_learning/test_preprocessing.py).

Otra técnica que se empleó en el módulo técnico es la prueba de caracterización o *golden test*. Antes de refactorizar el motor de indicadores heredado del colaborador de

Finanzas, se fijó su salida numérica sobre un universo sintético y sin red en un fichero de referencia; el refactor solo se dio por bueno cuando la salida seguía coincidiendo con esa base dentro de una tolerancia estricta. Así el cambio de estructura interna quedó demostrado como conservador del comportamiento, no simplemente supuesto.

La capa de sistema comprueba que la aplicación cumple los casos de uso a través de su interfaz real. El endpoint del informe consolidado se prueba con el `TestClient` de FastAPI y con sustitución de dependencias (`app.dependency_overrides`), que reemplaza los cuatro servicios de análisis y el LLM por dobles con salidas fijas. Sobre ese montaje se verifica tanto el camino satisfactorio, en el que la respuesta trae las cuatro secciones, la conclusión global y el aviso legal, como los bordes: un *ticker* demasiado largo devuelve 422, uno internacional con punto se acepta y normaliza, y la caída de un módulo se traduce en un informe parcial en lugar de un error total. El Código 3.24 recoge dos de estas comprobaciones.

```
1 def test_get_report_returns_200_with_full_response(client, mock_services):
2     response = client.get("/api/v1/report/AAPL")
3     assert response.status_code == 200
4     data = response.json()
5     assert data["ticker"] == "AAPL"
6     assert data["sentiment"] is not None
7     assert data["deep_learning"] is not None
8     assert data["global_conclusion"] == _LLM_CONCLUSION
9     assert data["partial_support"] is False
10    assert "disclaimer" in data
11
12
13 def test_get_report_invalid_ticker_returns_422(client, mock_services):
14     response = client.get("/api/v1/report/TOOLONGTICKERXYZ") # 16 > limite
15     assert response.status_code == 422
```

Código 3.24: Prueba de sistema del informe consolidado: camino completo y validación del *ticker* (extracto de `tests/api/test_report.py`).

En el frontend las pruebas van más allá de la lógica de los componentes y cubren también la accesibilidad, en coherencia con el objetivo de cumplir el nivel AA de las WCAG. Cada componente relevante se renderiza con Testing Library, se simula la

interacción del usuario con teclado y ratón y se pasa por *vitest-axe*, que falla si detecta violaciones de accesibilidad; las llamadas a la API se interceptan con Mock Service Worker para no depender de un backend en marcha. Las reglas *jsx-a11y* de ESLint añaden una segunda barrera en tiempo de análisis estático.

Toda esta batería se ejecuta de forma automática en GitHub Actions en cada *push* y en cada *pull request*. El flujo del backend primero comprueba estilo y formato con Ruff y, si pasa, ejecuta *pytest* con medida de cobertura y una puerta que hace fallar la construcción si la cobertura de **app** baja del 55 %. El del frontend encadena ESLint, la comprobación de tipos con el compilador de TypeScript, los *tests* de Vitest y la construcción de producción con Vite. Un tercer flujo pasa CodeQL sobre el código Python y TypeScript. Ninguna rama llega a la principal sin que los tres estén en verde, de modo que la reproducibilidad de los resultados no depende de la disciplina manual sino de la propia canalización.

3.3. El producto del desarrollo

El resultado del trabajo es FinSight, una aplicación web que convierte el nombre o el símbolo de una acción en un informe de análisis escrito en español y comprensible sin conocimientos financieros previos. La persona escribe un valor en la página de inicio, la aplicación lanza los cuatro módulos de análisis en paralelo, un modelo de lenguaje redacta la conclusión global y la interfaz muestra el conjunto en una sola página. El frontend es una aplicación de página única en React desplegada en Render como sitio estático, y el backend es la API de FastAPI descrita en los apartados anteriores; el navegador habla siempre con el mismo origen, que reescribe las rutas */api* hacia el servicio web.

Las capturas de esta sección se han tomado sobre la interfaz real en funcionamiento. Los valores numéricos que aparecen en los informes son datos de ejemplo representativos del entorno de desarrollo: los servicios en vivo (modelos entrenados por *ticker*, LLM y proveedores de noticias y precios) exigen credenciales y no intervienen en la demostración de la interfaz, de modo que las cifras concretas son ilustrativas aunque la disposición y el comportamiento son los del producto final.

3.3.1. Página de inicio

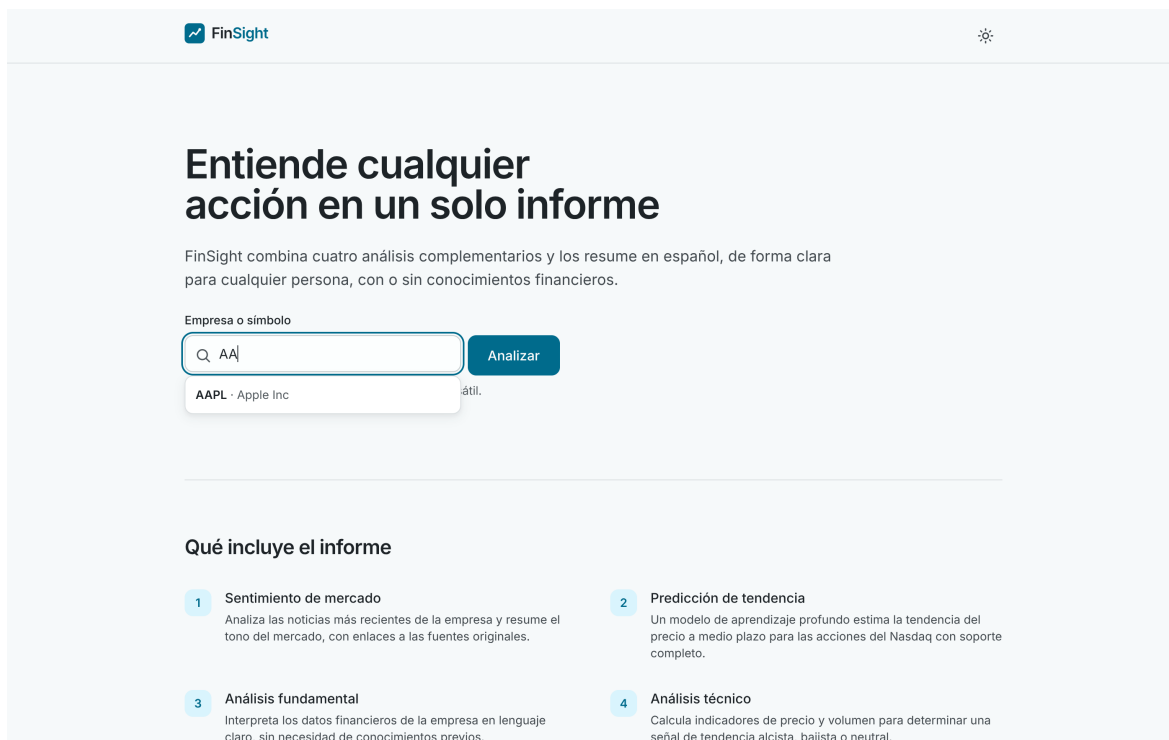
La página de inicio, en la Fig. 3.7, presenta la propuesta de la herramienta y un único punto de entrada: un buscador donde introducir la empresa o su símbolo. Debajo, un bloque explica en lenguaje llano qué contendrá el informe, con los cuatro análisis que lo componen. El buscador ofrece autocompletado: al teclear, la aplicación consulta el *endpoint* de búsqueda de símbolos y despliega las coincidencias, como muestra la Fig. 3.8. Aceptar una sugerencia o pulsar «Analizar» lleva a la ruta del informe del valor elegido.



Figura 3.7: Página de inicio: propuesta de la herramienta, buscador de valores y resumen de los cuatro análisis que integra el informe.

3.3.2. El informe de análisis

El informe es el núcleo del producto. La Fig. 3.9 recoge un informe completo. En la cabecera, cuatro tarjetas resumen de un vistazo el sentimiento, la tendencia predicha, la puntuación fundamental y la señal técnica. Debajo, cada módulo se despliega en su propia sección: el análisis de sentimiento muestra la etiqueta, la puntuación en el rango de -1 a 1 , la confianza y las noticias más influyentes con enlace a la fuente;



FinSight

Entiende cualquier acción en un solo informe

FinSight combina cuatro análisis complementarios y los resume en español, de forma clara para cualquier persona, con o sin conocimientos financieros.

Empresa o símbolo

Analizar

AAPL - Apple Inc. AAPL - Apple Inc. (NASDAQ: AAPL)

Qué incluye el informe

- 1 Sentimiento de mercado**
Analiza las noticias más recientes de la empresa y resume el tono del mercado, con enlaces a las fuentes originales.
- 2 Predicción de tendencia**
Un modelo de aprendizaje profundo estima la tendencia del precio a medio plazo para las acciones del Nasdaq con soporte completo.
- 3 Análisis fundamental**
Interpreta los datos financieros de la empresa en lenguaje claro, sin necesidad de conocimientos previos.
- 4 Análisis técnico**
Calcula indicadores de precio y volumen para determinar una señal de tendencia alcista, bajista o neutral.

Figura 3.8: Autocompletado del buscador: al escribir se consultan los símbolos coincidentes y se ofrecen como sugerencias.

la predicción de tendencia expone la etiqueta (alcista, bajista o neutral), el retorno estimado a diez días, el precio actual y el derivado, y las métricas de calidad del modelo (RMSE, MAE y acierto direccional) tal como se definen en el apartado de métricas; los análisis fundamental y técnico acompañan su puntuación de una explicación en lenguaje natural y de un desplegable con el detalle. La página se cierra con la conclusión global que sintetiza los cuatro módulos y con el aviso legal. Cada bloque generado por el modelo de lenguaje se marca de forma explícita para que la persona sepa qué parte procede de una síntesis automática.

Para acercar el análisis a quien no domina la jerga financiera, cada métrica de los desplegables de detalle lleva junto a su nombre un botón con una «i» de información. Al pulsarlo se despliega en línea una explicación breve del campo, tomada de un glosario interno, y al volver a pulsarlo se oculta. La Fig. 3.10 recoge el desplegable de ratios fundamentales con la explicación del ROE abierta. El mismo control acompaña a las métricas de las secciones principales, como la puntuación y la confianza del sentimiento

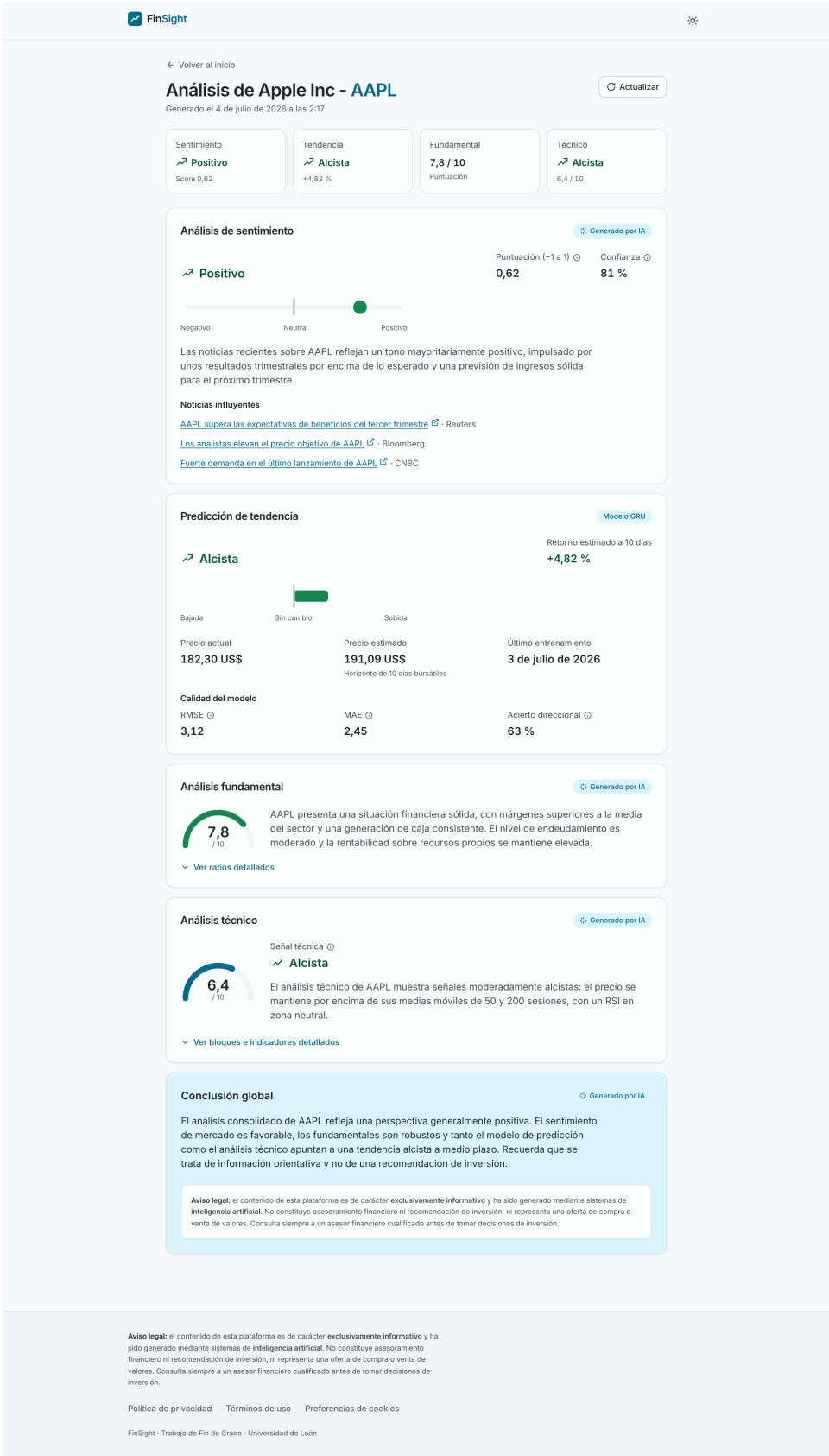


Figura 3.9: Informe completo de un valor: tarjetas de resumen, los cuatro módulos de análisis con su detalle y la conclusión global. Los datos mostrados son de ejemplo.

o las medidas de calidad del modelo de predicción, de modo que el significado de cada dato queda a un clic sin salir del informe.

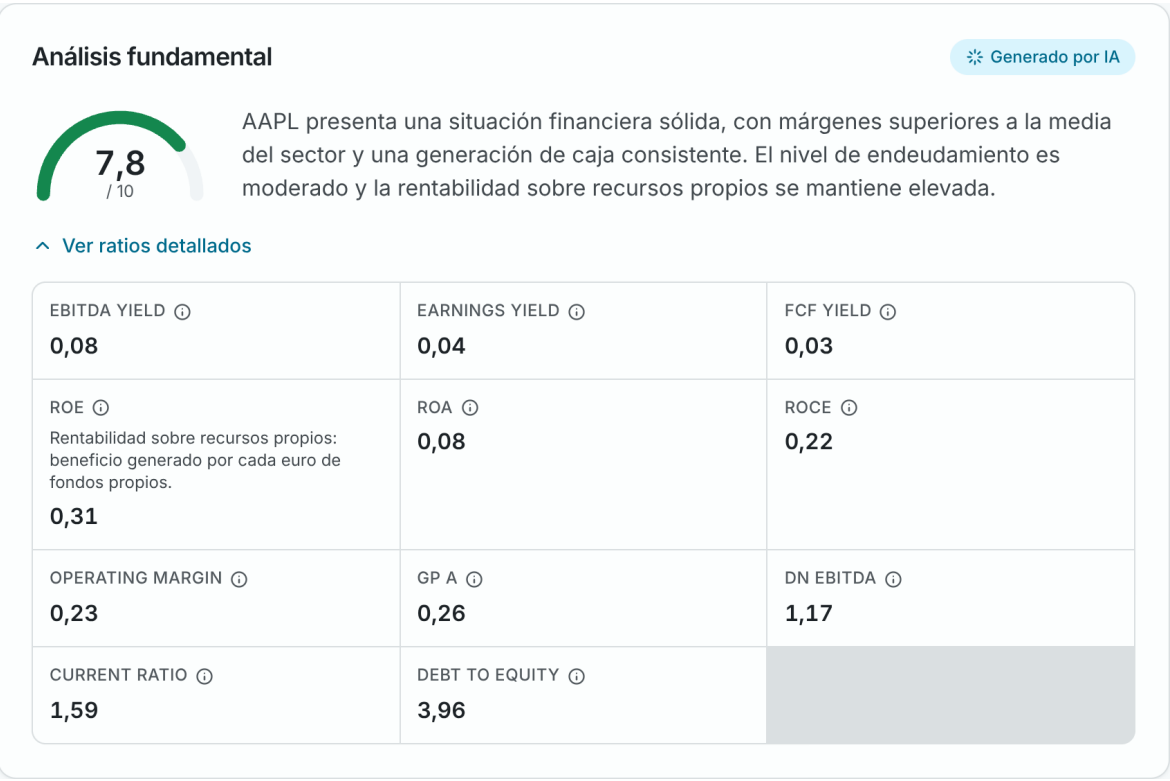


Figura 3.10: Desplegable de ratios fundamentales: cada métrica ofrece un botón de información («i») que revela una breve explicación del campo, aquí abierta para el ROE.

3.3.3. Cobertura parcial y estados de error

El sistema está pensado para degradarse con elegancia cuando algún módulo no puede completarse. Los modelos de *deep learning* solo cubren el universo del Nasdaq validado por la optimización, de modo que para un valor sin modelo disponible la aplicación no falla por completo: presenta el resto de análisis y avisa con claridad de lo que falta. La Fig. 3.11 ilustra este caso de soporte parcial, con la tarjeta de tendencia marcada como no disponible y un mensaje que lo explica, mientras el sentimiento, el fundamental y el técnico se muestran con normalidad. Cuando el fallo es de conexión, la página del informe presenta un mensaje de error con un botón para reintentar, previendo el arranque en frío del servidor tras un periodo de inactividad.

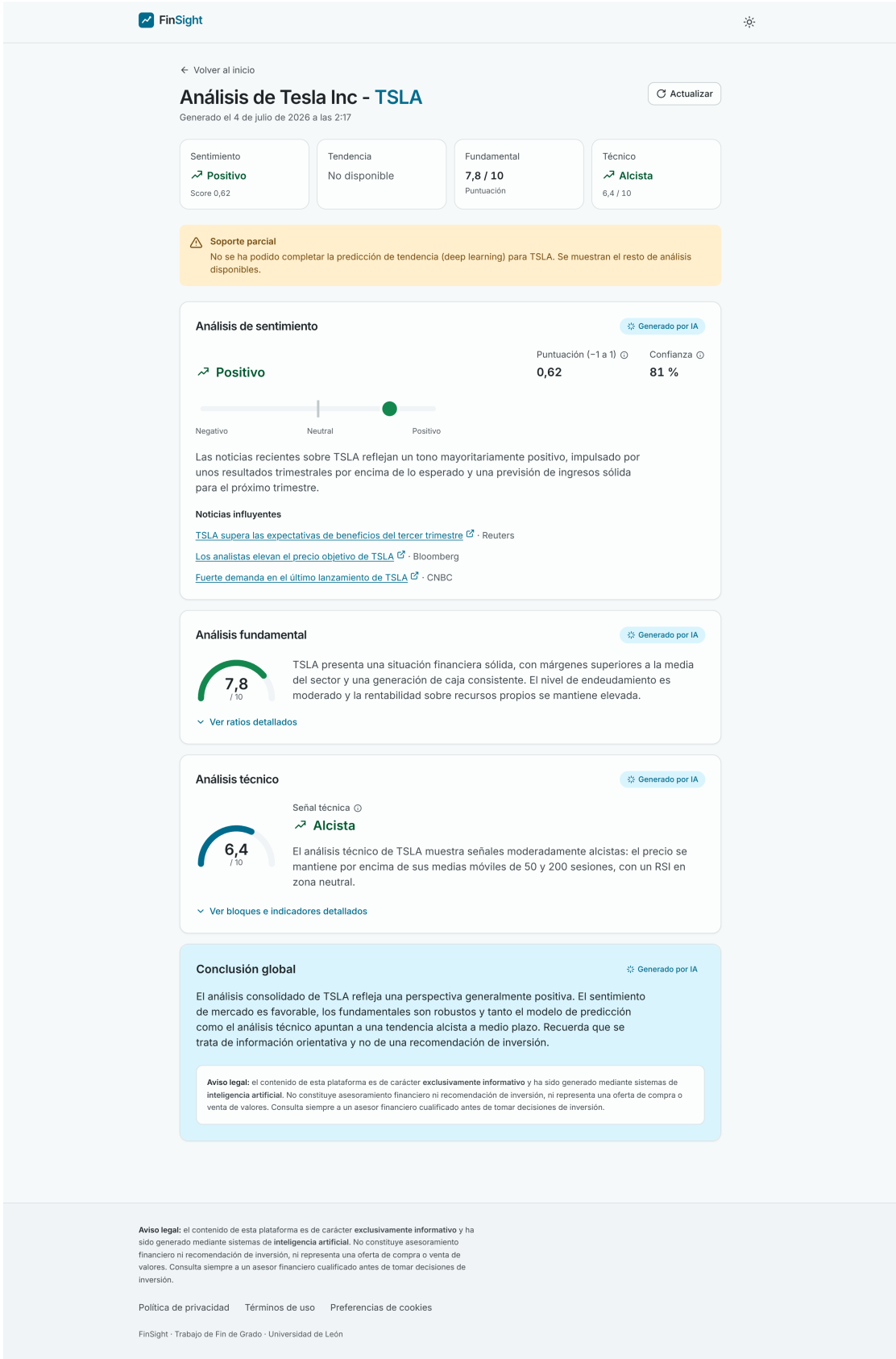


Figura 3.11: Informe con soporte parcial: no hay modelo de tendencia para el valor, así que ese bloque se marca como no disponible y el resto de análisis se presenta igualmente.

3.3.4. Consentimiento, privacidad y aviso legal

Al ser un servicio dirigido al público en España, la interfaz incorpora las salvaguardas legales desde el primer contacto. En la primera visita aparece un banner de consentimiento de cookies, en la Fig. 3.12, con opciones para aceptar, rechazar o configurar el almacenamiento. El diálogo de preferencias de la Fig. 3.13 permite un consentimiento granular por categorías: estrictamente necesarias, funcionales y analíticas, en línea con lo exigido por la normativa. El aviso legal que recuerda el carácter meramente informativo del análisis está presente de forma permanente en el pie de página y también dentro del propio informe.



Figura 3.12: Banner de consentimiento de cookies en la primera visita, con acceso directo a la configuración por categorías.

3.3.5. Tema visual y páginas de servicio

La interfaz ofrece tema claro y oscuro, conmutables desde la cabecera, con la elección recordada solo si se ha concedido el consentimiento funcional. La Fig. 3.14 muestra la página de inicio en tema oscuro. Completan el producto las páginas de servicio: la política de privacidad (Fig. 3.15) y los términos de uso (Fig. 3.16), enlazadas desde

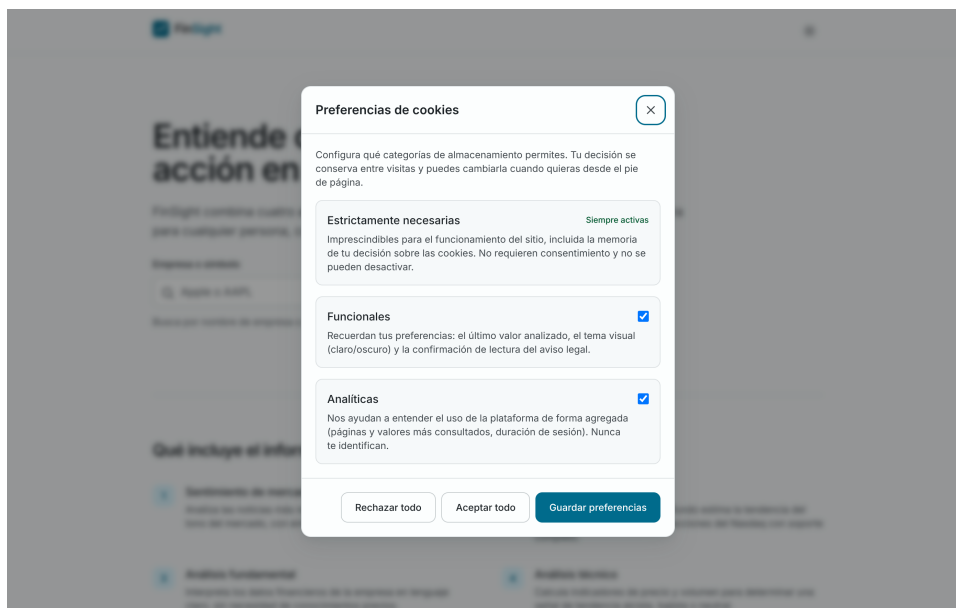


Figura 3.13: Diálogo de preferencias con consentimiento granular por categorías de almacenamiento.

el pie, y una página de error para rutas inexistentes (Fig. 3.17). Todo el conjunto se desarrolló atendiendo al nivel AA de las WCAG, lo que se refleja en el contraste, la navegación por teclado y el etiquetado accesible de los controles.



Figura 3.14: Página de inicio en tema oscuro, conmutable desde la cabecera.

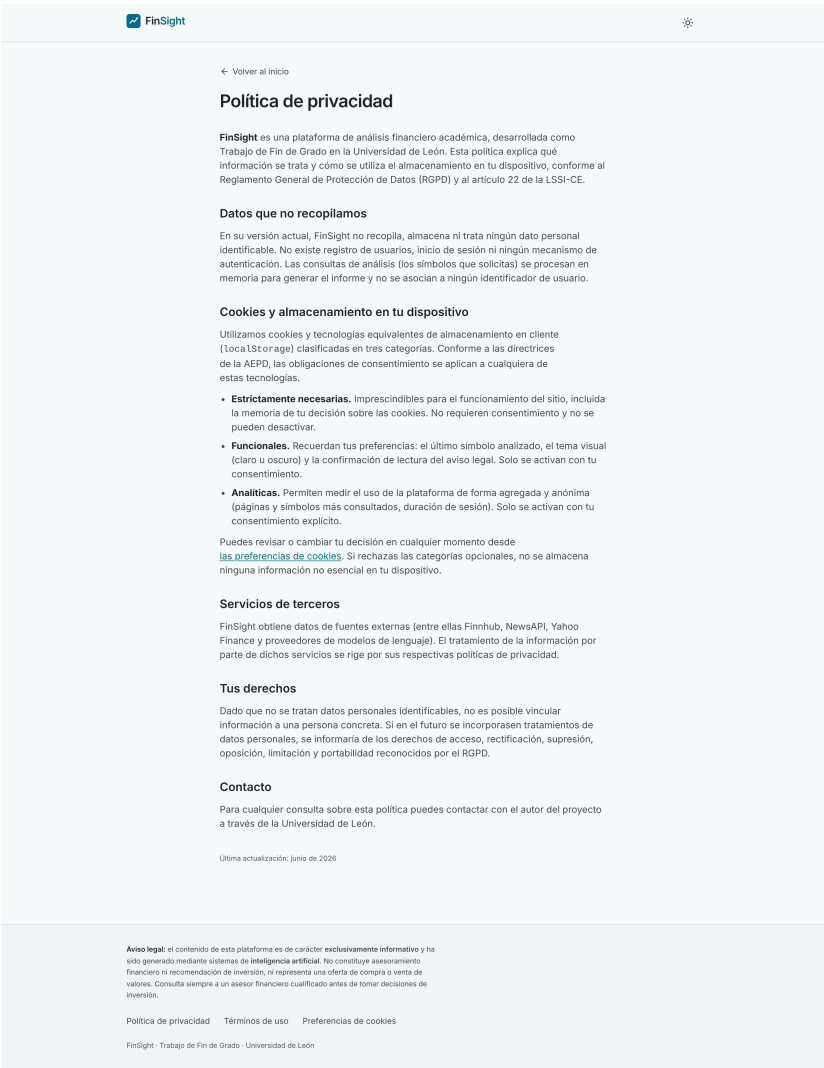


Figura 3.15: Página de política de privacidad, enlazada de forma permanente desde el pie.



Figura 3.16: Página de términos de uso.

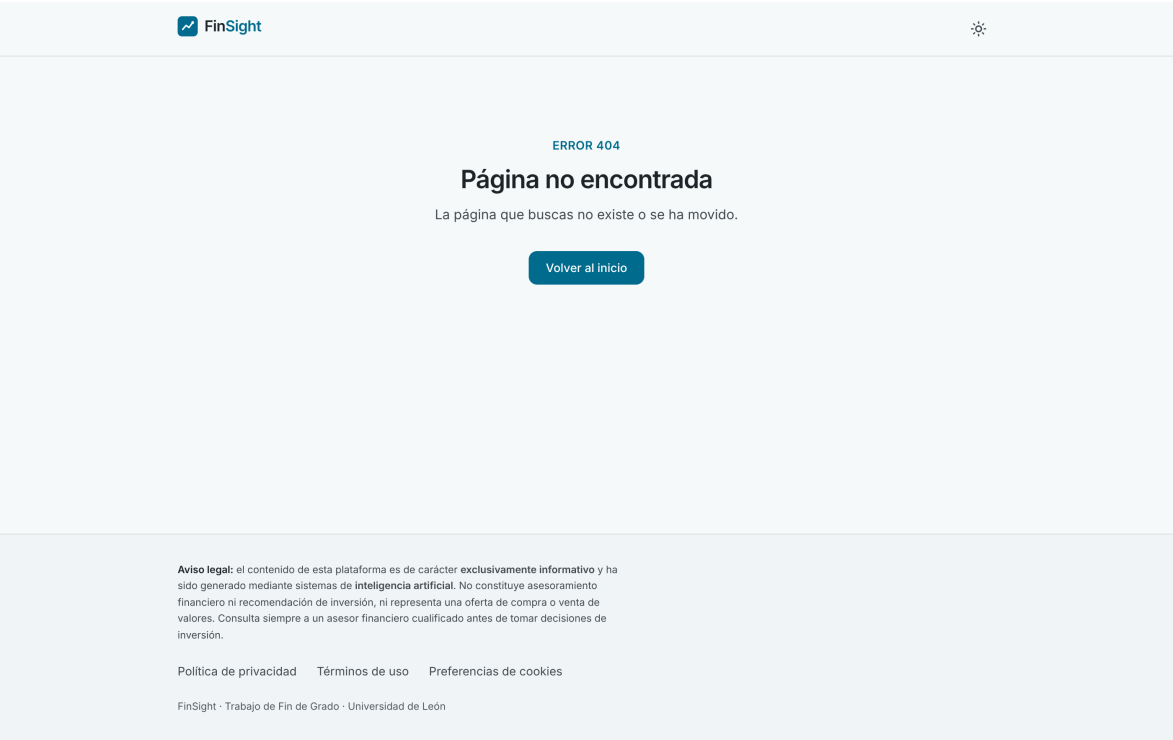


Figura 3.17: Página de error para rutas inexistentes, con vuelta al inicio.

Capítulo 4

Evaluación

Una solución se considera válida si resuelve el problema planteado y cumple los objetivos fijados en la introducción. Este capítulo describe cómo se ha comprobado esa validez y qué resultados se han obtenido. La evaluación combina la validación funcional mediante casos de prueba trazados a los objetivos y a los requisitos del SRS, la batería de pruebas automatizadas y las métricas del modelo descritas en el capítulo anterior, una prueba informal con un usuario sin formación financiera y la comprobación del rendimiento, la accesibilidad y la seguridad.

4.1. Proceso de evaluación

4.1.1. Forma de evaluación

La evaluación se apoya en varias fuentes de evidencia complementarias. La primera es la batería de pruebas automatizadas descrita en la sección de pruebas del capítulo anterior: 290 casos entre backend, frontend e investigación que se ejecutan en cada integración y actúan como red de regresión. La segunda es la evaluación cuantitativa del único módulo medible por error, la red GRU, cuyas métricas de RMSE, MAE, acierto direccional y coeficiente U de Theil se recogen también en el capítulo anterior. Los otros tres módulos no admiten esa medida, así que su calidad se valora de forma cualitativa, atendiendo a la coherencia del texto generado y a su fidelidad a los datos, y se acota con el aviso legal.

Sobre esa base, este capítulo añade una validación de aceptación orientada a comprobar que el producto satisface los objetivos y los requisitos. Para ello se definió un conjunto de casos de prueba de alto nivel, cada uno asociado a uno o varios objetivos de la introducción y a los requisitos funcionales o no funcionales correspondientes del SRS. A esto se suma una prueba informal con un usuario sin conocimientos financieros, cuyo fin no es un estudio de usabilidad formal, sino confirmar que una persona ajena al proyecto es capaz de buscar un valor, generar el informe e interpretar sus conclusiones. Por último, se comprueban el rendimiento, la accesibilidad y la seguridad frente a los umbrales fijados en los requisitos no funcionales.

4.1.2. Casos de prueba

La Tabla 4.1 recoge los casos de prueba de aceptación, con su trazabilidad a los requisitos y su resultado. Los casos funcionales se sustentan en la batería automatizada y se dan por superados; el rendimiento y la accesibilidad, de naturaleza cuantitativa, se analizan en la sección siguiente.

4.2. Análisis de resultados

En conjunto, los objetivos funcionales del proyecto se cumplen y quedan respaldados por la batería de pruebas y por los casos de aceptación. La plataforma integra los cuatro módulos de análisis, genera un informe consolidado en español, se presenta como una aplicación web accesible y aplica las salvaguardas de seguridad y de cumplimiento legal previstas. El análisis de sentimiento entrega su salida tripartita explicada con las noticias influyentes, el fundamental y el técnico se integran y se traducen a lenguaje natural, y el módulo de síntesis reúne las cuatro perspectivas en una única conclusión.

El objetivo de predicción de tendencia se cumple en lo funcional, pero conviene ser honesto con su alcance. La red GRU produce una señal de tendencia con sus métricas, si bien su ventaja sobre el azar es estrecha, con un acierto direccional en torno a 0,56 y un coeficiente U de Theil cercano a 0,98. Es una limitación asumida y descrita en el análisis de riesgos, que se mitiga presentando la predicción como una señal más dentro de un informe holístico, nunca como una recomendación.

ID	Descripción	Requisitos	Resultado
CP-01	Buscar un valor y solicitar el análisis desde la interfaz	RF-02, RF-03	Superado
CP-02	Generar el informe consolidado con los cuatro módulos y la conclusión global	RF-35, RF-36	Superado
CP-03	Ofrecer soporte parcial cuando el valor no tiene modelo de tendencia	RF-27	Superado
CP-04	Rechazar un <i>ticker</i> inválido y normalizar uno válido	RF-02, RNF-09	Superado
CP-05	Mostrar el aviso legal permanente en la interfaz y en el informe	RF-06, RF-37, RNF-32	Superado
CP-06	Gestionar el consentimiento de cookies granular por categorías	RNF-36, RNF-39	Superado
CP-07	Devolver el sentimiento con salida tripartita, explicación y noticias influyentes	RF-13 a RF-16	Superado (cualitativo)
CP-08	Predecir la tendencia con la red GRU y adjuntar sus metadatos	RF-21, RF-22, RF-23	Superado
CP-09	Limitar las peticiones por IP en los <i>end-points</i> de análisis	RNF-08	Superado
CP-10	Reentrenar los modelos de forma diaria y automática	RF-24, RNF-29	Superado
CP-11	Generar el informe por debajo de 60 segundos con el servicio activo	RNF-01	Superado en local / Ligeramente superior en producción
CP-12	Documentar el arranque en frío del backend	RNF-30	Superado
CP-13	Certificar la conformidad WCAG 2.1 nivel AA	RNF-14	Superado
CP-14	Comprender el informe un usuario sin formación financiera	Objetivo general	Superado (cualitativo)

Tabla 4.1: Casos de prueba de aceptación, con su trazabilidad a los requisitos del SRS y su resultado.

En cuanto al rendimiento, el arranque en frío del backend en Render, tras un periodo de inactividad, se midió en 2 minutos y 51 segundos; el frontend permanece siempre desplegado y no sufre esa espera (RNF-30). El tiempo de generación de un informe completo, una vez el servicio está activo, se contrasta con el objetivo de 60 segundos del RNF-01. Los valores obtenidos son de 37,20 segundos en el entorno local y de 1 minuto y 5 segundos en producción, cifras que se incorporarán para confirmar el cumplimiento del umbral.

La accesibilidad se valida en dos niveles. De forma automática, las pruebas del frontend incluyen comprobaciones con *vitest-axe* y las reglas *jsx-a11y* de ESLint, que no detectan violaciones en los componentes analizados. Además, se auditó la página del informe con Lighthouse, que otorgó una puntuación de accesibilidad de 100 sobre 100: superó sin incidencias las comprobaciones automáticas de contraste de color, roles y atributos ARIA, nombres de botones y enlaces, orden de los encabezados e idioma del documento. Conviene matizar que estas herramientas cubren solo una parte de los criterios de las WCAG 2.1 AA (RNF-14), de modo que una certificación plena requeriría además la revisión manual de los aspectos no automatizables, como el lector de pantalla.

La seguridad se comprueba frente a los requisitos de su categoría. El sistema fuerza HTTPS en producción, mantiene las claves fuera del código en variables de entorno, limita las peticiones por IP (verificado en CP-09), restringe el CORS al dominio del frontend, escapa la salida del LLM para prevenir XSS y analiza el código con CodeQL en la integración continua, cubriendo así los requisitos RNF-06 a RNF-13.

Por último, la prueba informal con un usuario sin formación financiera confirmó que una persona ajena al proyecto puede completar el flujo, buscar un valor, generar el informe e interpretar su conclusión, sin ayuda. No sustituye a un estudio de usabilidad formal, que queda como trabajo futuro, pero respalda el objetivo de acercar el análisis a quien no es experto.

Conclusión

Este trabajo se propuso acercar el análisis financiero de acciones a personas sin formación especializada, reuniendo cuatro perspectivas de análisis en un único informe en español y apoyándose en la inteligencia artificial para interpretarlas. Con la perspectiva del trabajo terminado, considero que los objetivos se han cumplido: la plataforma funciona, integra los cuatro módulos, genera el informe consolidado y atiende a la accesibilidad, la seguridad y el cumplimiento legal. Queda, eso sí, una limitación que no conviene esconder, y es que la predicción de tendencia aporta un margen estrecho sobre el azar. El resto del capítulo recoge las aportaciones del trabajo, las líneas de continuación, los problemas encontrados y una valoración personal.

Aportaciones realizadas

La principal aportación es la propia plataforma FinSight: una herramienta web gratuita que, a partir del nombre o el símbolo de una acción, entrega un informe de análisis en español comprensible sin conocimientos financieros. Frente a las alternativas actuales, que o bien cuestan dinero o bien se limitan a mostrar datos, FinSight interpreta esos datos y los explica.

Más allá del producto, el trabajo deja varias aportaciones concretas:

- La integración de cuatro análisis complementarios (sentimiento, tendencia, fundamental y técnico) en un único informe sintetizado por un modelo de lenguaje.
- Un módulo de sentimiento sobre una arquitectura GraphRAG apoyada en un grafo de conocimiento en Neo4j, que ancla las respuestas del modelo en noticias, entidades y eventos.

- Un estudio comparativo de modelos de series temporales con una metodología de evaluación causal, cuidando la ausencia de fuga de información, que condujo a una red GRU compacta y desplegable sin GPU.
- Una arquitectura mantenible, con un servicio LLM centralizado, patrones de diseño para orquestar los módulos, integración continua, pruebas y decisiones de diseño documentadas.
- El tratamiento de la accesibilidad, la seguridad y el cumplimiento normativo como requisitos de primer nivel, y no como añadidos de última hora.

Trabajos futuros

El trabajo abre varias líneas de continuación:

- Buscar un modelo de predicción que ofrezca una ventaja estratégica real. Centrarse en un número reducido de valores y enriquecerlos con más información podría mejorar el rendimiento, a costa de sacrificar la amplia cobertura que persigue el proyecto.
- Incorporar datos intradía y en tiempo real, y añadir una sección que los muestre, de modo que la aplicación no solo analice sino que también sirva como portal de consulta, en la línea de plataformas como Investing.com.
- Ampliar la cobertura más allá del Nasdaq, a otros mercados e índices.
- Permitir cuentas de usuario con cartera propia y su seguimiento, informes automáticos de las acciones favoritas e incluso un sistema de gestión de riesgo que evalúe la diversificación y el nivel de riesgo de la cartera.
- Integrarse o colaborar con un bróker externo para operar activos directamente desde la plataforma.
- Construir una sección de noticias propia mediante *scraping* de grandes medios, lo que aportaría un portal de noticias propio y reduciría la dependencia de las APIs externas.
- Generar los informes en otros idiomas para internacionalizar la herramienta.

- Realizar un estudio de usabilidad formal con usuarios, más allá de la prueba informal de la evaluación.
- Explorar una vía de monetización mediante una suscripción de pago que amplíe la versión gratuita, ofreciendo más análisis por informe, servicios adicionales o el acceso a modelos de lenguaje y de predicción más potentes, mientras el plan gratuito conserva una cobertura básica.

Problemas encontrados

El problema principal fue la escasa ventaja del modelo de predicción. Concentrarse en un único valor y alimentarlo con más información podría dar mejores resultados, pero la voluntad de ofrecer una cobertura amplia hace que el rendimiento quede apenas por encima del azar. En el fondo, refleja la dificultad de predecir un entorno tan volátil y aleatorio como la bolsa.

Durante la investigación apareció además una fuga de información. La señal de alarma fue un acierto casi perfecto sobre el conjunto de prueba: una precisión tan alta solo podía significar que el modelo estaba viendo, de algún modo, información futura. Localizar y corregir esa fuga fue determinante para que la evaluación posterior fuera honesta.

Los límites del *tier* gratuito supusieron otro inconveniente, más operativo que de diseño, al retrasar y ralentizar las ejecuciones. En cambio, las cuotas de las APIs de noticias no dieron problemas en el uso actual, aunque quedaría por ver cómo se comportarían si el producto escalara y tuviera que atender miles de peticiones en poco tiempo. Algo parecido ocurre con el coste de OpenAI: fue inferior a 10 euros en todo el proyecto, insignificante ahora, pero un factor a vigilar si el uso creciera. Por último, se echó en falta una mayor disponibilidad de conjuntos de noticias financieras etiquetadas: aunque el grafo reúne miles de noticias, podría ser bastante más grande.

Esa dependencia del plan gratuito condiciona también la cobertura de la versión desplegada. Como el entrenamiento bajo demanda no resulta viable en esa infraestructura, en producción solo están disponibles los modelos de *deep learning* previamente entrenados y subidos al repositorio, de modo que la predicción de tendencia se ofrece

hoy para en torno a diez o doce valores. La solución sería sencilla: subir el resto de modelos ya entrenados para la versión en producción.

Opiniones personales

En el plano personal, este trabajo me ha enseñado bastante. He aprendido a fondo la tecnología GraphRAG y, durante la fase de investigación, otras tecnologías afines que hoy son punteras y forman parte del día a día de muchas empresas. También me he enfrentado por primera vez a los modelos de series temporales, y he comprobado de primera mano lo difícil que es predecir el mercado y hasta qué punto la calidad y el tratamiento de los datos condicionan el resultado.

Lo más difícil no fue programar, sino diseñar el proyecto: decidir qué análisis integrar y cómo hacer que se complementaran entre sí. Esa fase de decisiones, más conceptual que técnica, fue la que más me hizo pensar y, en retrospectiva, la que más ha marcado el resultado final.

Uso de herramientas de inteligencia artificial generativa

Durante la elaboración de este Trabajo Fin de Grado se han utilizado herramientas de inteligencia artificial como apoyo en distintas fases del desarrollo. En concreto, se ha empleado Claude como asistente en la redacción de algunos apartados de la memoria, especialmente tablas y diagramas, con el objetivo de mejorar la claridad y la calidad de la expresión escrita y el aspecto didáctico y visual del trabajo. No obstante, todo el contenido finalmente incluido en este documento ha sido revisado, corregido y validado por el autor, quien asume la plena responsabilidad sobre su contenido.

Además, se ha utilizado Claude Code como herramienta de desarrollo software durante la implementación de diferentes programas en el trabajo, con el fin de agilizar el proceso de creación de la aplicación. En todos los casos, el código generado o sugerido por la herramienta ha sido revisado, comprendido, adaptado cuando ha sido necesario y verificado mediante las correspondientes pruebas antes de su incorporación al trabajo.

Se destaca el uso de Claude frente a otras herramientas de IA por su gran capacidad de personalización y de automatización. El uso de *skills* y la conexión con distintas plataformas externas hacen de Claude una herramienta muy completa.

Agradecimientos

Agradezco a mi tutor, Enrique, por haber apoyado la idea desde el principio y coordinar el trabajo interdisciplinar entre los dos ámbitos de este trabajo.

Agradezco a Lidia, por haber sido un apoyo indispensable a lo largo del curso, por entender mi situación y por ayudarme en todo lo que he necesitado.

Agradezco a mi familia y amigos por ser mi soporte vital, por apoyarme en mis proyectos personales y también por corregirme cuando la situación lo requiere.

Agradezco al deporte, por haberme enseñado los valores que hoy me permiten compaginar mi vida profesional con los estudios, y que se extienden a todos los ámbitos. Gracias al deporte por ser de los mejores maestros que he tenido.

Lista de referencias

- [1] Agencia Estatal de Administración Tributaria: Resolución de 24 de octubre de 2007, de la Agencia Estatal de Administración Tributaria, sobre procedimiento para la homologación de software de digitalización certificada de facturas. Boletín Oficial del Estado (BOE), núm. 262 (2007), disponible en: [https://www.boe.es/eli/es/res/2007/10/24/\(1\)](https://www.boe.es/eli/es/res/2007/10/24/(1))
- [2] Al-Khasawneh, M.A., Raza, A., Khan, S.U.R., Khan, Z.: Stock market trend prediction using deep learning approach. *Computational Economics* **66**, 453–484 (2025)
- [3] Anderson, D.J.: *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, Silver Spring, MD, USA (2010)
- [4] Apache Software Foundation: Apache License, Version 2.0. Apache Software Foundation (2004), disponible en: <https://www.apache.org/licenses/LICENSE-2.0>
- [5] Araci, D.: FinBERT: Financial sentiment analysis with pre-trained language models (2019)
- [6] Bollerslev, T.: Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics* **31**(3), 307–327 (1986). [https://doi.org/10.1016/0304-4076\(86\)90063-1](https://doi.org/10.1016/0304-4076(86)90063-1)
- [7] Box, G.E.P., Jenkins, G.M.: *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco (1976)
- [8] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using RNN encoder–decoder for

- statistical machine translation. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 1724–1734 (2014). <https://doi.org/10.3115/v1/D14-1179>
- [9] Comisión Europea: Reglamento Delegado (UE) 2017/565, por el que se completa la Directiva 2014/65/UE en lo relativo a los requisitos organizativos y las condiciones de funcionamiento de las empresas de servicios de inversión. Diario Oficial de la Unión Europea, L 87 (2017), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32017R0565>
- [10] Comisión Europea: Decisión de Ejecución (UE) 2023/1795, relativa al nivel adecuado de protección de los datos personales en el marco de privacidad de datos UE-EE. UU. Diario Oficial de la Unión Europea, L 231 (2023), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32023D1795>
- [11] Dolphin, R., Dursun, J., Chow, J., Blankenship, J., Adams, K., Pike, Q.: Extracting structured insights from financial news: An augmented LLM driven approach (2024), <https://arxiv.org/abs/2407.15788>
- [12] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J.: From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024), <https://arxiv.org/abs/2404.16130>
- [13] Engle, R.F.: Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica* **50**(4), 987–1007 (1982). <https://doi.org/10.2307/1912773>
- [14] Fan, Y., Zhang, Y., Guo, B., Luo, X., Peng, Q., Jin, Z.: A hybrid sparrow search algorithm of the hyperparameter optimization in deep learning. *Mathematics* **10**(16) (2022)
- [15] Fierro, A.A.: Predicción de Series Temporales con Redes Neuronales. Trabajo final de especialización, Universidad Nacional de La Plata (2020)
- [16] Gamma, E., Helm, R., Johnson, R., Vlissides, J.: Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, Boston, MA, USA (1994)

-
- [17] George, L., Sumathy, P., Vadivel, A.: GRAGLLM: GraphRAG-enhanced large language models for explainable multi-aspect sentiment analysis using prompt-based learning. *Neural Computing and Applications* **38**(8), 286 (2026). <https://doi.org/10.1007/s00521-026-12045-8>
- [18] Groq, Inc.: Groq Terms of Use. Groq (2024), disponible en: <https://groq.com/terms-of-use/>
- [19] Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997). <https://doi.org/10.1162/neco.1997.9.8.1735>
- [20] Jefatura del Estado: Ley 34/2002, de 11 de julio, de servicios de la sociedad de la información y de comercio electrónico. Boletín Oficial del Estado (BOE), núm. 166 (2002), disponible en: <https://www.boe.es/eli/es/l/2002/07/11/34/con>
- [21] Jefatura del Estado: Ley 25/2007, de 18 de octubre, de conservación de datos relativos a las comunicaciones electrónicas y a las redes públicas de comunicaciones. Boletín Oficial del Estado (BOE), núm. 251 (2007), disponible en: <https://www.boe.es/eli/es/l/2007/10/18/25/con>
- [22] Jefatura del Estado: Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. Boletín Oficial del Estado (BOE), núm. 294 (2018), disponible en: <https://www.boe.es/eli/es/lo/2018/12/05/3/con>
- [23] Jefatura del Estado: Real Decreto-ley 19/2018, de 23 de noviembre, de servicios de pago y otras medidas urgentes en materia financiera. Boletín Oficial del Estado (BOE), núm. 284 (2018), disponible en: <https://www.boe.es/eli/es/rdl/2018/11/23/19/con>
- [24] Jefatura del Estado: Ley 11/2021, de 9 de julio, de medidas de prevención y lucha contra el fraude fiscal. Boletín Oficial del Estado (BOE), núm. 163 (2021), disponible en: <https://www.boe.es/eli/es/l/2021/07/09/11/con>
- [25] Jefatura del Estado: Ley Orgánica 7/2021, de 26 de mayo, de protección de datos personales tratados para fines de prevención, detección, investigación y enjuiciamiento de infracciones penales y de ejecución de sanciones penales. Boletín Oficial
-

- del Estado (BOE), núm. 126 (2021), disponible en: <https://www.boe.es/eli/es/1o/2021/05/26/7/con>
- [26] Jefatura del Estado: Ley 18/2022, de 28 de septiembre, de creación y crecimiento de empresas. Boletín Oficial del Estado (BOE), núm. 234 (2022), disponible en: <https://www.boe.es/eli/es/1/2022/09/28/18/con>
- [27] Kelly, B., Xiu, D.: Financial machine learning. SSRN Electronic Journal (2023), SSRN working paper 4501707
- [28] Kim, Y., Yoo, S., Park, S.: A rule-based stock trading recommendation system using sentiment analysis and technical indicators. *Electronics* **14**(4) (2025). <https://doi.org/10.3390/electronics14040773>, <https://www.mdpi.com/2079-9292/14/4/773>
- [29] Lefort, B., Benhamou, E., Guez, B., Ohana, J.J., Setrouk, E., Etienne, A.: Fin-MarBa: A market-informed dataset for financial sentiment classification (2025), iJCAI 2025 FinLLM Workshop
- [30] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 33, pp. 9459–9474 (2020)
- [31] Li, L.: Deep learning-based financial time series forecasting via sliding window and variational mode decomposition. *arXiv preprint arXiv:2508.12565* (2025)
- [32] Liu, F., Qin, P., You, J., Fu, Y.: Sparrow search algorithm-optimized long short-term memory model for stock trend prediction. *Computational Intelligence and Neuroscience* **2022**, 3680419 (2022)
- [33] Lu, P., Li, H.: A stock price prediction model based on VMD-SSA-LSTM. In: *2025 Asia-Europe Conference on Cybersecurity, Internet of Things and Soft Computing (CITSC)*. IEEE (2025). <https://doi.org/10.1109/CITSC64390.2025.00067>
- [34] Malo, P., Sinha, A., Korhonen, P., Wallenius, J., Takala, P.: Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology* **65**(4), 782–796 (2014)

-
- [35] Meta Platforms, Inc.: Llama 3.1 Community License Agreement. Meta AI (2024), disponible en: https://www.llama.com/llama3_1/license/
- [36] Ministerio de Asuntos Económicos y Transformación Digital: Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad. Boletín Oficial del Estado (BOE), núm. 106 (2022), disponible en: <https://www.boe.es/eli/es/rd/2022/05/03/311/con>
- [37] Ministerio de Cultura: Real Decreto Legislativo 1/1996, de 12 de abril, por el que se aprueba el texto refundido de la Ley de Propiedad Intelectual. Boletín Oficial del Estado (BOE), núm. 97 (1996), disponible en: <https://www.boe.es/eli/es/rdlg/1996/04/12/1/con>
- [38] Ministerio de Economía y Hacienda: Orden EHA/962/2007, de 10 de abril, por la que se desarrollan determinadas disposiciones sobre facturación telemática y conservación electrónica de facturas. Boletín Oficial del Estado (BOE), núm. 90 (2007), disponible en: <https://www.boe.es/eli/es/o/2007/04/10/eha962>
- [39] Ministerio de Hacienda: Orden HAC/1177/2024 (Desarrollo técnico Verifactu). Boletín Oficial del Estado (BOE), núm. 254 (2024), disponible en: <https://www.boe.es/eli/es/o/2024/10/17/hac1177>
- [40] Ministerio de Hacienda y Función Pública: Real Decreto 1007/2023 (Reglamento Verifactu). Boletín Oficial del Estado (BOE), núm. 291 (2023), disponible en: <https://www.boe.es/eli/es/rd/2023/12/05/1007>
- [41] Ministerio de la Presidencia: Real Decreto Legislativo 1/2007, de 16 de noviembre, por el que se aprueba el texto refundido de la Ley General para la Defensa de los Consumidores y Usuarios. Boletín Oficial del Estado (BOE), núm. 287 (2007), disponible en: <https://www.boe.es/eli/es/rdlg/2007/11/16/1/con>
- [42] Ministerio de Política Territorial y Función Pública: Real Decreto 1112/2018, de 7 de septiembre, sobre accesibilidad de los sitios web y aplicaciones para dispositivos móviles del sector público. Boletín Oficial del Estado (BOE), núm. 227 (2018), disponible en: <https://www.boe.es/eli/es/rd/2018/09/07/1112/con>
-

-
- [43] Ministerio de Sanidad, Servicios Sociales e Igualdad: Real Decreto Legislativo 1/2013, de 29 de noviembre, por el que se aprueba el Texto Refundido de la Ley General de derechos de las personas con discapacidad y de su inclusión social. Boletín Oficial del Estado (BOE), núm. 289 (2013), disponible en: <https://www.boe.es/eli/es/rdlg/2013/11/29/1/con>
- [44] Neo4j, Inc.: neo4j-graphrag: Neo4j GraphRAG for python. Documentación oficial (2024), <https://neo4j.com/docs/neo4j-graphrag-python/>
- [45] Open Source Initiative: The 3-Clause BSD License. Open Source Initiative (2024), disponible en: <https://opensource.org/license/bsd-3-clause>
- [46] Open Source Initiative: The MIT License. Open Source Initiative (2024), disponible en: <https://opensource.org/license/mit>
- [47] OpenAI: Terms of Use. OpenAI (2024), disponible en: <https://openai.com/policies/terms-of-use/>
- [48] OpenAI: Data Processing Addendum. OpenAI (2025), disponible en: <https://openai.com/policies/data-processing-addendum/>
- [49] OpenAI: Usage Policies. OpenAI (2025), disponible en: <https://openai.com/policies/usage-policies/>
- [50] Parlamento Europeo y Consejo de la Unión Europea: Directiva 2014/65/UE, relativa a los mercados de instrumentos financieros. Diario Oficial de la Unión Europea, L 173 (2014), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32014L0065>
- [51] Parlamento Europeo y Consejo de la Unión Europea: Directiva (UE) 2015/2366, sobre servicios de pago en el mercado interior. Diario Oficial de la Unión Europea, L 337 (2015), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32015L2366>
- [52] Parlamento Europeo y Consejo de la Unión Europea: Directiva (UE) 2016/2102, sobre la accesibilidad de los sitios web y aplicaciones para dispositivos móviles de los organismos del sector público. Diario Oficial de la Unión Europea,

- L 327 (2016), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016L2102>
- [53] Parlamento Europeo y Consejo de la Unión Europea: Reglamento (UE) 2016/679, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos. Diario Oficial de la Unión Europea, L 119 (2016), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>
- [54] Parlamento Europeo y Consejo de la Unión Europea: Directiva (UE) 2022/2555, relativa a las medidas destinadas a garantizar un elevado nivel común de ciberseguridad en toda la Unión. Diario Oficial de la Unión Europea, L 333 (2022), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32022L2555>
- [55] Parlamento Europeo y Consejo de la Unión Europea: Reglamento (UE) 2024/1689, por el que se establecen normas armonizadas en materia de inteligencia artificial. Diario Oficial de la Unión Europea, L 2024/1689 (2024), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32024R1689>
- [56] Parlamento Europeo y Consejo de la Unión Europea: Reglamento (UE) 2024/2847, relativo a los requisitos horizontales de ciberseguridad para los productos con elementos digitales. Diario Oficial de la Unión Europea, L 2024/2847 (2024), disponible en: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32024R2847>
- [57] Pilla, P., Mekonen, R.: Forecasting S&P 500 using LSTM models. arXiv preprint arXiv:2501.17366 (2025)
- [58] Purwantara, I.M.A., Kusriani, Setyanto, A., Utami, E.: Forecasting the highest and lowest prices in financial markets using a VMD-LSTM hybrid model. COGITO Smart Journal **11**(2) (2025)
- [59] svaningelgem: Nasdaq daily stock prices. Kaggle dataset, <https://www.kaggle.com/datasets/svaningelgem/nasdaq-daily-stock-prices>

-
- [60] Tang, Y., Yang, Y., Huang, A.H., Tam, A., Tang, J.Z.: FinEntity: Entity-level sentiment classification for financial texts. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 15465–15471. Association for Computational Linguistics, Singapore (2023)
- [61] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 30, pp. 5998–6008 (2017)
- [62] World Wide Web Consortium: Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation (2018), disponible en: <https://www.w3.org/TR/WCAG21/>
- [63] World Wide Web Consortium: Accessible Rich Internet Applications (WAI-ARIA) 1.2. W3C Recommendation (2023), disponible en: <https://www.w3.org/TR/wai-aria-1.2/>
- [64] Xu, H.: Stock price prediction using decision tree classifier and LSTM network. In: Proceedings of the 2023 International Conference on Machine Learning and Automation (2023). <https://doi.org/10.54254/2755-2721/37/20230512>
- [65] Zaheer, S., Anjum, N., Hussain, S., Algarni, A.D., Iqbal, J., Bourouis, S., Ullah, S.S.: A multi parameter forecasting for stock time series data using LSTM and deep learning model. *Mathematics* **11**(3) (2023)
- [66] Zhao, Y., Chen, J., Shimada, H., Sasaoka, T.: Non-ferrous metal price point and interval prediction based on variational mode decomposition and optimized LSTM network. *Mathematics* (2023), verificar volumen/número en la fuente

Anexo A

Control de versiones

Todo el desarrollo se ha gestionado con Git como sistema de control de versiones y GitHub como repositorio remoto. El trabajo se reparte en dos repositorios: el del proyecto previo y el de FinSight.

El primero, accesible en <https://github.com/jminac00/financial-sentiment-graphrag>, corresponde a una aplicación de línea de comandos que obtiene noticias financieras, las procesa con GraphRAG y genera una evaluación cuantitativa del sentimiento para apoyar decisiones de inversión. Se desarrolló durante el primer cuatrimestre en el marco de otra asignatura, y sirvió como prueba de concepto y como base de conocimiento del módulo de sentimiento de FinSight; su historial documenta la investigación inicial sobre la arquitectura GraphRAG. La Figura A.1 muestra su repositorio.

El segundo repositorio, accesible en <https://github.com/jminac00/FinSight>, contiene la aplicación completa. Reúne 184 *commits* realizados entre el 4 de junio y el 4 de julio de 2026, integrados en la rama principal a través de 41 *pull requests*. La Figura A.2 recoge una vista de su historial.

Convención de commits

Los mensajes de *commit* siguen la convención Conventional Commits, con un tipo y un ámbito por cambio, de modo que el historial es legible y trazable. La Tabla A.1 recoge el reparto por tipo de los *commits* de trabajo, al margen de los de fusión.

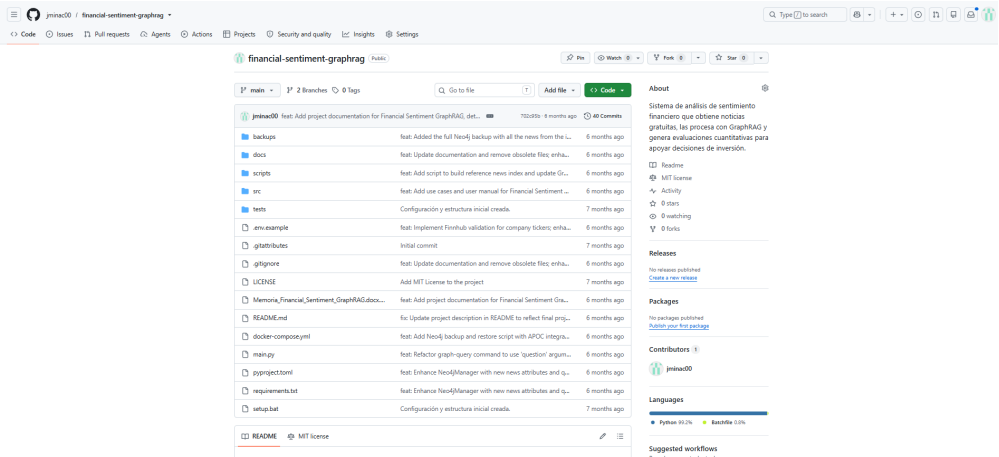


Figura A.1: Repositorio del proyecto previo de análisis de sentimiento (CLI GraphRAG), origen del módulo de sentimiento de FinSight.

Tipo	Número	Uso
feat	53	Nueva funcionalidad
test	32	Añadir o modificar pruebas
docs	16	Solo documentación
chore	14	Mantenimiento y dependencias
fix	13	Corrección de errores
refactor	6	Refactorización sin cambio de comportamiento
ci	6	Cambios en la integración continua

Tabla A.1: Reparto por tipo de los commits de trabajo del repositorio de FinSight.

jminac00 / FinSight Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings Q Type ↵ to search +	
Commits	
main All users All time	
Commits on Jul 4, 2026	
feat(deep-learning): train missing models on demand in local (#90) jminac00 authored 5 days ago · ✓ 6 / 6 Verified 87fc928 <>	
chore(frontend): ignore TypeScript incremental build cache (#88) jminac00 authored 5 days ago · ✓ 6 / 6 Verified 16656c9 <>	
Merge pull request #86 from jminac00/fix/relax-ticker-validation-international jminac00 authored 5 days ago · ✓ 6 / 6 Verified 56d7d8a <>	
fix(api): accept international ticker symbols in validation jminac00 committed 5 days ago · ✓ 8 / 8 8223465 <>	
test(api): expect ticker validation to accept international symbols jminac00 committed 5 days ago 97b8a40 <>	
Commits on Jul 3, 2026	
Report warns on any missing module and shows the company name (#84) jminac00 authored 5 days ago · ✓ 6 / 6 Verified 23f82cf <>	
fix(technical): seed universe snapshots to survive fresh deploys (#82) jminac00 authored 5 days ago · ✓ 6 / 6 Verified fae8ae9 <>	
chore(fundamental): refresh universe seed snapshots (#81) jminac00 authored 5 days ago · ✗ 2 / 6 Verified 2b867db <>	
Merge pull request #79 from jminac00/feature/report-metric-glossary-and-collapsible-details jminac00 authored 5 days ago · ✓ 6 / 6 Verified 1e9c973 <>	
test(frontend): cover the collapsed default state and indicator curation jminac00 committed 5 days ago · ✓ 8 / 8 f8c97dd <>	
feat(frontend): collapse fundamental and technical details behind a disclosure jminac00 committed 5 days ago 5844787 <>	
feat(frontend): add an accessible details disclosure primitive jminac00 committed 5 days ago deb2abb <>	
feat(frontend): explain every fundamental ratio and curated technical indicator jminac00 committed 5 days ago f2eb6ed <>	
test(frontend): add failing tests for indicator curation, disclosure and glossary coverage jminac00 committed 5 days ago 1e7f03f <>	
Merge pull request #77 from jminac00/fix/render-rewrite-backend-hostname jminac00 authored 5 days ago · ✓ 6 / 6 Verified 7e93652 <>	
fix(infra): correct backend hostname in frontend rewrite rule jminac00 committed 5 days ago · ✓ 8 / 8 98c0caa <>	
Merge pull request #75 from jminac00/feature/search-autocomplete jminac00 authored 5 days ago · ✓ 6 / 6 Verified ac86b86 <>	
Merge pull request #73 from jminac00/feature/metric-glossary-tooltips jminac00 authored 5 days ago · ✗ 2 / 6 Verified 6d3feb3 <>	
Merge pull request #71 from jminac00/feature/report-visual-redesign jminac00 authored 5 days ago · ✗ 2 / 6 Verified 89b0763 <>	
Merge pull request #69 from jminac00/fix/report-contract-and-error-states jminac00 authored 5 days ago · ✗ 3 / 6 Verified 735a895 <>	
Merge pull request #67 from jminac00/feature/symbol-search-endpoint jminac00 authored 5 days ago · ✗ 2 / 6 Verified d7669d6 <>	
fix(frontend): render nested backend metric bags without crashing jminac00 committed 5 days ago · ✓ 8 / 8 19c5e85 <>	
test(frontend): make fixtures faithful to nested backend detail bags jminac00 committed 5 days ago ceb86e1 <>	
test(frontend): cover the search combobox jminac00 committed 5 days ago · ✓ 8 / 8 f7b517c <>	
feat(frontend): turn the search box into a company/ticker combobox jminac00 committed 5 days ago 946152d <>	
feat(frontend): add symbol search client and a debounced search hook jminac00 committed 5 days ago f7e80d6 <>	
test(frontend): cover the metric info disclosure jminac00 committed last week · ✓ 8 / 8 b8f4a69 <>	
feat(frontend): explain report metrics with accessible info disclosures jminac00 committed last week 9a6247f <>	
test(frontend): cover the report overview links and unavailable modules jminac00 committed last week · ✓ 8 / 8 05189a4 <>	
feat(frontend): elevate the report with an overview and inline visualizations jminac00 committed last week 85e2342 <>	
test(frontend): cover the report data visualizations jminac00 committed last week 3a734f6 <>	
feat(frontend): add accessible SVG data visualizations for the report jminac00 committed last week 6b7c40e <>	
feat(frontend): resolve the API base URL from env for cross-origin deploys jminac00 committed last week b60cb2f <>	
fix(frontend): stop implying coverage is limited to the Nasdaq jminac00 committed last week · ✓ 8 / 8 7089e86 <>	
fix(frontend): align technical result with backend signal and block scores jminac00 committed last week 639a912 <>	

Figura A.2: Historial de commits del repositorio de FinSight en GitHub.

A modo de ejemplo, algunos mensajes reales del repositorio son:

- `feat(deep-learning): add GRU training pipeline`
- `fix(api): accept international ticker symbols in validation`
- `test(frontend): cover the search combobox`
- `ci(frontend): run type check and tests`

Ramas e integración

El flujo de trabajo parte de la rama `main`. Cada unidad de trabajo se desarrolla en una rama independiente con nombre descriptivo, según su propósito (`feature/`, `fix/` o `test/`), por ejemplo `feature/symbol-search-endpoint` o `fix/relax-ticker-validation-interna`. La rama se integra mediante una *pull request* que referencia el *issue* correspondiente y que no se fusiona hasta que la integración continua está en verde.

La integración continua se implementa con GitHub Actions en tres flujos, que se ejecutan en cada *push* y cada *pull request* sobre `main`. El flujo de *backend* comprueba el estilo y el formato con Ruff y ejecuta la batería de *pytest* con medida de cobertura. El flujo de *frontend* pasa el análisis estático con ESLint, la comprobación de tipos de TypeScript y las pruebas con Vitest, y termina con la compilación de producción de Vite. El tercer flujo ejecuta el análisis de seguridad con CodeQL. Las Figuras A.3 y A.4 ilustran una *pull request* con sus comprobaciones y el conjunto de ejecuciones de los flujos.

Planificación con Kanban

La planificación del trabajo se gestiona con un tablero Kanban sobre GitHub Projects, con las columnas descritas en la metodología de la introducción (Backlog, Iteración actual, En progreso, En revisión y Hecho) y transiciones parcialmente automatizadas a partir de la apertura de *issues* y *pull requests*. La Figura A.5 muestra el tablero.

The screenshot displays a GitHub pull request interface. At the top, the repository is identified as 'jminac00 / FinSight'. The pull request title is 'refactor(market-data): shared yfinance OHLC fetch in app/core #49'. It shows that the pull request was merged 3 weeks ago, with 2 commits and 3 files changed. The pull request is currently closed.

The main content area shows the pull request details, including a summary, changes, verification, and out of scope items. The summary states: 'Extracts the single-ticker OHLC download (get_price_history) from the vendored technical engine into a shared app/core/market_data module, so the technical engine and the upcoming Deep Learning data layer consume one implementation without coupling the two service modules.' The changes list includes: 'New app/core/market_data.py with get_price_history(ticker, period="3y") — identical behavior and output shape, English docstring/messages, structured logging.', 'technical/engine/utils/data_loader.py imports the shared function instead of defining it (no behavior change).', and 'New tests/core/test_market_data.py: network-free unit tests (Multindex flattening, NaN-close dropping, empty/all-NaN error paths, call arguments).'. The verification section shows: 'uv run pytest → 142 passed, including the technical golden baseline (unchanged → behavior preserved).', 'ruff check and ruff format --check clean.'. The out of scope items list includes: 'DL per-ticker training loader/CLI, DService inference + real API, and the daily retraining scheduler job.'.

The pull request is closed, and the branch 'refactor/shared-market-data' has been deleted. The pull request was successfully merged and closed. The pull request was created by jminac00 and is currently closed.

The pull request details include a summary, changes, verification, and out of scope items. The summary states: 'Extracts the single-ticker OHLC download (get_price_history) from the vendored technical engine into a shared app/core/market_data module, so the technical engine and the upcoming Deep Learning data layer consume one implementation without coupling the two service modules.' The changes list includes: 'New app/core/market_data.py with get_price_history(ticker, period="3y") — identical behavior and output shape, English docstring/messages, structured logging.', 'technical/engine/utils/data_loader.py imports the shared function instead of defining it (no behavior change).', and 'New tests/core/test_market_data.py: network-free unit tests (Multindex flattening, NaN-close dropping, empty/all-NaN error paths, call arguments).'. The verification section shows: 'uv run pytest → 142 passed, including the technical golden baseline (unchanged → behavior preserved).', 'ruff check and ruff format --check clean.'. The out of scope items list includes: 'DL per-ticker training loader/CLI, DService inference + real API, and the daily retraining scheduler job.'.

The pull request is closed, and the branch 'refactor/shared-market-data' has been deleted. The pull request was successfully merged and closed. The pull request was created by jminac00 and is currently closed.

The pull request details include a summary, changes, verification, and out of scope items. The summary states: 'Extracts the single-ticker OHLC download (get_price_history) from the vendored technical engine into a shared app/core/market_data module, so the technical engine and the upcoming Deep Learning data layer consume one implementation without coupling the two service modules.' The changes list includes: 'New app/core/market_data.py with get_price_history(ticker, period="3y") — identical behavior and output shape, English docstring/messages, structured logging.', 'technical/engine/utils/data_loader.py imports the shared function instead of defining it (no behavior change).', and 'New tests/core/test_market_data.py: network-free unit tests (Multindex flattening, NaN-close dropping, empty/all-NaN error paths, call arguments).'. The verification section shows: 'uv run pytest → 142 passed, including the technical golden baseline (unchanged → behavior preserved).', 'ruff check and ruff format --check clean.'. The out of scope items list includes: 'DL per-ticker training loader/CLI, DService inference + real API, and the daily retraining scheduler job.'.

Figura A.3: Pull request con las comprobaciones de integración continua superadas antes de su fusión en main.

The screenshot displays the GitHub Actions interface for the repository `jminac00 / FinSight`. The left sidebar shows the 'Actions' tab with a 'New workflow' button. The main area is titled 'All workflows' and shows 'Showing runs from all workflows'. A search bar is present at the top right of the workflow runs list.

The workflow runs are listed in a table with the following columns: Workflow, Event, Status, Branch, and Actor. The table shows 312 workflow runs. The first few runs are:

- CodeQL** (Workflow: CodeQL #108: Scheduled, Branch: main, Status: Success, Actor: jminac00)
- feat(deep-learning): train missing models on demand in local (#90)** (Workflow: Frontend CI #102: Commit 87f9c28 pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- feat(deep-learning): train missing models on demand in local (#90)** (Workflow: Backend CI #102: Commit 87f9c28 pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- feat(deep-learning): train missing models on demand in local (#90)** (Workflow: CodeQL #107: Commit 87f9c28 pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- chore(frontend): ignore TypeScript incremental build cache (#88)** (Workflow: Backend CI #101: Commit 86856c9 pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- chore(frontend): ignore TypeScript incremental build cache (#88)** (Workflow: Frontend CI #101: Commit 86856c9 pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- chore(frontend): ignore TypeScript incremental build cache (#88)** (Workflow: CodeQL #106: Commit 86856c9 pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- feat(deep-learning): train missing models on demand in local** (Workflow: Frontend CI #100: Pull request #90 opened by jminac00, Branch: feature/di-on-demand-traini..., Status: Success, Actor: jminac00)
- feat(deep-learning): train missing models on demand in local** (Workflow: Backend CI #100: Pull request #90 opened by jminac00, Branch: feature/di-on-demand-traini..., Status: Success, Actor: jminac00)
- feat(deep-learning): train missing models on demand in local** (Workflow: CodeQL #105: Pull request #90 opened by jminac00, Branch: feature/di-on-demand-traini..., Status: Success, Actor: jminac00)
- chore(frontend): ignore TypeScript incremental build cache** (Workflow: CodeQL #104: Pull request #88 opened by jminac00, Branch: chore/ignore-ts-buildinfo..., Status: Success, Actor: jminac00)
- chore(frontend): ignore TypeScript incremental build cache** (Workflow: Frontend CI #99: Pull request #88 opened by jminac00, Branch: chore/ignore-ts-buildinfo..., Status: Success, Actor: jminac00)
- chore(frontend): ignore TypeScript incremental build cache** (Workflow: Backend CI #99: Pull request #88 opened by jminac00, Branch: chore/ignore-ts-buildinfo..., Status: Success, Actor: jminac00)
- Merge pull request #86 from jminac00/fix/relax-ticker-validation-inte...** (Workflow: Frontend CI #98: Commit 56d7d8a pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- Merge pull request #86 from jminac00/fix/relax-ticker-validation-inte...** (Workflow: Backend CI #98: Commit 56d7d8a pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- Merge pull request #86 from jminac00/fix/relax-ticker-validation-inte...** (Workflow: CodeQL #103: Commit 56d7d8a pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- fix(frontend): Accept international ticker symbols in validation** (Workflow: Frontend CI #97: Pull request #86 opened by jminac00, Branch: fix/relax-ticker-validation..., Status: Success, Actor: jminac00)
- fix(frontend): Accept international ticker symbols in validation** (Workflow: CodeQL #102: Pull request #86 opened by jminac00, Branch: fix/relax-ticker-validation..., Status: Success, Actor: jminac00)
- fix(frontend): Accept international ticker symbols in validation** (Workflow: Backend CI #97: Pull request #86 opened by jminac00, Branch: fix/relax-ticker-validation..., Status: Success, Actor: jminac00)
- Report warns on any missing module and shows the company name (...)** (Workflow: Frontend CI #96: Commit 2382c1f pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- Report warns on any missing module and shows the company name (...)** (Workflow: Backend CI #96: Commit 2382c1f pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- Report warns on any missing module and shows the company name (...)** (Workflow: CodeQL #101: Commit 2382c1f pushed by jminac00, Branch: main, Status: Success, Actor: jminac00)
- feat(frontend): Report warns on any missing module and shows the co...** (Workflow: CodeQL #100: Pull request #84 opened by jminac00, Branch: feature/report-missing-mods..., Status: Success, Actor: jminac00)
- feat(frontend): Report warns on any missing module and shows the co...** (Workflow: Backend CI #95: Pull request #84 opened by jminac00, Branch: feature/report-missing-mods..., Status: Success, Actor: jminac00)
- feat(frontend): Report warns on any missing module and shows the co...** (Workflow: Frontend CI #95: Pull request #84 opened by jminac00, Branch: feature/report-missing-mods..., Status: Success, Actor: jminac00)

The table includes pagination controls at the bottom: < Previous 1 2 3 4 5 ... 12 13 Next >.

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Figura A.4: Ejecuciones de los flujos de integración continua en GitHub Actions.

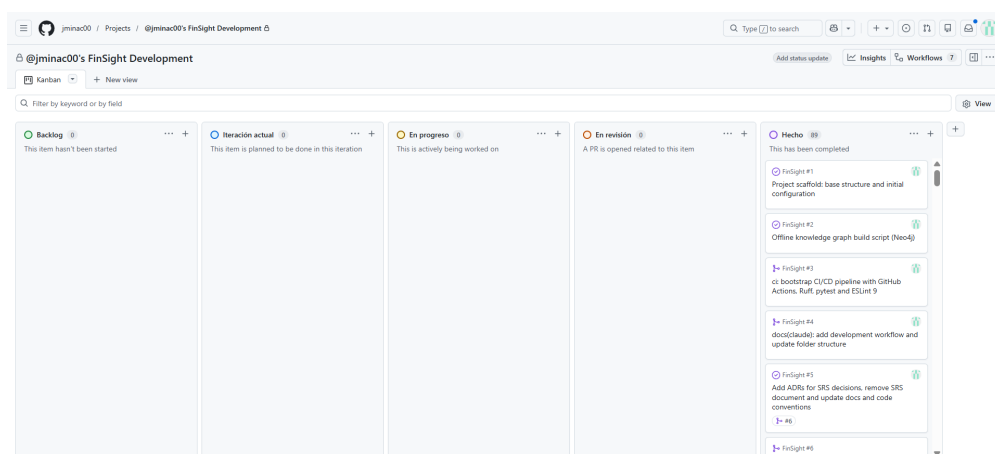


Figura A.5: Tablero Kanban del proyecto terminado en GitHub Projects.

Anexo B

Seguimiento de proyecto fin de grado

A diferencia del capítulo de gestión del proyecto, que plantea una simulación profesional, este anexo recoge el seguimiento real del Trabajo Fin de Grado: con qué medios se ha supervisado el avance y en qué medida la ejecución se ha ajustado a la planificación prevista.

B.1. Forma de seguimiento

El seguimiento se apoyó en las mismas herramientas con las que se desarrolló el proyecto. El tablero Kanban de GitHub Projects, junto con los *issues*, los *commits* y las *pull requests*, funcionó como registro vivo del estado del trabajo en cada momento, de modo que el avance era visible sin necesidad de informes adicionales. Las decisiones de diseño relevantes se documentaron como *Architecture Decision Records*, lo que dejó constancia del porqué de cada elección y facilitó retomar el hilo tras cada pausa.

El trabajo se coordinó con el tutor mediante reuniones de seguimiento, en las que se revisaba el estado del proyecto y se orientaban las siguientes iteraciones. Además, al tratarse de un trabajo interdisciplinar, hubo una coordinación específica con el colaborador del Grado en Finanzas para la definición y la integración de los análisis técnico y fundamental. La Figura B.1 muestra el estado de los hitos en el repositorio.

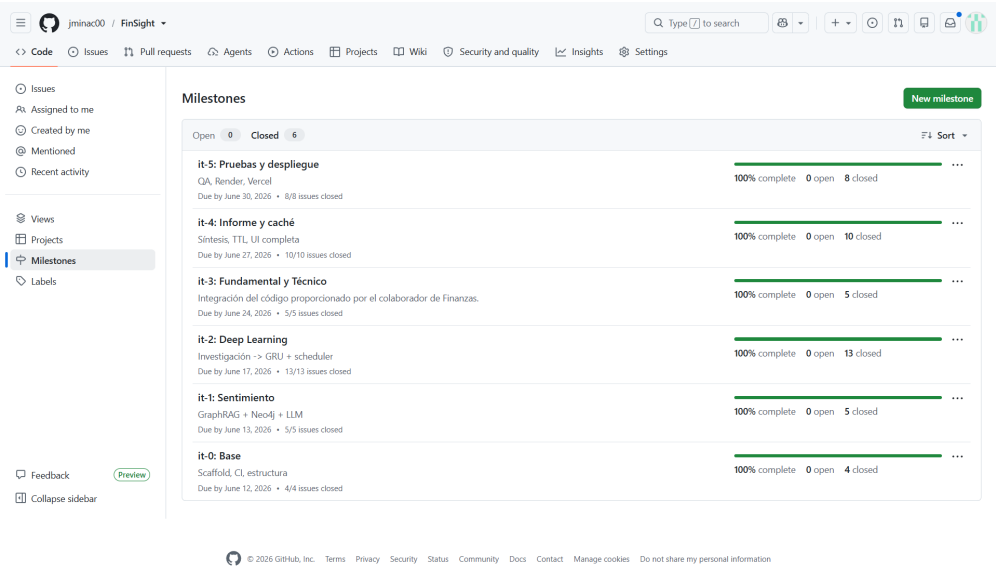


Figura B.1: Estado de los hitos del proyecto en GitHub, empleado como registro de seguimiento.

B.2. Planificación inicial

La planificación inicial organizó el desarrollo en seis iteraciones, materializadas como *milestones* en GitHub y precedidas por la fase de investigación y del prototipo previo. Cada iteración fijó una fecha objetivo, recogida en la Tabla B.1, y agrupaba los *issues* de sus tareas. La Figura ?? representa este plan como diagrama de Gantt.

Iteración	Objetivo	Fecha prevista
it-0	Base: esqueleto, integración continua y estructura	12 jun 2026
it-1	Sentimiento: GraphRAG, Neo4j y servicio LLM	13 jun 2026
it-2	Deep learning: investigación, GRU y reentrenamiento	17 jun 2026
it-3	Análisis fundamental y técnico: integración	24 jun 2026
it-4	Informe consolidado, cachés e interfaz completa	27 jun 2026
it-5	Pruebas de calidad y despliegue	30 jun 2026

Tabla B.1: Fechas objetivo de las seis iteraciones según la planificación inicial.

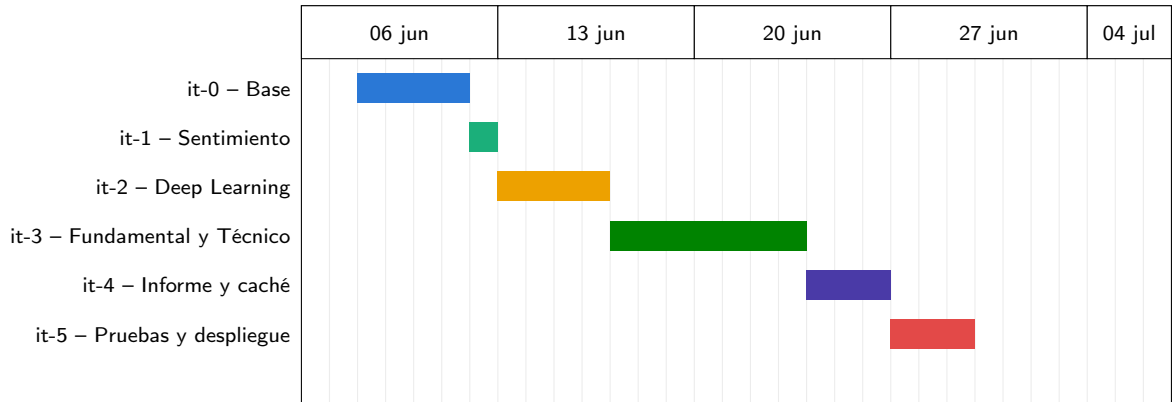


Figura B.2: Planificación inicial del proyecto como diagrama de Gantt.

B.3. Planificación final

La ejecución siguió en lo esencial el plan previsto. El grueso del desarrollo se concentró en la segunda mitad de junio, y las cuatro primeras iteraciones se cerraron dentro de sus fechas objetivo. Las dos últimas, la del informe consolidado con la interfaz completa y la de pruebas y despliegue, se prolongaron unos días y se cerraron a comienzos de julio, junto con la finalización del módulo de aprendizaje profundo. La desviación, de pocos días sobre un plazo ya ajustado, se debió al alcance de ese módulo y al pulido final de la interfaz y del despliegue. La Figura ?? recoge el cronograma real y permite compararlo con el planificado.

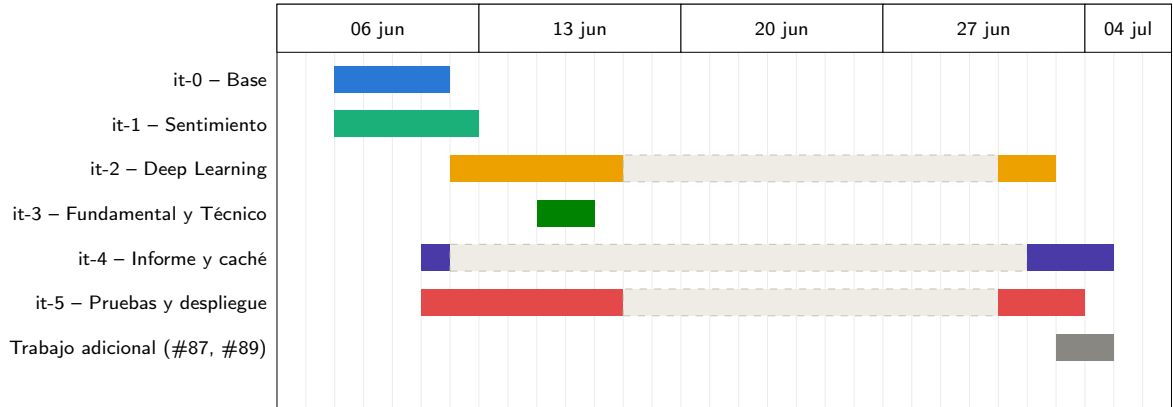


Figura B.3: Cronograma real del proyecto, comparable con la planificación inicial.

En conjunto, el proyecto se completó dentro del margen de la convocatoria de julio de 2026 y cumplió los objetivos previstos, con una desviación temporal reducida y concentrada en las iteraciones finales.

Anexo C

Manual de usuario

FinSight es una aplicación web que analiza acciones, en especial del mercado estadounidense (Nasdaq) y resume el resultado en un informe en español pensado para cualquier persona, tenga o no formación financiera. Este manual explica cómo usarla: cómo buscar un valor, cómo se genera el informe y cómo interpretar cada una de sus partes. Las pantallas a las que se hace referencia se recogen en el apartado «El producto del desarrollo».

C.1. Acceso y requisitos

Para usar FinSight solo hace falta un navegador web actual (Chrome, Firefox, Edge o Safari en sus versiones recientes) y conexión a internet; no se instala nada. La aplicación se abre desde su dirección web y funciona igual en ordenador, tableta o móvil, ya que la interfaz se adapta al tamaño de la pantalla. La primera petición del día puede tardar algo más de lo habitual: el servidor gratuito en el que se aloja entra en reposo tras 15 minutos de inactividad y necesita unos segundos para despertar. Si la primera carga se demora, basta con esperar y reintentar.

Conviene tener presente el carácter de la herramienta desde el principio. FinSight ofrece información orientativa generada de forma automática; no es asesoramiento financiero ni una recomendación de compra o venta. Este aviso acompaña de forma permanente a la interfaz y al propio informe.

C.2. Buscar un valor y generar el informe

La página de inicio (Fig. 3.7) muestra un buscador central. Para generar un informe se escribe el nombre de la empresa o su símbolo bursátil, por ejemplo «Apple» o «AAPL». A medida que se teclea, el buscador propone coincidencias en una lista desplegable (Fig. 3.8), que se pueden elegir con el ratón o con las flechas del teclado. Al seleccionar una sugerencia o pulsar el botón «Analizar», la aplicación abre el informe del valor elegido.

El símbolo debe tener entre dos y cinco letras, y la herramienta lo convierte a mayúsculas automáticamente. Los modelos de predicción de tendencia están preparados para acciones del Nasdaq; para otros valores el resto de análisis sigue estando disponible, como se explica más abajo. Tras lanzar el análisis, la aplicación trabaja unos segundos mientras consulta las noticias, ejecuta el modelo de predicción y calcula los indicadores, y después presenta el informe completo en una sola página.

C.3. Cómo leer el informe

El informe (Fig. 3.9) empieza con cuatro tarjetas de resumen que dan una lectura rápida: el sentimiento de las noticias, la tendencia prevista, la puntuación fundamental y la señal técnica. Debajo, cada uno de esos apartados se desarrolla con detalle. Las secciones elaboradas por el modelo de lenguaje llevan una marca de «Generado por IA» para que quede claro qué texto procede de una síntesis automática.

El análisis de sentimiento indica si el tono de las noticias recientes sobre la empresa es positivo, neutral o negativo. Lo acompaña una puntuación entre -1 (muy negativo) y 1 (muy positivo), un porcentaje de confianza y la lista de noticias más influyentes, cada una con enlace a su fuente original para poder ampliarla.

La predicción de tendencia resume hacia dónde apunta el precio en un horizonte de diez sesiones bursátiles, con una etiqueta alcista, bajista o neutral. Muestra el retorno estimado en ese plazo, el precio actual y el precio que se derivaría de esa estimación. La etiqueta neutral aparece cuando el movimiento previsto es pequeño, para no presentar como señal lo que es indistinguible del ruido del mercado. Junto a la predicción se

ofrecen tres medidas de calidad del propio modelo: el RMSE y el MAE, que expresan el error típico de las predicciones y que, cuanto más bajos, mejor; y el acierto direccional, el porcentaje de veces que el modelo acierta la dirección del movimiento. Estas cifras ayudan a juzgar cuánta confianza merece la predicción, que por su naturaleza es limitada.

El análisis fundamental interpreta la salud financiera de la empresa, como la rentabilidad, el endeudamiento o los márgenes, y la resume en una puntuación sobre diez acompañada de una explicación en lenguaje sencillo; un desplegable permite ver los ratios en detalle. El análisis técnico hace lo propio con el comportamiento del precio y el volumen, y resume en una señal lo que indican herramientas como el RSI, el MACD o las medias móviles, con su detalle también desplegable.

Dentro de esos desplegables de detalle, junto a cada métrica aparece un pequeño botón con una «i» de información (Fig. 3.10). Al pulsarlo se despliega una breve explicación de lo que significa ese campo, redactada para quien no está familiarizado con la terminología financiera, y al volver a pulsarlo se oculta. Este mismo botón acompaña también a las métricas de las secciones principales, como la puntuación y la confianza del sentimiento o las medidas de calidad del modelo de predicción, de modo que en ningún momento hace falta salir del informe para entender qué representa cada dato.

El informe se cierra con una conclusión global que combina los cuatro análisis en una valoración de conjunto, y con el aviso legal que recuerda que todo el contenido es informativo. En la cabecera del informe, el botón «Actualizar» vuelve a generar el análisis con los datos más recientes, y el enlace «Volver al inicio» regresa al buscador.

C.4. Cuando falta algún análisis

Si para el valor consultado no hay modelo de predicción disponible, la aplicación no deja de funcionar: muestra el resto de análisis y avisa de lo que falta con un mensaje de «soporte parcial», dejando marcada como no disponible únicamente la parte afectada (Fig. 3.11). Así, el sentimiento, el análisis fundamental y el técnico siguen siendo útiles aunque no exista predicción de tendencia para ese valor.

Si en cambio se produce un problema de conexión, el informe muestra un mensaje de error con un botón para reintentar. Suele deberse al arranque del servidor tras un periodo de inactividad, de modo que reintentar pasados unos segundos resuelve el caso más habitual.

C.5. Tema claro y oscuro

Desde el icono de la cabecera se alterna entre el tema claro y el oscuro (Fig. 3.14), para adaptarse a las preferencias de cada persona y a las condiciones de luz. Si se han aceptado las cookies funcionales, la aplicación recuerda la elección entre visitas.

C.6. Privacidad y cookies

En la primera visita, FinSight muestra un aviso de cookies (Fig. 3.12) con la posibilidad de aceptarlas, rechazarlas o configurarlas. El diálogo de preferencias (Fig. 3.13) permite decidir por categorías: las estrictamente necesarias, que hacen funcionar el sitio y no se pueden desactivar, y las funcionales y analíticas, que se activan solo con permiso. Esta elección se puede cambiar en cualquier momento desde el enlace «Preferencias de cookies» del pie de página.

El pie de página enlaza también con la política de privacidad (Fig. 3.15) y con los términos de uso (Fig. 3.16), donde se detallan el tratamiento de los datos y las condiciones del servicio. Si se escribe una dirección que no existe, la aplicación muestra una página de aviso con un enlace para volver al inicio (Fig. 3.17).

C.7. Preguntas frecuentes

¿Por qué el primer análisis tarda más? El servidor gratuito entra en reposo tras un rato sin uso y tarda unos segundos en arrancar. A partir de la primera petición, el resto responde con normalidad.

¿Por qué no hay predicción de tendencia para algunos valores? El modelo de predicción se ha validado sobre acciones del Nasdaq. Para valores fuera de ese ámbito, la aplicación muestra igualmente el análisis de sentimiento, el fundamental y el técnico.

¿Se guardan mis búsquedas? Solo si se aceptan las cookies funcionales, en cuyo caso la aplicación recuerda en el propio navegador el último valor analizado para facilitar la siguiente visita. Sin ese permiso no se conserva esa información.

¿Los datos son en tiempo real? No. El análisis se basa en datos de cierre de mercado, no en cotizaciones intradía, por lo que refleja la situación al último cierre disponible.

¿Puedo basar mis inversiones en el informe? No conviene hacerlo sin más. FinSight es una herramienta informativa que ayuda a entender una acción, no un asesor financiero. Para decisiones de inversión es recomendable contrastar con un profesional cualificado.